

Journey QA Thomas

Panduan Lengkap Menjadi Quality Assurance Engineer — dari Requirement Analysis hingga Automation

Penulis: QA Thomas

Daftar Isi

1. Chapter 0: Selamat Datang — Apa Itu QA?
 2. Chapter 01: Requirement Analysis — Quality Gate Pertama
 3. Chapter 02: Test Design — Strategi, Estimasi, dan Test Case
 4. Chapter 03: Test Execution — Smoke, Functional, Exploratory, Regression
 5. Chapter 04: Bug Reporting — Severity, Template, dan Eskalasi
 6. Chapter 05: Reporting & Documentation — UAT, Release Notes, Knowledge Base
 7. Chapter 06: Stakeholder Engagement — Demo, Meeting, UAT Coordination
 8. Chapter 07: Automation Testing dengan Playwright
 9. Chapter 08: Tools & Workflow — Notion, Slack, Drive, Handoff
 10. Chapter 09: Penutup — Resources, Templates, dan Next Steps
-

Chapter 0: Selamat Datang — Apa Itu QA?

Saya pernah ketemu seorang programmer yang bilang, "QA itu yang klik-klik tombol kan? Kerjanya nyari tombol mana yang error?"

Kalau Anda mengangguk membaca kalimat itu, selamat — Anda baru saja menemukan alasan kenapa buku ini dibuat. Karena selama Anda masih menganggap QA sebagai "*tester yang klik-klik tombol*", Anda akan terus melepas bug ke production, terus tertekan tiap mau release, dan terus merasa tim engineering Anda lambat padahal masalahnya bukan di sana.

QA bukan pekerjaan klik-klik. QA adalah **quality gate** — gerbang yang menentukan apakah sebuah fitur layak sampai ke tangan user atau tidak. Buku ini menulis ulang pekerjaan itu dari nol, dengan satu prinsip pegangan yang sederhana tapi sering dilupakan.

Kenapa Bug yang Lolos ke Production Itu Mahal

Sebelum kita bicara teknis, mari sepakati dulu kenapa peran ini penting. Industri sudah lama punya angka klasik soal biaya bug:

Bug ditemukan di tahap	Biaya relatif untuk diperbaiki
Requirement / desain	1x
Coding	5x
Testing	10x
Production	30-100x

Angkanya bisa diperdebatkan, tapi intinya: **semakin telat sebuah bug ditemukan, semakin mahal biaya memperbaikinya**. Bukan cuma biaya engineering — termasuk juga reputasi, churn user, dan jam tidur tim oncall jam 3 pagi.

Pekerjaan QA pada dasarnya adalah **menggeser deteksi bug sejauh mungkin ke kiri** (shift-left). Jangan ketemu bug di production, ketemu di staging. Jangan ketemu di staging, ketemu di code review. Jangan ketemu di code review, ketemu di PRD. Buku ini akan mengajarkan Anda satu-satu cara melakukan itu.

Prime Directive — Mantra yang Harus Anda Hafal

Sebelum lanjut, hafalkan kalimat ini:

"Komunikasi untuk mengecek apakah flow sudah sesuai dengan criteria user story."

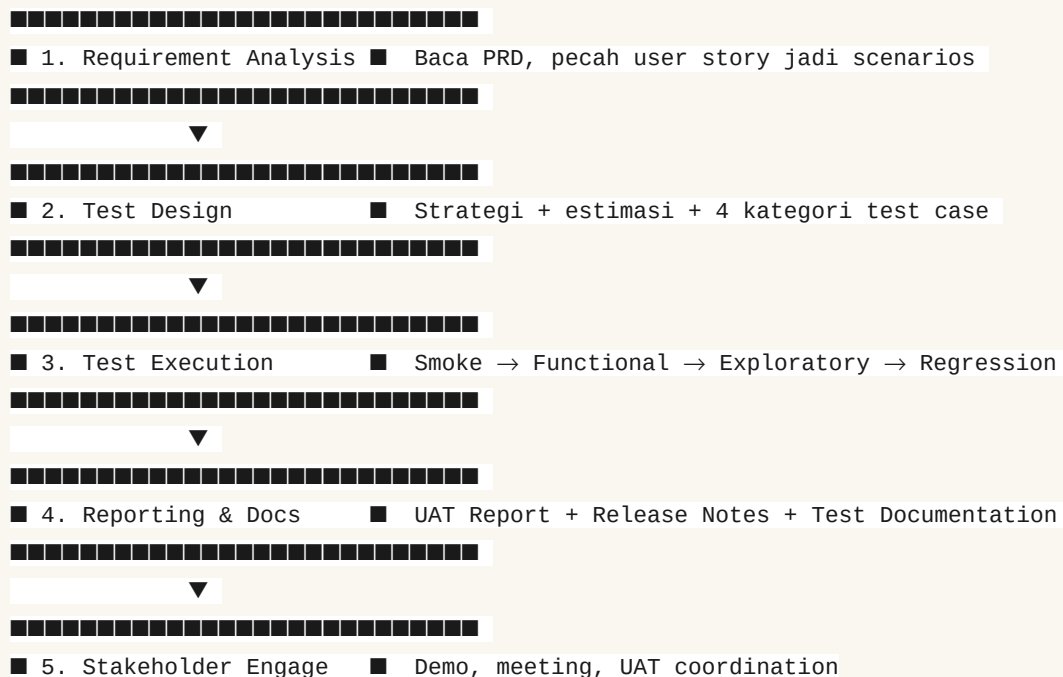
Itu prime directive seorang QA. Sederhana, tapi konsekuensinya besar:

- Kalau code berjalan tapi **menyimpang dari acceptance criteria** → tetap bug. File bug-nya.
- Kalau implementasi "secara teknis benar" tapi **user story bilang lain** → tetap bug. File bug-nya.
- Kalau "tidak ada error di console" tapi **expected behavior tidak terjadi** → tetap bug. File bug-nya.

QA bukan menilai apakah code-nya benar. **QA menilai apakah code-nya menjawab kebutuhan yang ditulis di user story.** Dua hal yang berbeda. Pegang ini erat-erat.

Anatomi Pekerjaan: 5 Sub-Flow QA

Pekerjaan QA bukan satu aktivitas, tapi lima aktivitas yang saling berurutan. Kita akan bedah satu per satu di chapter berikutnya.





Sub-flow ini bukan teori. Ini urutan kerja harian seorang QA Engineer di tim modern. Kalau Anda melompati salah satu, hampir pasti akan ada bug yang lolos atau release yang macet.

1. Requirement Analysis (Chapter 1)

Pintu masuk pertama. Sebelum engineering nulis satu baris kode, QA harus sudah membaca PRD dan menemukan *ambiguity* — kalimat-kalimat seperti "*sistem menangani input invalid secara baik*" yang tidak bisa dites. Setelah itu memecah tiap user story jadi daftar skenario (happy path, edge case, failure mode) yang akan jadi bahan test case.

2. Test Design (Chapter 2)

Setelah skenario disepakati, QA menulis **test strategy** (apa yang dites, bagaimana, dengan apa, kapan) dan **test case** dalam empat kategori wajib: positive, negative, edge, dan data preparation. Plus — yang sering dilupakan — **estimasi waktu**. Tanpa estimasi waktu, Anda akan terus dipaksa selesai dalam "secepatnya" yang berarti tidak akan pernah selesai.

3. Test Execution (Chapter 3)

Eksekusi mengikuti urutan fail-fast: smoke testing dulu (5 menit, kalau gagal langsung stop), baru functional testing per user story, lalu exploratory untuk hunting edge case, dan terakhir regression untuk memastikan tidak ada feature lain yang rusak. Tiap cycle menghasilkan **Test Execution Report**.

4. Reporting & Documentation (Chapter 5)

Empat jenis dokumen yang jadi tanggung jawab QA: **UAT Report** untuk stakeholder, **Release Notes** untuk user, **Test Documentation** untuk knowledge base internal, dan **Test Execution Report** per cycle. Dokumen-dokumen ini yang membuat pekerjaan QA bisa diaudit dan diwariskan ke tim berikutnya.

5. Stakeholder Engagement (Chapter 6)

QA tidak kerja sendirian. Ada demo ke stakeholder, ada meeting standup-sprint planning-retro, ada koordinasi UAT dengan user beneran. Bagian ini sering disepelekan — padahal QA yang

tidak hadir di sprint planning akan terus dapat scope yang tidak realistis.

Severity Rubric — Bahasa Bersama Tim

Salah satu kontribusi paling berharga seorang QA adalah *bahasa bersama* untuk membicarakan tingkat keparahan bug. Kita pakai 4 level:

Level	Definisi	Contoh
P0	Production down, data loss, security breach	Login total tidak bisa, payment gagal semua user
P1	Core feature rusak, tidak ada workaround	Search return 500, signup intermittent
P2	Feature impaired tapi ada workaround	Urutan sort salah, typo di UI
P3	Cosmetic, minor inconvenience	Pixel misalign, edge timing

Severity ini menentukan **routing eskalasi**: - P0/P1 → file di Notion Bugs DB + post ke channel `#incidents` Slack + DM ke Tech Lead langsung. - P2 → file di Notion + post ke `#qa`. - P3 → file di Notion saja, akan muncul di backlog review berikutnya.

Detail dan template lengkap dibahas di Chapter 4.

Mindset yang Membedakan QA Biasa dengan QA Hebat

Sebelum kita masuk ke teknis, saya mau Anda menyerap empat mindset ini lebih dulu. Tanpa ini, semua teknik di chapter berikutnya hanya akan jadi checklist mati.

1. Adversarial, bukan defensive. QA yang baik berpikir "*bagaimana saya membuat fitur ini gagal?*" bukan "*bagaimana saya membuktikan fitur ini jalan?*". Anggap diri Anda lawan dari fitur tersebut.

2. Trace ke user story, selalu. Setiap test case yang Anda tulis harus bisa Anda kaitkan ke user story tertentu. Kalau tidak bisa, Anda sedang tes hal yang salah.

3. File bug sambil jalan, jangan di-batch. Anti-pattern paling umum: QA menumpuk semua bug di akhir cycle lalu dump 30 ticket sekaligus. Itu menyebalkan untuk semua orang. File begitu Anda menemukan, satu per satu.

4. Anda adalah gerbang, bukan tukang stempel. Kalau acceptance criteria tidak terpenuhi, jangan approve walaupun ditekan "*sudah deadline*". Tugas Anda menjaga gerbang. Stempel kosong tidak punya nilai.

Apa yang Akan Anda Pelajari di Buku Ini

Sembilan chapter berikutnya akan mengisi tiap sub-flow di atas dengan detail praktis:

- **Chapter 1:** Review PRD step-by-step + decomposing user story jadi scenarios.
- **Chapter 2:** Template Test Strategy, estimasi waktu, 4 kategori test case dengan contoh kode.
- **Chapter 3:** Urutan eksekusi yang benar + format Test Execution Report.
- **Chapter 4:** Bug Reporting — anatomi bug ticket yang baik + flow eskalasi.
- **Chapter 5:** Empat dokumen QA + konvensi distribusinya.
- **Chapter 6:** Demo yang tidak meledak, meeting yang efektif, UAT yang terkoordinasi.
- **Chapter 7:** Automation Testing dengan Playwright — setup, Page Object Model, contoh nyata.
- **Chapter 8:** Tooling lengkap — Notion, Slack, Drive, dan workflow handoff antar role.
- **Chapter 9:** Penutup + 10-pertanyaan self-assessment + next step untuk 7 hari pertama.

Tiap chapter punya **rangkuman** dan **checklist actionable** di akhir. Jangan dilewati.

Rangkuman

- **QA bukan klik-klik tombol.** QA adalah quality gate yang memastikan flow sesuai dengan acceptance criteria user story.
- **Prime directive:** *"Komunikasi untuk mengecek apakah flow sudah sesuai dengan criteria user story."* Kalau menyimpang dari criteria, walaupun code-nya jalan, itu bug.
- **Bug ditemukan lebih awal jauh lebih murah.** Tugas QA adalah men-shift-left deteksi bug — idealnya sampai ke tahap review PRD.
- **5 sub-flow** yang berurutan: Requirement Analysis → Test Design → Test Execution → Reporting & Documentation → Stakeholder Engagement.
- **Severity P0-P3** menentukan urgensi dan routing eskalasi bug.
- Empat mindset penentu: adversarial, trace-to-story, file-as-you-find, gate-not-stamp.

Checklist Actionable

Sebelum lanjut ke Chapter 1, pastikan Anda:

- [] Menghafal prime directive di luar kepala.
- [] Bisa menyebut 5 sub-flow QA tanpa lihat catatan.
- [] Bisa membedakan P0, P1, P2, P3 dengan contoh kasus.
- [] Setuju bahwa "code jalan tapi menyimpang dari acceptance criteria" = bug.
- [] Mengidentifikasi minimal 1 fitur di project Anda saat ini yang akan Anda jadikan "studi kasus" sambil membaca buku ini.
- [] Punya akses ke PRD, Notion (atau task tracker lain), Slack workspace tim Anda.
- [] Siap "shift-left" — datang ke meeting PRD review berikutnya, bukan menunggu fitur jadi.

Kalau semua check terisi, kita siap masuk ke quality gate pertama: **membaca PRD dengan mata QA**. Lanjut ke Chapter 1.

Chapter 01: Requirement Analysis

— Quality Gate Pertama

Selamat datang di babak pertama perjalanan Anda sebagai Quality Assurance Engineer! Jika Anda membayangkan pekerjaan QA hanya tentang menemukan *bug* di akhir siklus pengembangan, siapkan diri untuk sebuah kejutan. Peran QA modern jauh lebih strategis dan proaktif. Gerbang kualitas (Quality Gate) pertama, dan arguably yang paling krusial, dimulai jauh sebelum baris kode pertama ditulis: yaitu pada tahap *Requirement Analysis*.

Bayangkan ini: sebuah tim *engineering* menghabiskan satu minggu penuh untuk mengembangkan sebuah fitur berdasarkan dokumen yang ambigu. Saat fitur tersebut selesai, Anda sebagai QA menemukan bahwa ada bagian yang tidak jelas, atau bahkan bertentangan dengan ekspektasi pengguna. Apa yang terjadi? Fitur harus dirombak, waktu terbuang, *deadline* terancam, dan semangat tim menurun. Ini adalah skenario mimpi buruk yang bisa dihindari jika QA terlibat sejak awal.

Dalam bab ini, kita akan menyelami bagaimana Anda, sebagai QA Engineer, dapat menjadi penjaga kualitas yang proaktif. Kita akan belajar cara membaca Product Requirement Document (PRD) secara kritis, mengidentifikasi celah dan ambiguitas, serta memecah *user story* menjadi skenario-skenario yang siap diuji. Ini adalah fondasi penting yang akan menghemat banyak waktu, tenaga, dan frustrasi di kemudian hari. Mari kita mulai menjadi Quality Gate pertama yang tangguh!

Memahami Peran QA dalam Requirement Analysis

Sebagai Quality Assurance Engineer, tugas Anda bukan hanya memastikan produk berfungsi sesuai spesifikasi, tetapi juga memastikan spesifikasi itu sendiri sudah benar, lengkap, dan jelas. Keterlibatan di tahap analisis kebutuhan adalah investasi waktu yang sangat berharga. Semakin dini Anda menemukan potensi masalah atau ambiguitas, semakin murah biaya perbaikannya.

Mengapa ini disebut "Quality Gate Pertama"? Karena di sinilah Anda memiliki kesempatan emas untuk: 1. **Mencegah *bug* sejak awal:** Dengan mengidentifikasi persyaratan yang tidak jelas atau hilang, Anda mencegah *developer* membangun sesuatu yang salah atau tidak lengkap. 2.

Membangun pemahaman bersama: Memastikan semua pihak (Product Manager, Designer, Developer, QA) memiliki pemahaman yang sama tentang apa yang akan dibangun. 3.

Mempercepat siklus pengembangan: Mengurangi kebutuhan untuk pengerjaan ulang (*rework*) di tahap akhir, yang pada akhirnya mempercepat waktu peluncuran produk. 4. **Meningkatkan kualitas secara keseluruhan:** Dengan fondasi persyaratan yang kuat, proses *development* dan *testing* selanjutnya akan jauh lebih efisien dan efektif.

Mari kita lihat bagaimana Anda bisa menjalankan peran krusial ini.

Langkah 1: Mereview PRD — Gerbang Kualitas yang Tidak Boleh Terlewat

Product Requirement Document (PRD) adalah dokumen kunci yang menjelaskan apa yang akan dibangun, mengapa dibangun, dan bagaimana fitur tersebut akan berfungsi dari perspektif pengguna. Bagi QA, PRD adalah peta jalan pertama Anda. Mereview PRD bukan sekadar membaca, tetapi menganalisis secara kritis untuk mencari celah, ambiguitas, dan potensi masalah.

Kapan Anda harus mereview PRD? Jawabannya jelas: **SEBELUM tim *engineering* mulai menulis kode.** Ini adalah mantra yang harus selalu Anda ingat.

Checklist Review PRD

Berikut adalah daftar periksa yang bisa Anda gunakan untuk setiap PRD yang Anda review. Ini adalah 8 item penting yang akan membantu Anda mengidentifikasi masalah potensial sedini mungkin:

[] Setiap *user story* memiliki kriteria penerimaan (**Acceptance Criteria**) dalam format **Given/When/Then**.

- **Mengapa penting:** Format Given/When/Then (sering disebut Gherkin Syntax) adalah standar dalam Behavior-Driven Development (BDD). Ini membantu mendefinisikan perilaku sistem yang diharapkan dengan sangat jelas.

- **Yang dicari QA:** Pastikan setiap *user story* memiliki kriteria yang spesifik, terukur, dan dapat diuji. Jika hanya ada deskripsi naratif, mintalah PM untuk memecahnya menjadi Given/When/Then.

Contoh:

- **Buruk:** "Pengguna bisa login." (Terlalu umum)

Baik:

- Given pengguna berada di halaman login
- When pengguna memasukkan email dan password yang benar
- And pengguna menekan tombol "Login"
- Then pengguna diarahkan ke halaman *dashboard*

[] **Persyaratan fungsional (*Functional Requirements*) bersifat dapat diuji (setiap FR dapat dipetakan ke setidaknya satu *test case*).**

- **Mengapa penting:** Persyaratan yang tidak dapat diuji berarti Anda tidak tahu kapan fitur itu "selesai" atau "bekerja dengan benar."
- **Yang dicari QA:** Baca setiap FR dan tanyakan pada diri sendiri: "Bagaimana cara saya menguji ini?" Jika Anda kesulitan membayangkan *test case* untuk suatu FR, itu adalah pertanda ada ambiguitas atau kurangnya detail.

[] **Persyaratan non-fungsional (*Non-functional Requirements*) memiliki target yang terukur.**

- **Mengapa penting:** NFR seperti kecepatan, keamanan, dan skalabilitas sangat penting bagi pengalaman pengguna, tetapi seringkali ditulis secara samar. Kata-kata seperti "cepat," "aman," atau "mudah digunakan" tidak cukup.
- **Yang dicari QA:** Pastikan ada angka atau metrik yang jelas.

Contoh:

- **Buruk:** "Aplikasi harus cepat."
- **Baik:** "Waktu pemuatan halaman *dashboard* harus kurang dari 2 detik untuk 90% pengguna di koneksi 4G."
- **Buruk:** "Data pengguna harus aman."
- **Baik:** "Semua data pengguna harus dienkripsi saat transit (TLS 1.2+) dan saat disimpan (AES-256)."

[] **Edge cases** (kasus-kasus batas/ekstrem) eksplisit dan mencakup skenario seperti kegagalan jaringan, status parsial, pengguna bersamaan, *input* kosong/null, *input* yang salah format.

- **Mengapa penting:** Aplikasi yang stabil tidak hanya berfungsi di "jalan lurus" (*happy path*), tetapi juga menangani situasi yang tidak terduga. *Edge cases* adalah sumber *bug* yang paling umum.

Yang dicari QA: Apakah PRD membahas apa yang terjadi jika:

- Koneksi internet terputus di tengah proses?
- Server mengembalikan error 500?
- Pengguna mencoba mengirim data kosong?
- Dua pengguna mencoba memperbarui data yang sama secara bersamaan?
- Format *input* tidak sesuai (misalnya, email tanpa '@' atau domain)?
- Data yang diharapkan tidak ada (*null state*)?

[] **Diagram alur** (*Flow Diagram*) ada dan cocok dengan narasi.

- **Mengapa penting:** Diagram visual seringkali lebih mudah dipahami daripada teks panjang. Ini membantu memvisualisasikan perjalanan pengguna dan interaksi sistem.

• **Yang dicari QA:** Bandingkan diagram alur dengan deskripsi tekstual. Apakah ada perbedaan? Apakah ada langkah yang hilang di salah satu sisi? Diagram juga bisa membantu mengidentifikasi *edge cases* yang mungkin terlewat.

[] **Bagian "Di luar cakupan"** (*Out-of-scope*) jelas.

- **Mengapa penting:** Menentukan apa yang *tidak* akan dibangun sama pentingnya dengan apa yang akan dibangun. Ini mencegah perdebatan lingkup (*scope arguments*) di tengah *sprint*.

• **Yang dicari QA:** Apakah ada fitur terkait yang mungkin diasumsikan oleh *stakeholder* atau pengguna tetapi tidak akan disertakan dalam rilis ini? Memiliki bagian *out-of-scope* yang jelas membantu mengelola ekspektasi.

[] **Daftar event analitik** ada (Anda akan memverifikasi event ini terpicu).

- **Mengapa penting:** Banyak fitur modern membutuhkan pelacakan penggunaan untuk analisis data dan pengambilan keputusan bisnis.

- **Yang dicari QA:** Apakah PRD menyebutkan *event* apa saja yang perlu dikirim ke sistem analitik (misalnya, Google Analytics, Mixpanel, Amplitude) saat pengguna melakukan tindakan tertentu? Anda perlu tahu ini untuk menguji apakah *event* tersebut terpicu dengan benar.

- **Contoh:** "Ketika pengguna mengklik tombol 'Beli Sekarang', *event purchase_initiated* harus terpicu dengan parameter *product_id* dan *quantity*."

[] Bagian pemeriksaan pra-sprint (*Pre-sprint check*) sudah tersedia.

- **Mengapa penting:** Ini adalah daftar item yang harus disiapkan atau diverifikasi sebelum *sprint* dimulai.

- **Yang dicari QA:** Apakah ada dependensi eksternal, konfigurasi *environment*, atau data uji yang perlu disiapkan? Misalnya, "API pembayaran pihak ketiga sudah terintegrasi di *staging*."

Cara Memberikan Feedback Konstruktif di PRD

Menemukan celah atau ambiguitas adalah satu hal, mengomunikasikannya secara efektif adalah hal lain. Tujuan Anda bukan hanya menunjukkan kesalahan, tetapi membantu meningkatkan kualitas dokumen dan, pada akhirnya, produk.

Langkah-langkah saat menemukan celah: 1. **Berikan komentar spesifik pada tiket PRD (misalnya di Notion, Jira, Confluence) dengan pertanyaan yang jelas.** Hindari komentar umum seperti "Ini tidak jelas." 2. **Tag Product Manager (PM) yang bersangkutan.** Ini memastikan PM segera melihat dan menindaklanjuti komentar Anda. 3. **Blokir serah terima (*handoff*) ke Tech Lead atau tim *engineering* hingga masalah terselesaikan.** Ini adalah langkah krusial untuk menegaskan bahwa PRD belum siap untuk pengembangan.

Contoh Kasus: 'Handle Invalid Input Gracefully'

Misalkan Anda menemukan *functional requirement* seperti ini di PRD: "Sistem harus menangani *input* yang tidak valid dengan anggun (*gracefully*)."

Ini adalah contoh klasik dari persyaratan yang ambigu. Apa arti "dengan anggun" bagi Anda, bagi PM, dan bagi *developer*? Bisa jadi sangat berbeda.

Berikut adalah contoh dialog dan cara Anda memberikan komentar yang konstruktif:

Dialog Internal (Pikiran QA Thomas): * "Dengan anggun? Apa maksudnya? Menampilkan pesan error? Menolak *input* secara diam-diam? Mengarahkan ke halaman lain? Ini bisa diinterpretasikan banyak cara, dan *developer* bisa saja memilih implementasi yang tidak sesuai harapan PM atau pengguna." * "Saya perlu meminta PM untuk mendefinisikan perilaku spesifiknya."

Komentar di PRD (Notion/Jira):

****Komentar QA (Thomas):****

****Ref:**** FR-3: "Sistem harus menangani *input* yang tidak valid dengan anggun."

****Pertanyaan:**** Mohon diperjelas definisi "menangani dengan anggun" untuk *input* yang tidak valid.

Apakah yang dimaksud adalah:

1. Menampilkan pesan error yang jelas dan informatif kepada pengguna (misal: "Format email tidak valid")
2. Menolak *input* secara diam-diam tanpa *feedback* kepada pengguna?
3. Mengarahkan pengguna ke halaman *error* umum?
4. Melakukan koreksi otomatis pada *input* jika memungkinkan?

Mohon diberikan contoh spesifik untuk beberapa skenario *input* tidak valid (misalnya, email tanpa '@')

Tanpa klarifikasi ini, ada risiko implementasi yang berbeda dari ekspektasi dan memerlukan *rework* di

****@PM_Thomas**** - Mohon bantuannya untuk klarifikasi ini sebelum *handoff* ke tim *engineering*.

Penjelasan Komentar: * **Spesifik:** Merujuk langsung ke FR yang bermasalah. * **Bertanya, bukan menuduh:** Menggunakan pertanyaan terbuka untuk mendorong PM memberikan detail. * **Memberikan opsi:** Memberikan beberapa kemungkinan interpretasi membantu PM memikirkan skenario dan memilih yang paling tepat. * **Menjelaskan dampak:** Mengingatkan tentang risiko jika tidak diklarifikasi (implementasi salah, *rework*). * **Panggilan tindakan:** Men-tag PM secara langsung. * **Menegaskan pentingnya waktu:** "sebelum *handoff* ke tim *engineering*."

Dengan pendekatan ini, Anda tidak hanya menemukan masalah tetapi juga memfasilitasi solusi, menjadikan diri Anda sebagai mitra strategis bagi tim produk.

Langkah 2: Menganalisis User Story dan Memecahnya menjadi Skenario

Setelah PRD Anda melewati tahap review dan semua pertanyaan telah terjawab, langkah selanjutnya adalah menganalisis setiap *user story* dan memecahnya menjadi daftar skenario yang dapat diuji. Ini adalah jembatan antara persyaratan tingkat tinggi dan *test case* yang akan Anda tulis nanti.

Apa itu User Story? *User story* adalah deskripsi singkat dan sederhana tentang sebuah fitur dari perspektif pengguna akhir. Format umumnya adalah: "Sebagai [Tipe Pengguna], saya ingin [Tindakan], sehingga [Manfaat]." Contoh: "Sebagai pengguna baru, saya ingin melihat *guided tour* pada saat login pertama, sehingga saya bisa memahami fitur-fitur utama aplikasi."

Tujuan dari memecah *user story* menjadi skenario adalah untuk memastikan Anda telah memikirkan semua kemungkinan cara pengguna berinteraksi dengan fitur tersebut, termasuk *happy path*, variasi, dan mode kegagalan.

Decomposing User Story menjadi Skenario

Untuk setiap *user story*, Anda harus menghasilkan **daftar skenario** yang komprehensif. Skenario ini dapat dikategorikan menjadi beberapa jenis:

Happy Path (Jalur Sukses Utama):

- Ini adalah skenario ideal di mana pengguna mengikuti alur yang diharapkan dan semuanya berjalan lancar. Ini adalah jalur paling dasar yang harus selalu berfungsi.

- **Contoh:** Pengguna memasukkan data yang valid dan berhasil menyelesaikan transaksi.

Variasi / Jalur Alternatif (*Alternative Paths*):

- Skenario ini mencakup interaksi pengguna yang valid tetapi sedikit berbeda dari *happy path*. Ini mungkin melibatkan pilihan yang berbeda, data yang berbeda, atau kondisi sistem yang berbeda.

- **Contoh:** Pengguna memilih opsi "Lewati" di *guided tour*. Pengguna login dengan akun yang sudah ada.

Mode Kegagalan / Skenario Negatif (*Failure Modes / Negative Scenarios*):

- Skenario ini menguji bagaimana sistem bereaksi terhadap *input* yang tidak valid, kondisi error, atau tindakan yang tidak diharapkan dari pengguna. Ini adalah bagian krusial untuk memastikan ketahanan dan stabilitas aplikasi.

- **Contoh:** Kegagalan jaringan saat memuat data, *input* yang salah format, API mengembalikan error, pengguna mencoba mengakses fitur tanpa izin.

Contoh Nyata: Daftar Skenario untuk Fitur 'Guided Tour First Login'

Mari kita ambil *user story* yang disebutkan sebelumnya: **Story:** Sebagai pengguna baru, saya melihat *guided tour* pada login pertama.

Berikut adalah daftar skenario yang bisa Anda buat, lengkap dengan penjelasan mengapa setiap skenario penting untuk diuji:

Story: As a new user, I see a guided tour on first login.

Scenarios:

Happy Path: Pengguna baru mendarat di halaman utama → modal *tour* muncul dalam 1 detik.

- **Penjelasan:** Ini adalah skenario dasar. Memastikan *tour* muncul secara otomatis dan cepat untuk pengguna yang baru pertama kali login. Aspek "dalam 1 detik" adalah persyaratan non-fungsional (kinerja) yang penting untuk pengalaman pengguna.
- **Apa yang diuji:** Logika identifikasi pengguna baru, kecepatan pemuatan UI, tampilan *tour* default.

Pengguna Kembali (*Returning User*): Pengguna yang sudah ada login → *tour* TIDAK muncul.

- **Penjelasan:** Penting untuk memastikan *tour* tidak mengganggu pengguna yang sudah familiar dengan aplikasi. Ini menguji logika identifikasi pengguna lama.
- **Apa yang diuji:** Logika kondisi untuk menampilkan/menyembunyikan *tour* berdasarkan status pengguna.

Penolakan (*Dismissal*): Pengguna mengklik "Lewati" → *tour* menghilang, *event* "tour_skipped" terpicu.

- **Penjelasan:** Menguji fungsionalitas tombol "Lewati" dan memastikan *event* analitik yang relevan terpicu. *Event* ini penting untuk PM dalam memahami perilaku pengguna.

- **Apa yang diuji:** Interaksi tombol, penutupan *modal*, pelacakan *event* analitik.

Penyelesaian (*Completion*): Pengguna menyelesaikan semua 4 langkah → *event* "tour_completed" terpicu.

- **Penjelasan:** Menguji seluruh alur *tour* hingga selesai dan memastikan *event* analitik "tour_completed" terpicu.

- **Apa yang diuji:** Navigasi langkah *tour*, transisi antar langkah, pelacakan *event* analitik akhir.

Parsial (*Partial*): Pengguna menutup *browser* di langkah 2 → membuka kembali → *tour* melanjutkan di langkah 2.

- **Penjelasan:** Menguji persistensi status *tour*. Jika pengguna meninggalkan *tour* di tengah jalan, apakah progresnya disimpan dan dilanjutkan saat mereka kembali? Ini adalah contoh *state management*.

- **Apa yang diuji:** Penyimpanan progres *tour* (misalnya di *local storage* atau *backend*), resume *tour* dari langkah terakhir yang dilihat.

Jaringan Lambat (*Slow Network*): Aset *tour* membutuhkan waktu >3 detik untuk dimuat → *fallback* ke *skeleton UI*, tidak ada *error JS*.

- **Penjelasan:** Menguji perilaku aplikasi dalam kondisi jaringan yang kurang ideal. Apakah ada indikator pemuatan yang baik? Apakah aplikasi tetap stabil?

- **Apa yang diuji:** Penanganan *timeout* pemuatan aset, tampilan *placeholder* (*skeleton*), stabilitas aplikasi tanpa *error JavaScript*.

Kegagalan API (*API Failure*): *Endpoint* metadata *tour* mengembalikan 500 → *tour* dilewati, *error* dicatat (*logged*).

- **Penjelasan:** Menguji penanganan error dari *backend*. Jika API yang menyediakan konten *tour* gagal, apakah aplikasi crash atau dapat menanganinya dengan anggun (misalnya, melewati *tour* dan mencatat *error* untuk investigasi)?

- **Apa yang diuji:** Penanganan *error* HTTP (misalnya 500 Internal Server Error), *fallback* behavior, pencatatan *error* di *console* atau sistem *logging*.

Tampilan Seluler Potret (*Mobile Portrait*): *Tour* muat di *viewport*, semua teks terbaca jelas.

- **Penjelasan:** Menguji responsivitas UI *tour* pada orientasi potret di perangkat seluler.
- **Apa yang diuji:** Responsivitas CSS, ukuran font, *layout* elemen, tidak ada *clipping* teks atau elemen tersembunyi.

Tampilan Seluler Lanskap (*Mobile Landscape*): *Tour* reposisi, tidak ada *clipping*.

- **Penjelasan:** Menguji responsivitas UI *tour* pada orientasi lanskap di perangkat seluler.
- **Apa yang diuji:** Adaptasi *layout* untuk orientasi lanskap, elemen tidak tumpang tindih atau terpotong.

Pembaca Layar (*Screen Reader*): Konten *tour* dapat dibaca, urutan fokus benar.

- **Penjelasan:** Menguji aksesibilitas *tour* untuk pengguna dengan kebutuhan khusus yang menggunakan pembaca layar.
- **Apa yang diuji:** Atribut ARIA, *semantic HTML*, urutan tabulasi yang logis, semua teks dan kontrol dapat diakses dan dibaca oleh *screen reader*.

Daftar skenario ini akan menjadi *input* utama Anda untuk tahap selanjutnya: Test Design (merancang *test case*). Semakin detail dan komprehensif daftar skenario Anda, semakin mudah dan efektif proses pembuatan *test case*.

Output dari Analisis Kebutuhan

Setelah Anda selesai mereview PRD dan membuat daftar skenario untuk setiap *user story*, Anda perlu mendokumentasikan hasil analisis Anda. Output ini biasanya berupa tugas atau dokumen "Analisis Pengujian" yang terhubung dengan PRD asli. Ini adalah cara Anda mengomunikasikan temuan Anda kepada tim dan memastikan semua orang berada di halaman yang sama.

Berikut adalah contoh struktur dan konten yang bisa Anda masukkan dalam tugas "Analisis Pengujian" di sistem manajemen proyek Anda (misalnya Notion, Jira):

Judul: Analisis Pengujian: Fitur Guided Tour Login

Pemilik: QA Thomas

Reporter: QA Thomas

Prioritas: P1 (Jika ada pertanyaan yang memblokir, atau P2 jika hanya klarifikasi)

Sumber: QA

Label: test-analysis, requirement-analysis

Ringkasan Review PRD

Link PRD: [URL ke dokumen PRD di Notion/Jira/Confluence]

Pertanyaan Terbuka untuk Product Manager

* [] FR-3: Mohon definisikan "menangani *invalid input* dengan anggun" secara spesifik. (Lihat komentar)

* [] NFR-1: Target waktu pemuatan untuk *dashboard* disebutkan "<2 detik". Apakah ini untuk 90% atau 100%?

* [] Edge Case: Apakah ada persyaratan untuk penanganan *race condition* jika dua pengguna mencoba login bersamaan?

Klarifikasi yang Didapat (setelah diskusi dengan PM)

* [x] FR-3: "Menangani dengan anggun" berarti menampilkan pesan error validasi di bawah *field* input.

* [x] NFR-1: Target <2 detik adalah untuk 90% pengguna di koneksi 4G.

* [x] Edge Case: Untuk *race condition* pendaftaran email, sistem akan mengembalikan error "Email sudah terdaftar".

Skenario yang Diidentifikasi

Story: Sebagai pengguna baru, saya melihat *guided tour* pada login pertama.

* **Happy path:** Pengguna baru login → *tour* muncul dalam 1 detik.

* **Pengguna Kembali:** Pengguna lama login → *tour* TIDAK muncul.

* **Penolakan:** Pengguna klik "Lewati" → *tour* hilang, *event* "tour_skipped" terpicu.

* **Penyelesaian:** Pengguna menyelesaikan semua langkah → *event* "tour_completed" terpicu.

* **Parsial:** Pengguna menutup *browser* di langkah 2 → membuka kembali → *tour* melanjutkan di langkah 3.

* **Jaringan Lambat:** Aset *tour* butuh >3 detik → *skeleton UI*, tanpa *error* JS.

* **Kegagalan API:** *Endpoint* metadata *tour* 500 → *tour* dilewati, *error* dicatat.

* **Seluler Potret:** *Tour* muat di *viewport*, semua teks terbaca jelas.

* **Seluler Lanskap:** *Tour* reposisi, tidak ada *clipping*.

* **Pembaca Layar:** Konten *tour* dapat dibaca, urutan fokus benar.

Estimasi Usaha Desain Pengujian

~ 8 jam (untuk fitur 'Guided Tour Login' ini, termasuk menulis *test cases* manual dan merencanakan tes otomatis)

Dokumen ini menjadi *single source of truth* untuk pemahaman QA tentang fitur tersebut. Ini juga menunjukkan kepada tim bahwa Anda telah melakukan analisis mendalam.

Kriteria Gerbang (Gate Criteria): Kapan Analisis Dianggap Selesai?

Penting untuk memiliki titik henti yang jelas sebelum Anda melanjutkan ke fase berikutnya (Desain Pengujian). Ini adalah "gerbang" yang harus Anda lewati untuk memastikan kualitas fondasi sudah kokoh.

Anda **jangan mulai menulis *test case*** sampai semua kriteria berikut terpenuhi:

[] Semua pertanyaan review PRD telah dijawab dan diselesaikan oleh Product Manager.

- Tidak boleh ada ambiguitas atau celah yang tidak terjawab. Semua orang harus memiliki pemahaman yang sama.

[] Semua skenario pengujian telah didaftarkan dan dikategorikan (happy path, variasi, failure modes) untuk setiap *user story*.

- Daftar skenario Anda harus komprehensif dan mencakup semua aspek yang dapat diuji dari fitur tersebut.

[] Product Manager telah menyetujui cakupan skenario yang Anda identifikasi.

- Ini adalah langkah validasi yang sangat penting. PM perlu mengonfirmasi bahwa daftar skenario Anda sudah sesuai dengan ekspektasi mereka terhadap apa yang akan diuji. Jika ada skenario yang terlewat atau prioritas yang berbeda, ini adalah kesempatan terakhir untuk menyelaraskannya.

Setelah Anda mendapatkan "lampu hijau" dari PM dan semua poin di atas terpenuhi, barulah Anda bisa dengan percaya diri melangkah ke fase berikutnya: Desain Pengujian, di mana Anda akan mengubah skenario-skenario ini menjadi *test case* yang konkret.

Rangkuman

Bab ini telah membawa kita pada Quality Gate pertama dan terpenting dalam perjalanan QA Anda: Analisis Kebutuhan. Kita telah belajar bahwa peran QA bukan hanya menemukan *bug* di akhir, tetapi menjadi penjaga kualitas proaktif yang memastikan fondasi produk dibangun di atas persyaratan yang jelas, lengkap, dan dapat diuji.

Anda kini memiliki alat untuk: * Menganalisis Product Requirement Document (PRD) secara kritis menggunakan daftar periksa 8 poin yang komprehensif. * Memberikan *feedback* yang konstruktif dan spesifik kepada Product Manager, mengubah ambiguitas menjadi klarifikasi yang dapat diuji. * Memecah *user story* menjadi daftar skenario pengujian yang detail, mencakup *happy path*, variasi, dan mode kegagalan, seperti yang kita lihat pada contoh *guided tour first login*. * Mendokumentasikan hasil analisis Anda dalam format yang jelas dan actionable. * Mengetahui kapan analisis Anda dianggap selesai, dengan persetujuan dari Product Manager, sebelum melangkah ke tahap selanjutnya.

Dengan menguasai tahap ini, Anda tidak hanya mencegah *bug* sebelum mereka lahir, tetapi juga membangun hubungan yang kuat dengan tim produk dan *engineering*, menunjukkan nilai strategis Anda sebagai Quality Assurance Engineer.

Checklist Actionable

Berikut adalah daftar tindakan yang dapat Anda lakukan segera setelah membaca bab ini:

- [] **Pahami Alur Kerja PRD:** Ketahui di mana PRD disimpan di perusahaan Anda (Notion, Jira, Confluence, dll.) dan siapa Product Manager yang bertanggung jawab.
- [] **Jadwalkan Keterlibatan Dini:** Bicarakan dengan PM Anda untuk memastikan Anda dilibatkan dalam review PRD *sebelum engineering* mulai koding.
- [] **Terapkan Checklist Review PRD:** Gunakan 8 poin checklist yang dibahas dalam bab ini setiap kali Anda mereview PRD. Buat template pribadi Anda jika perlu.
- [] **Latih Komunikasi Konstruktif:** Saat menemukan celah, tulis komentar di PRD yang spesifik, mengajukan pertanyaan, dan menawarkan opsi solusi, bukan hanya menunjuk masalah.

- [] **Pecah User Story menjadi Skenario:** Untuk setiap *user story* dalam PRD, buat daftar skenario yang mencakup *happy path*, variasi, dan *failure modes*. Gunakan contoh 'guided tour' sebagai referensi.
 - [] **Dokumentasikan Output Analisis:** Buat tugas "Analisis Pengujian" di sistem manajemen proyek Anda yang merangkum temuan PRD dan daftar skenario.
 - [] **Pastikan Kriteria Gerbang Terpenuhi:** Jangan mulai menulis *test case* sampai semua pertanyaan terjawab, semua skenario terdaftar, dan PM telah menyetujui cakupan skenario Anda.
 - [] **Ajukan Pertanyaan:** Jika Anda tidak yakin tentang suatu persyaratan, jangan ragu untuk bertanya. Lebih baik bertanya sekarang daripada menemukan *bug* nanti.
-

Chapter 02: Test Design — Strategi, Estimasi, dan Test Case

Setelah Anda berhasil menyelami dunia *Requirement Analysis* di bab sebelumnya, kini saatnya kita melangkah lebih jauh. Memahami kebutuhan produk adalah fondasi, namun bagaimana kita memastikan bahwa kebutuhan tersebut benar-benar terpenuhi oleh produk yang akan dibangun? Jawabannya ada di *Test Design*.

Banyak yang mengira peran QA baru dimulai setelah *developer* selesai menulis kode. Namun, di *Journey QA Thomas*, kami percaya bahwa peran QA adalah penjaga kualitas sejak dini. *Test Design* adalah momen krusial di mana Anda, sebagai QA Engineer, mulai merancang cetak biru pengujian Anda, bahkan sebelum satu baris kode pun ditulis oleh tim *engineering*. Ini adalah kesempatan emas untuk mengidentifikasi potensi masalah, menyelaraskan ekspektasi, dan membangun fondasi yang kokoh untuk pengujian yang efektif.

Pada bab ini, kita akan membahas secara mendalam bagaimana membangun *Test Strategy* yang komprehensif, melakukan estimasi waktu yang realistis, dan menulis *Test Case* dalam empat kategori penting. Anda akan melihat bagaimana setiap langkah ini berkontribusi pada kualitas produk secara keseluruhan, dan bagaimana Anda dapat menjadi Quality Gate yang proaktif,

bukan hanya reaktif. Mari kita mulai merancang!

1. Merancang Strategi Pengujian (Test Strategy)

Test Strategy adalah dokumen hidup yang menjadi panduan utama Anda dalam melakukan pengujian untuk sebuah fitur atau proyek. Ini bukan sekadar daftar tugas, melainkan sebuah peta jalan yang menjawab pertanyaan-pertanyaan fundamental: **APA** yang akan kita uji, **BAGAIMANA** cara mengujinya, **DENGAN APA** pengujian itu dilakukan, dan **KAPAN** pengujian itu harus selesai.

Menyusun *Test Strategy* secara dini memiliki beberapa keuntungan: 1. **Klarifikasi Scope**: Memastikan semua pihak (PM, Developer, QA) memiliki pemahaman yang sama tentang batasan pengujian. 2. **Identifikasi Risiko**: Mengidentifikasi potensi masalah lebih awal dan merencanakan mitigasinya. 3. **Estimasi Akurat**: Memberikan dasar yang kuat untuk estimasi waktu dan sumber daya. 4. **Standarisasi**: Menjaga konsistensi dalam pendekatan pengujian di seluruh proyek.

Di Thomas, kami menggunakan *template* standar yang disimpan sebagai tugas di Notion, dengan label test-strategy. Ini memastikan konsistensi dan kemudahan pelacakan. Mari kita bedah setiap bagian dari *template* ini.

Template Test Strategy

Berikut adalah *template* yang kami gunakan. Anda bisa mengadaptasinya sesuai kebutuhan tim Anda.

```
# Test Strategy: <Nama Fitur>

## Scope
- In scope: <Daftar user story, fungsionalitas, atau modul yang akan diuji secara eksplisit>
- Out of scope: <Daftar fungsionalitas atau modul yang secara eksplisit TIDAK akan diuji dalam iterasi>

## Test Approach
- Automation Smoke: <Ya/Tidak, skenario mana yang akan diotomatisasi untuk smoke test>
- Functional Testing: <Pendekatan pengujian fungsional, misal: berdasarkan user story, end-to-end, integration>
- Regression: <Rencana untuk pengujian regresi; apakah akan menjalankan seluruh suite yang ada, atau m>
- Exploratory: <Alokasi waktu (time-box), area fokus untuk pengujian eksplorasi>
- Performance: <Jika berlaku, target performa yang akan diuji (misal: response time, throughput)>
- Accessibility: <Jika berlaku, standar aksesibilitas yang akan dipatuhi (misal: WCAG)>

## Environments
```

- Test data: <Bagaimana data uji akan disiapkan (misal: seeded, manual, via API)>
- Devices: <Daftar perangkat yang akan digunakan untuk pengujian (misal: Desktop, Mobile Android, Mobile iOS)>
- Browsers: <Daftar browser yang akan didukung dan diuji (misal: Chrome, Firefox, Safari, Edge)>
- Versions / builds tested: <Rencana versi/build yang akan diuji (misal: versi staging terbaru, build produksi)>

Entry Criteria

- [] PRD (Product Requirement Document) final dan disetujui
- [] Test scenarios (dari Chapter 01) disetujui oleh PM/Tech Lead
- [] Build aplikasi tersedia di environment staging
- [] Data uji awal telah disiapkan (seeded)

Exit Criteria

- [] Semua test case P0/P1 berhasil (passing)
- [] Tidak ada bug P0/P1 yang masih terbuka
- [] Suite regresi berhasil dijalankan tanpa blokir (passing)
- [] UAT (User Acceptance Testing) telah ditandatangani oleh pemangku kepentingan

Estimasi Waktu (WAJIB)

- Penyusunan Test Case: <h> jam
- Persiapan data uji: <h> jam
- Eksekusi (pass pertama): <h> jam
- Verifikasi bug & pengerjaan ulang: <h> jam
- Regresi: <h> jam
- Pelaporan: <h> jam
- Buffer (20%): <h> jam

****TOTAL: <h> jam / <d> hari****

Risks

- <Risiko yang diidentifikasi> → <Rencana mitigasi>

Mari kita bahas lebih lanjut setiap bagiannya:

a. Scope: Batasan Pengujian

Bagian ini adalah tentang mendefinisikan apa yang akan Anda uji dan apa yang tidak. Penting untuk sangat eksplisit di sini.

- **In scope:** Cantumkan semua *user story*, fungsionalitas, atau area modul yang akan menjadi fokus pengujian Anda. Ini membantu memastikan tidak ada yang terlewat dan semua ekspektasi terpenuhi. Misalnya, jika Anda menguji fitur "Pembayaran Online", yang termasuk dalam *scope* bisa jadi "Pengguna dapat membayar dengan kartu kredit", "Pengguna dapat melihat riwayat transaksi", dll.

- **Out of scope:** Ini sama pentingnya. Secara eksplisit nyatakan fungsionalitas yang *tidak* akan diuji dalam iterasi ini. Mengapa? Untuk menghindari kesalahpahaman dan mengelola

ekspektasi. Mungkin fitur tertentu akan diuji di fase berikutnya, atau mungkin itu adalah fungsionalitas lama yang tidak terpengaruh oleh perubahan saat ini.

b. Test Approach: Bagaimana Kita Akan Menguji?

Bagian ini menjelaskan metodologi dan jenis pengujian yang akan Anda terapkan.

- **Automation Smoke:** Apakah ada *smoke test* yang akan diotomatisasi? *Smoke test* adalah pengujian cepat untuk memastikan fungsionalitas inti aplikasi berjalan. Jika ya, skenario mana saja yang akan di-otomatisasi?
- **Functional Testing:** Ini adalah pengujian utama untuk memastikan fungsionalitas bekerja sesuai PRD. Anda bisa menjelaskan pendekatan Anda, apakah itu *end-to-end*, *integrasi*, atau berbasis *user story*.
- **Regression:** Bagaimana Anda akan menangani *regression testing*? Apakah Anda akan menjalankan seluruh *suite* regresi yang ada, atau hanya subset yang relevan dengan perubahan saat ini?
- **Exploratory:** Alokasikan waktu untuk *exploratory testing*. Ini adalah pengujian tanpa skenario yang terstruktur, di mana Anda bebas menjelajahi aplikasi untuk menemukan bug yang tidak terduga. Tentukan *time-box* (misalnya, 2 jam) dan area fokusnya.
- **Performance & Accessibility:** Jika fitur memiliki persyaratan performa atau aksesibilitas, sebutkan target dan standar yang akan diuji.

c. Environments: Dengan Apa Kita Menguji?

Detail teknis tentang lingkungan pengujian Anda.

- **Test data:** Jelaskan bagaimana Anda akan mendapatkan data yang dibutuhkan untuk pengujian. Apakah Anda akan menggunakan data yang sudah ada, membuat data baru secara manual, atau menggunakan *script* otomatis (*seeded data*)?
- **Devices & Browsers:** Daftar perangkat (misalnya, iPhone 13, Samsung Galaxy S22, Desktop) dan *browser* (misalnya, Chrome, Firefox, Safari) yang akan Anda gunakan untuk pengujian. Ini penting untuk memastikan kompatibilitas.
- **Versions / builds tested:** Tentukan versi aplikasi atau nomor *build* yang akan Anda uji. Pastikan Anda memiliki mekanisme untuk melacak versi yang sedang diuji.

d. Entry Criteria: Kapan Kita Boleh Mulai Menguji?

Ini adalah daftar prasyarat yang harus terpenuhi sebelum Anda dapat *memulai* eksekusi pengujian. Tanpa ini, Anda berisiko membuang waktu menguji *build* yang tidak stabil atau persyaratan yang belum final.

- **PRD final:** Dokumen persyaratan produk harus sudah disetujui dan tidak ada lagi perubahan signifikan.
- **Test scenarios approved:** Skenario pengujian yang Anda buat (dari Chapter 01) telah ditinjau dan disetujui oleh tim.
- **Build available in staging:** Versi aplikasi yang akan diuji sudah ter-deploy di lingkungan *staging* dan siap diakses.
- **Test data seeded:** Data yang diperlukan untuk pengujian sudah tersedia dan siap digunakan.

e. Exit Criteria: Kapan Kita Boleh Menyatakan Pengujian Selesai?

Ini adalah kondisi yang harus terpenuhi agar pengujian dapat dianggap selesai dan fitur siap untuk dirilis.

- **All P0/P1 test cases passing:** Semua *test case* dengan prioritas tertinggi (P0 dan P1) harus berhasil.
- **No open P0/P1 bugs:** Tidak ada bug dengan prioritas P0 atau P1 yang masih terbuka. Bug P2 dan P3 mungkin bisa diterima dengan persetujuan tim, tergantung risikonya.
- **Regression suite passing:** *Suite* regresi telah dijalankan dan semua *test case* di dalamnya berhasil.
- **UAT signed off:** *User Acceptance Testing* (UAT) telah diselesaikan dan disetujui oleh *stakeholder* terkait.

f. Estimasi Waktu (WAJIB)

Ini adalah bagian yang seringkali dianggap remeh, namun sangat krusial. Estimasi waktu yang akurat membantu tim perencanaan proyek (PM, Tech Lead) dalam menentukan jadwal rilis dan kapasitas tim. **Setiap Test Strategy harus memiliki bagian estimasi waktu ini.**

Berikut adalah komponen-komponen yang perlu Anda estimasi: * **Penyusunan Test Case:** Berapa jam yang Anda butuhkan untuk menulis semua *test case* untuk fitur ini? * **Persiapan data uji:** Berapa jam untuk menyiapkan semua data yang dibutuhkan? Ini bisa termasuk membuat akun, mengimpor data, atau memanipulasi basis data. * **Eksekusi (pass pertama):** Berapa jam untuk menjalankan semua *test case* untuk pertama kalinya? * **Verifikasi bug & pengerjaan ulang:** Alokasikan waktu untuk memverifikasi perbaikan bug dan melakukan *re-testing* jika ada bagian aplikasi yang diubah. * **Regresi:** Berapa jam untuk menjalankan *regression test* yang relevan? * **Pelaporan:** Waktu untuk menyusun laporan pengujian, *release notes*, dll. * **Buffer (20%):** Selalu tambahkan *buffer*! Proyek jarang berjalan persis sesuai rencana. *Buffer* 20% adalah praktik yang baik untuk mengakomodasi hal-hal tak terduga.

Jumlahkan semua komponen ini untuk mendapatkan total estimasi waktu dalam jam, lalu konversikan ke hari kerja.

Contoh Pengisian Estimasi Waktu: Misalkan Anda mengestimasi fitur "Pembayaran Online" sebagai berikut: * Penyusunan Test Case: 8 jam * Persiapan data uji: 4 jam * Eksekusi (pass pertama): 12 jam * Verifikasi bug & pengerjaan ulang: 6 jam * Regresi: 4 jam * Pelaporan: 2 jam * Buffer (20% dari 36 jam): 7.2 jam (bulatkan menjadi 8 jam) **TOTAL: 44 jam / 5.5 hari kerja** (dengan asumsi 8 jam kerja per hari)

g. Risks: Identifikasi dan Mitigasi

Bagian terakhir dari *Test Strategy* adalah mengidentifikasi potensi risiko yang dapat menghambat proses pengujian atau kualitas produk, serta rencana mitigasinya.

- **Contoh Risiko:** Keterlambatan *build*, perubahan persyaratan mendadak, lingkungan pengujian tidak stabil, keterbatasan sumber daya, dll.
- **Mitigasi:** Apa yang akan Anda lakukan untuk mengurangi dampak risiko tersebut? Misalnya, jika risikonya adalah "keterlambatan *build*", mitigasinya bisa berupa "komunikasi proaktif dengan tim *development* dan PM untuk mendapatkan *update* status *build* secara berkala".

Dengan *Test Strategy* yang solid, Anda telah meletakkan dasar yang kuat untuk pengujian yang terstruktur dan efisien. Selanjutnya, kita akan membahas cara membuat *Test Case* yang spesifik dan komprehensif.

2. Membangun Test Case: Empat Kategori Krusial

Setelah *Test Strategy* Anda terbentuk, langkah berikutnya adalah menerjemahkan *test scenarios* yang telah Anda buat di Chapter 01 menjadi *Test Case* yang spesifik dan dapat dieksekusi. Di Thomas, kami menekankan bahwa setiap set *test case* harus mencakup empat kategori berikut untuk memastikan cakupan pengujian yang menyeluruh:

1. **Positive Cases (Skenario Sukses)**
2. **Negative Cases (Skenario Gagal)**
3. **Edge Cases (Skenario Batasan/Tak Biasa)**
4. **Data Preparation (Persiapan Data Uji)**

Jangan pernah melewatkan salah satu kategori ini. Masing-masing memiliki peran penting dalam memastikan kualitas produk. Untuk memberikan contoh konkret, kita akan menggunakan fitur "Login Pengguna" sebagai studi kasus.

Format Test Case Standar

Sebelum masuk ke kategorinya, mari kita lihat format standar *Test Case* yang kami gunakan. Format ini memastikan semua informasi penting tercatat dan *test case* mudah dipahami serta dieksekusi.

```
### TC-[ID]: [Judul singkat deskriptif]

**Type:** Positive | Negative | Edge | Data Preparation
**Linked Story:** [ID User Story atau tautan PRD yang terkait]
**Linked AC:** [Nomor Acceptance Criteria dari user story yang divalidasi]
**Prioritas:** P0 | P1 | P2 | P3
**Estimasi waktu:** [menit]

**Preconditions (Prasyarat):**
- [kondisi #1 yang harus terpenuhi sebelum memulai]
- [kondisi #2 yang harus terpenuhi sebelum memulai]

**Test data (Data Uji):**
- [input atau referensi fixture yang digunakan]

**Steps (Langkah-langkah):**
1. [aksi yang tepat]
2. [aksi yang tepat]
```

3. [aksi yang tepat]

****Expected result (Hasil yang diharapkan):****

- [hasil yang dapat diamati – harus jelas dan tidak ambigu]

****Tear-down (Jika diperlukan):****

- [mengembalikan kondisi semula, membersihkan data]

Pentingnya Estimasi Waktu per Test Case: Seperti yang telah dibahas di *Test Strategy*, setiap *test case* **wajib** memiliki estimasi waktu. Mengapa ini begitu penting? * **Agregasi ke Test Strategy:** Estimasi ini akan diakumulasikan untuk membentuk estimasi total dalam *Test Strategy* Anda, yang dibutuhkan oleh PM dan Tech Lead untuk perencanaan *sprint*. * **Identifikasi Utang Pengujian:** *Test case* manual yang memakan waktu lama (misalnya, 30 menit) adalah kandidat kuat untuk diotomatisasi. Ini membantu Anda mengidentifikasi "utang pengujian" Anda. * **Ekspektasi SLA QA:** Menetapkan ekspektasi waktu untuk eksekusi pengujian.

Berikut adalah panduan kasar untuk estimasi waktu: * **< 5 menit:** Sangat cepat, kandidat kuat untuk otomatisasi. * **5-15 menit:** Manual tapi cepat, bisa diotomatisasi jika sering diulang. * **15-30 menit:** Membutuhkan *setup* yang kompleks, ROI otomatisasi tinggi. * **30+ menit:** Segera eskalasi! Hampir selalu layak untuk diotomatisasi.

Sekarang, mari kita lihat contoh *Test Case* untuk fitur Login berdasarkan empat kategori.

a. Positive Cases ("Happy Path")

Positive Cases memvalidasi bahwa *user story* berfungsi seperti yang diharapkan ketika pengguna melakukan tindakan yang benar dan sistem merespons sesuai spesifikasi. Ini adalah skenario "jalan bahagia" di mana semuanya berjalan lancar. **Setiap Acceptance Criteria (AC) setidaknya membutuhkan satu positive case.**

Contoh:

TC-P-001: Pengguna terverifikasi berhasil login dan melihat dashboard

****Type:**** Positive

****Linked Story:**** US-001: Sebagai pengguna, saya ingin login ke aplikasi

****Linked AC:**** AC1: Pengguna dapat login dengan email dan password yang valid.

****Prioritas:**** P0

****Estimasi waktu:**** 5 menit

****Preconditions:****

- Pengguna uji `tc001@thomas.local` sudah terdaftar, terverifikasi, dan memiliki password yang valid.
- Pengguna memiliki data dashboard 7 hari terakhir.

****Test data:****

- Email: `tc001@thomas.local`
- Password: `Password123!`

****Steps:****

1. Navigasi ke halaman `/login`.
2. Masukkan email `tc001@thomas.local` dan password `Password123!`.
3. Klik tombol "Sign In".

****Expected result:****

- Pengguna diarahkan ke halaman `/dashboard` dalam waktu 2 detik.
- Dashboard menampilkan 3 kartu data (Penggunaan, Tren, Anomali) yang terisi.
- Tidak ada *error* di konsol *browser*.
- *Auth cookie* terpasang dengan benar.

b. Negative Cases (Input atau Alur yang Sengaja Salah)

Negative Cases memvalidasi bahwa sistem dapat menangani input yang salah, tidak valid, atau alur yang tidak diharapkan dengan aman dan anggun. Tujuannya adalah memastikan sistem tidak *crash*, tidak merusak data, dan memberikan pesan kesalahan yang jelas dan informatif.

Contoh:

TC-N-001: Login gagal dengan password yang salah

****Type:** Negative**

****Linked Story:**** US-001: Sebagai pengguna, saya ingin login ke aplikasi

****Linked AC:**** AC1 (implisit: kebutuhan autentikasi yang aman)

****Prioritas:** P1**

****Estimasi waktu:** 3 menit**

****Preconditions:****

- Pengguna uji `tc001@thomas.local` sudah terdaftar dan terverifikasi.

****Test data:****

- Email: `tc001@thomas.local`
- Password: `salahpassword` (password yang salah)

****Steps:****

1. Navigasi ke halaman `/login`.
2. Masukkan email `tc001@thomas.local` (email benar) dan password `salahpassword` (password salah).
3. Klik tombol "Sign In".

****Expected result:****

- Pesan kesalahan generik "Email atau password tidak valid" ditampilkan di bawah form login.

- Pengguna TIDAK diarahkan ke halaman lain; tetap berada di halaman `/login`.
- Form tetap terisi dengan email (bukan password).
- Tidak ada `*auth cookie*` yang terpasang.
- Percobaan login dicatat untuk `*rate limiting*` (misalnya, setelah 5 kali percobaan gagal, akun dikunci).

c. Edge Cases (Kondisi Batasan atau Input Tak Biasa)

Edge Cases adalah skenario yang menguji kondisi batas, input yang tidak biasa tetapi valid, atau situasi yang jarang terjadi. Ini membantu menemukan *bug* yang mungkin terlewat oleh *positive* atau *negative cases* standar.

Pola Umum Edge Cases:

- * **Nilai Batas:** 0, 1, nilai maksimum - 1, nilai maksimum, nilai maksimum + 1.
- * **Input Kosong/Null/Sangat Panjang:** Apa yang terjadi jika *field* kosong, atau diisi dengan ribuan karakter?
- * **Karakter Khusus:** Unicode, emoji, bahasa RTL (Right-to-Left).
- * **Kondisi Jaringan:** Jaringan lambat (3G *throttling*).
- * **Aksi Bersamaan:** Membuka aplikasi di dua *tab browser* secara bersamaan.
- * **Waktu:** Tengah malam, pergantian waktu musim panas (DST), tahun kabisat.
- * **Batasan Izin:** Pengguna yang baru saja dicabut/diberikan izin.

Contoh untuk Fitur Login:

```
### TC-E-001: Login dengan email yang mengandung tanda plus (alias email)
**Tipe:** Edge
**Linked Story:** US-001: Sebagai pengguna, saya ingin login ke aplikasi
**Linked AC:** AC1 (implisit: penanganan format email yang fleksibel)
**Prioritas:** P2
**Estimasi waktu:** 4 menit

**Preconditions:**
- Pengguna uji `tc001+alias@thomas.local` sudah terdaftar dan terverifikasi (alias email yang valid).

**Test data:**
- Email: `tc001+alias@thomas.local`
- Password: `Password123!`

**Steps:**
1. Navigasi ke halaman `/login`.
2. Masukkan email `tc001+alias@thomas.local` dan password `Password123!`.
3. Klik tombol "Sign In".

**Expected result:**
- Pengguna berhasil login dan diarahkan ke halaman `/dashboard`.
- Alias email ditangani dengan benar oleh sistem.
```

TC-E-002: Login dengan password yang sangat panjang (256 karakter)

Type: Edge

Linked Story: US-001: Sebagai pengguna, saya ingin login ke aplikasi

Linked AC: AC1 (implisit: batasan input password)

Prioritas: P2

Estimasi waktu: 5 menit

Preconditions:

- Pengguna uji `tc002@thomas.local` sudah terdaftar dengan password 256 karakter yang valid.

Test data:

- Email: `tc002@thomas.local`

- Password: [password 256 karakter yang valid]

Steps:

1. Navigasi ke halaman `/login`.
2. Masukkan email `tc002@thomas.local` dan password yang sangat panjang.
3. Klik tombol "Sign In".

Expected result:

- Pengguna berhasil login dan diarahkan ke halaman `/dashboard`.
- Sistem menangani panjang password dengan benar tanpa *error* atau *truncation*.

TC-E-003: Login dengan spasi di awal/akhir email

Type: Edge

Linked Story: US-001: Sebagai pengguna, saya ingin login ke aplikasi

Linked AC: AC1 (implisit: *trimming* spasi input)

Prioritas: P2

Estimasi waktu: 3 menit

Preconditions:

- Pengguna uji `tc003@thomas.local` sudah terdaftar.

Test data:

- Email: ` tc003@thomas.local ` (dengan spasi di awal dan akhir)

- Password: `Password123!`

Steps:

1. Navigasi ke halaman `/login`.
2. Masukkan email dengan spasi di awal/akhir dan password.
3. Klik tombol "Sign In".

Expected result:

- Sistem secara otomatis *trim* spasi di awal/akhir email.

- Pengguna berhasil login dan diarahkan ke halaman `/dashboard`.

█

█

TC-E-004: Login dari negara baru (memicu geofencing)

Tipe: Edge

Linked Story: US-001: Sebagai pengguna, saya ingin login ke aplikasi

Linked AC: AC1 (implisit: keamanan akun)

Prioritas: P1

Estimasi waktu: 7 menit

█

Preconditions:

- Pengguna uji `tc004@thomas.local` belum pernah login dari lokasi geografis "Jerman".

- Sistem memiliki fitur deteksi login dari lokasi baru (geofencing) yang memerlukan verifikasi tambahan

█

Test data:

- Email: `tc004@thomas.local`

- Password: `Password123!`

- IP Address: [IP Address dari Jerman, dapat disimulasikan dengan VPN/proxy]

█

Steps:

1. Navigasi ke halaman `/login` menggunakan VPN/proxy ke Jerman.

2. Masukkan email dan password.

3. Klik tombol "Sign In".

█

Expected result:

- Sistem mendeteksi login dari lokasi baru.

- Pengguna diminta untuk melakukan verifikasi tambahan (misal: kode OTP ke email/telepon).

- Email notifikasi "Login dari lokasi baru" terkirim ke email pengguna.

d. Data Preparation (Persiapan Data Uji)

Data Preparation adalah *test case* itu sendiri, bukan hanya catatan di *precondition*. Ini adalah langkah-langkah yang diperlukan untuk membangun kondisi data yang spesifik untuk *test case* lain. Mengapa ini penting? Karena *setup* data bisa gagal, dan kegagalan ini perlu dilacak dan diperbaiki.

Contoh:

TC-DP-001: Seed pengguna uji standar dengan data dashboard 7 hari

Tipe: Data Preparation

Linked Stories: US-001, US-002, dst. (Beberapa kasus dashboard)

Prioritas: P0 (memblokir semua pengujian dashboard)

Estimasi waktu: 10 menit (satu kali)

****Preconditions:****

- Akses ke lingkungan development/staging.
- Script seeding data `scripts/seed\_test\_data.py` tersedia dan berfungsi.

****Steps:****

1. Jalankan `*script* `scripts/seed_test_data.py --user tc001@thomas.local --days 7`` di terminal.
2. Verifikasi di database PostgreSQL: ``SELECT count(*) FROM events WHERE user_email='tc001@thomas.local``

****Expected result:****

- Script keluar dengan kode 0 (berhasil).
- Jumlah `*event*` di PostgreSQL sesuai dengan yang diharapkan (biasanya 100-200 `*event*` tergantung konfigurasi).

TC-DP-002: Siapkan pengguna dengan 2FA (Two-Factor Authentication) diaktifkan

****Type:** Data Preparation**

****Linked Stories:**** US-003: Sebagai pengguna, saya ingin mengaktifkan 2FA

****Prioritas:** P1**

****Estimasi waktu:** 5 menit**

****Preconditions:****

- Pengguna ``tc002fa@thomas.local`` sudah terdaftar.

****Steps:****

1. Login sebagai ``tc002fa@thomas.local``.
2. Navigasi ke halaman Pengaturan Akun.
3. Aktifkan 2FA melalui aplikasi Authenticator.
4. Verifikasi dengan kode OTP.

****Expected result:****

- 2FA berhasil diaktifkan untuk akun ``tc002fa@thomas.local``.
- Pengguna tidak dapat login tanpa kode 2FA.

TC-DP-003: Siapkan pengguna yang terkunci (locked-out user)

****Type:** Data Preparation**

****Linked Stories:**** US-001 (implisit: keamanan login)

****Prioritas:** P1**

****Estimasi waktu:** 3 menit**

****Preconditions:****

- Pengguna ``tc003locked@thomas.local`` sudah terdaftar.

****Steps:****

1. Navigasi ke halaman ``/login``.
2. Masukkan email ``tc003locked@thomas.local`` dan password yang salah sebanyak 5 kali berturut-turut.

3. Amati pesan kesalahan dan status akun.

****Expected result:****

- Setelah 5 percobaan gagal, akun `tc003locked@thomas.local` terkunci selama 5 menit.
- Pesan kesalahan yang sesuai ditampilkan ("Akun Anda sementara terkunci. Coba lagi dalam 5 menit.").

Anti-Pola dalam Penulisan Test Case

Ada beberapa kebiasaan buruk yang harus Anda hindari saat menulis *test case*:

- **"Uji bahwa login berfungsi"**: Terlalu samar. Selalu sebutkan AC yang tepat, langkah-langkah yang presisi, dan hasil yang diharapkan.
- **Menggabungkan *positive* dan *negative case* dalam satu TC**: Pisahkan. Setiap *test case* harus fokus pada satu tujuan pengujian.
- **Melewatkan *data preparation TC* dengan alasan "itu hanya *setup*"**: Jangan. *Setup* data bisa gagal, dan kegagalan itu perlu dilacak dan dipelihara.
- **Tidak menyertakan estimasi waktu**: Diagram alur kerja Thomas secara eksplisit mewajibkan ini. PM membutuhkan ini untuk perencanaan.
- **Tidak ada tautan ke AC**: *Test case* tanpa tautan ke *user story* atau AC adalah "tes yatim piatu". Ini mengalahkan tujuan utama QA untuk memverifikasi bahwa *Acceptance Criteria* terpenuhi.

3. Menyimpan dan Mengelola Test Case

Setelah Anda selesai menulis semua *test case* Anda, langkah berikutnya adalah menyimpannya di lokasi yang terorganisir dan mudah diakses. Di Thomas, kami menggunakan Notion sebagai sistem manajemen tugas dan dokumentasi utama kami.

Penyimpanan di Notion

Setiap set *test case* untuk sebuah fitur disimpan sebagai sub-tugas di bawah PRD induk di Notion. Ini memungkinkan keterkaitan yang jelas antara persyaratan dan pengujian.

Anda bisa membuat tugas secara manual, atau jika Anda memiliki alat bantu, Anda bisa membuatnya secara programatik:

```
from notion_connector import NotionTicketing # Asumsi ada modul ini
nt = NotionTicketing.from_env()

# Contoh membuat tugas untuk Test Case
nt.create_task(
    title=f"Test Cases: Fitur Login Pengguna",
    owner="qa-thomas",
    reporter="qa-thomas",
    priority="P1",
    source="QA",
    epic_id=f"<ID-Epic-Fitur-Login>", # ID Epic dari fitur Login
    labels=["test-cases"],
    body_markdown=f"<markdown-dari-semua-test-case-di-atas>", # Semua test case dalam satu dokumen markdown
)
```

Catatan: Dalam praktik, untuk *test case* yang sangat banyak, mungkin lebih praktis untuk membuat setiap *test case* sebagai tugas terpisah di Notion, dengan judul **TC-XXX: [Judul Test Case]**. Ini memungkinkan pelacakan status (Pass/Fail) per *test case* secara individual. Setiap tugas *test case* ini kemudian akan dilabeli **test-case** dan kategori spesifiknya (**positive**, **negative**, **edge**, **data-prep**), serta ditautkan ke *epic* fitur induk.

Untuk Suite yang Lebih Besar: CSV di Google Drive

Untuk *suite* pengujian yang sangat besar (ratusan atau ribuan *test case*), menyimpan semuanya sebagai *body markdown* di satu tugas Notion mungkin kurang efisien untuk pelacakan individual. Dalam kasus ini, kami akan melampirkan file CSV di Google Drive (satu baris per *test case*) dan menautkan file tersebut dari tugas Notion.

Format Kolom CSV Test Case Standar

Saat menggunakan CSV, pastikan Anda memiliki kolom standar ini untuk konsistensi dan kemudahan analisis:

Kolom	Deskripsi	Contoh
TC ID	ID unik untuk <i>test case</i> .	TC-P-001

Category	Kategori <i>test case</i> (Positive, Negative, Edge, Data Preparation).	Positive
Title	Judul singkat dan deskriptif.	Login dengan kredensial valid
Pre-condition	Kondisi yang harus terpenuhi sebelum <i>test case</i> dieksekusi.	Pengguna terdaftar
Steps	Langkah-langkah detail untuk mengeksekusi <i>test case</i> .	1. Navigasi ke /login; 2. Masukkan email & password; ...
Expected	Hasil yang diharapkan setelah langkah-langkah dieksekusi.	Pengguna diarahkan ke /dashboard
Linked Story	ID atau tautan ke <i>user story</i> yang terkait.	US-001
Automatable	Indikasi apakah <i>test case</i> ini kandidat untuk otomatisasi (Ya/Tidak).	Ya
Priority	Prioritas <i>test case</i> (P0, P1, P2, P3).	P0
Estimasi Waktu	Estimasi waktu eksekusi <i>test case</i> dalam menit.	5

Menggunakan format yang konsisten, baik di Notion maupun CSV, sangat penting untuk menjaga keteraturan dan mempermudah kolaborasi dalam tim QA.

4. Mengidentifikasi Kandidat Otomatisasi

Salah satu praktik terbaik dalam *Test Design* adalah mengidentifikasi *test case* mana yang menjadi kandidat kuat untuk otomatisasi **sejak awal**. Ini membantu Anda dan tim *engineering* merencanakan pengembangan *automation script* secara paralel dengan pengembangan fitur. Mengotomatisasi *test case* yang tepat dapat menghemat waktu dan sumber daya dalam jangka panjang, terutama untuk pengujian regresi.

Di Thomas, kami memiliki aturan standar untuk menentukan kandidat otomatisasi:

Semua *test case* P0/P1 → Otomatisasi Wajib

- *Test case* dengan prioritas P0 (Critical) dan P1 (High) adalah yang paling penting untuk memastikan fungsionalitas inti aplikasi. Kegagalan pada *test case* ini dapat menyebabkan dampak serius pada pengguna atau bisnis. Oleh karena itu, otomatisasi mereka adalah prioritas utama untuk memastikan stabilitas dan kualitas secara berkelanjutan.

- Contoh: Login, registrasi, pembayaran, membuat pesanan.

P2 → Otomatisasi Bersyarat (jika *effort* rendah)

- *Test case* P2 (Medium) memiliki tingkat kepentingan sedang. Otomatisasi mereka dipertimbangkan jika upaya yang dibutuhkan untuk menulis *automation script* relatif rendah. Ini bisa berarti *test case* tersebut sederhana, tidak memerlukan banyak *setup* data, atau menggunakan komponen yang sudah memiliki *helper function* otomatisasi.

- Contoh: Memperbarui profil pengguna, filter pencarian sederhana.

P3 → Manual Saja (kecuali sangat trivial)

- *Test case* P3 (Low) biasanya melibatkan fungsionalitas minor, *edge cases* yang sangat jarang, atau skenario yang membutuhkan interaksi manusia yang kompleks (misalnya, verifikasi visual yang sulit diotomatisasi). Umumnya, *test case* ini akan tetap diuji secara manual. Pengecualian adalah jika *test case* tersebut sangat trivial dan dapat diotomatisasi dengan sedikit atau tanpa *effort* tambahan.

- Contoh: Mengubah tema aplikasi, memeriksa *tooltip* kecil.

Proses Penyerahan Kandidat Otomatisasi

Setelah Anda mengidentifikasi *test case* mana yang akan diotomatisasi, Anda perlu menyerahkan daftar ini kepada tim *engineering*. Di Thomas, ini dilakukan dalam sebuah *ticket* terpisah di Notion dengan label *automation-pending*. *Ticket* ini akan berisi daftar ID *test case* yang ditargetkan untuk otomatisasi, serta tautan ke *test case* aslinya.

Contoh *Ticket* Notion untuk Otomatisasi:

Otomatisasi Test Case: Fitur Login Pengguna

****Owner:**** <Nama Developer Otomatisasi>

****Reporter:**** qa-thomas

****Labels:**** `automation-pending`, `feature-login`

****Linked Epic:**** <ID-Epic-Fitur-Login>

****Deskripsi:****

Berikut adalah daftar test case untuk fitur Login Pengguna yang diidentifikasi sebagai kandidat otomat

****Daftar Test Case untuk Otomatisasi:****

- TC-P-001: Pengguna terverifikasi berhasil login dan melihat dashboard

- Prioritas: P0

- Tautan TC: [Link ke Notion TC-P-001]

- TC-N-001: Login gagal dengan password yang salah

- Prioritas: P1

- Tautan TC: [Link ke Notion TC-N-001]

- TC-E-004: Login dari negara baru (memicu geofencing)

- Prioritas: P1

- Tautan TC: [Link ke Notion TC-E-004]

- TC-DP-001: Seed pengguna uji standar dengan data dashboard 7 hari

- Prioritas: P0

- Tautan TC: [Link ke Notion TC-DP-001]

****Catatan:****

Prioritas P0/P1 wajib diotomatisasi. TC P2/P3 akan diuji manual kecuali ada waktu dan effort minimal.

Terima kasih.

Dengan mengidentifikasi kandidat otomatisasi di tahap *Test Design*, Anda tidak hanya membantu memastikan cakupan pengujian yang efisien, tetapi juga mendorong mentalitas "otomatisasi terlebih dahulu" dalam tim, yang pada akhirnya akan mempercepat siklus rilis dan meningkatkan kualitas produk secara keseluruhan.

5. Gerbang Menuju Eksekusi (Gate to Test Execution)

Selamat! Anda telah melalui proses *Test Design* yang komprehensif. Sebelum kita melangkah ke bab berikutnya, yaitu *Test Execution*, penting untuk diingat bahwa *Test Design* adalah sebuah gerbang. Ada beberapa kriteria yang harus dipenuhi sebelum Anda dapat secara resmi memulai fase eksekusi pengujian. Ini memastikan bahwa Anda memulai pengujian dengan fondasi yang kokoh dan mengurangi risiko pemborosan waktu.

Berikut adalah *checklist* yang harus Anda pastikan terpenuhi:

[] Dokumen *Test Strategy* telah disetujui:

- Ini berarti *scope*, pendekatan, lingkungan, serta *entry* dan *exit criteria* telah disepakati oleh semua *stakeholder* terkait (PM, Tech Lead, dll.).
- Pastikan tidak ada pertanyaan atau keberatan yang belum terselesaikan mengenai rencana pengujian Anda.

[] Keempat kategori *test case* telah lengkap:

- Pastikan Anda telah menulis *test case* untuk *Positive Cases*, *Negative Cases*, *Edge Cases*, dan *Data Preparation*.
- Jangan ada kategori yang terlewat, karena setiap kategori memiliki perannya masing-masing dalam memastikan cakupan pengujian yang menyeluruh.

[] Estimasi waktu telah ditinjau dan disetujui oleh PM/Tech Lead:

- Estimasi waktu yang Anda buat dalam *Test Strategy* dan per *test case* harus sudah ditinjau dan disetujui oleh Project Manager dan/atau Tech Lead.
- Ini penting untuk perencanaan *sprint* dan alokasi sumber daya. Jika ada perbedaan, diskusikan hingga mencapai kesepakatan.

[] Rencana persiapan data uji sudah siap:

- Anda harus memiliki pemahaman yang jelas tentang bagaimana data yang dibutuhkan untuk setiap *test case* akan disiapkan.
- Apakah itu melalui *script seeding*, pembuatan manual, atau melalui API? Pastikan semua metode sudah jelas dan siap dieksekusi.

[] Kandidat otomatisasi telah diidentifikasi dan dikomunikasikan:

- Daftar *test case* yang menjadi kandidat untuk otomatisasi telah Anda tentukan dan diserahkan kepada tim *engineering* (misalnya, melalui *ticket* Notion).
- Ini memastikan bahwa tim *development* dapat mulai merencanakan atau bahkan memulai pengembangan *automation script* secara paralel.

Memenuhi semua kriteria ini adalah tanda bahwa Anda siap untuk beralih ke fase *Test Execution* dengan kepercayaan diri dan efisiensi. Anda telah melakukan pekerjaan rumah Anda, dan sekarang saatnya untuk melihat bagaimana aplikasi benar-benar bekerja!

Rangkuman

Bab ini telah membawa Anda melampaui analisis persyaratan dan masuk ke jantungnya peran QA: *Test Design*. Kita telah mempelajari bahwa menjadi QA Engineer bukan sekadar "klik-klik tombol", melainkan merancang sebuah strategi yang kokoh untuk memastikan kualitas.

Berikut poin-poin penting yang Anda pelajari:

- **Test Strategy:** Anda kini memahami pentingnya *Test Strategy* sebagai peta jalan pengujian, yang mencakup *scope*, pendekatan, lingkungan, *entry/exit criteria*, estimasi waktu, dan manajemen risiko. Dokumen ini adalah jembatan komunikasi antara QA dan *stakeholder* lainnya.
- **Estimasi Waktu:** Anda belajar cara menyusun estimasi waktu yang realistis untuk setiap fase pengujian, termasuk *buffer*, dan mengapa ini krusial untuk perencanaan proyek. Estimasi ini harus ada di *Test Strategy* dan juga *per test case*.
- **Empat Kategori Test Case:** Anda telah menguasai empat pilar *Test Case* – *Positive*, *Negative*, *Edge*, dan *Data Preparation* – lengkap dengan contoh konkret untuk fitur login. Memastikan cakupan di keempat kategori ini akan memberikan Anda kepercayaan diri yang lebih besar terhadap kualitas produk.
- **Format dan Penyimpanan Test Case:** Anda kini tahu bagaimana menyimpan *test case* secara terorganisir di Notion atau CSV, menggunakan format standar yang memudahkan pelacakan dan kolaborasi.

- **Kandidat Otomatisasi:** Anda telah belajar cara mengidentifikasi *test case* mana yang menjadi kandidat utama untuk otomatisasi (P0/P1 wajib, P2 kondisional, P3 manual), sebuah langkah proaktif yang menghemat waktu dan sumber daya di masa depan.

- **Quality Gate:** Anda memahami bahwa *Test Design* adalah *Quality Gate* yang krusial. Memastikan semua kriteria *Test Design* terpenuhi sebelum masuk ke *Test Execution* akan menghemat banyak masalah di kemudian hari.

Dengan bekal *Test Design* ini, Anda tidak hanya siap untuk menguji, tetapi juga untuk memimpin inisiatif kualitas dalam tim Anda. Anda telah mengubah persyaratan abstrak menjadi rencana pengujian yang konkret dan dapat dieksekusi.

Checklist Actionable

Berikut adalah daftar langkah-langkah yang dapat Anda terapkan segera setelah membaca bab ini:

- [] **Buat *template Test Strategy*** di Notion atau alat manajemen proyek pilihan Anda, berdasarkan *template* yang telah diberikan.
- [] **Pilih satu fitur yang sedang atau akan dikembangkan** di proyek Anda.
- [] **Susun *Test Strategy* lengkap** untuk fitur tersebut, isi setiap bagian: *Scope*, *Approach*, *Environments*, *Entry/Exit Criteria*, *Estimasi Waktu*, dan *Risks*.
- [] **Lakukan estimasi waktu secara detail** untuk setiap komponen dalam *Test Strategy* Anda (penyusunan TC, eksekusi, regresi, dll.), dan pastikan ada *buffer* 20%.
- [] **Mulai tulis *Test Case* untuk fitur tersebut**, pastikan Anda mencakup keempat kategori: *Positive*, *Negative*, *Edge*, dan *Data Preparation*. Gunakan fitur Login sebagai inspirasi.
- [] **Sertakan estimasi waktu (dalam menit) untuk setiap *test case*** yang Anda buat.
- [] **Pastikan setiap *test case* tertaut ke *User Story* dan *Acceptance Criteria*** yang relevan.
- [] **Identifikasi kandidat otomatisasi** untuk *test case* yang baru Anda buat, tandai P0/P1 sebagai wajib otomatisasi.

- [] **Simpan *test case* Anda di Notion** (sebagai tugas terpisah atau *body markdown* di tugas induk) atau dalam format CSV standar.
 - [] **Presentasikan *Test Strategy* dan *Test Case* Anda** kepada PM dan Tech Lead untuk mendapatkan *feedback* dan persetujuan, pastikan semua *Entry Criteria* terpenuhi sebelum melangkah ke *Test Execution*.
-

Chapter 03: Test Execution — Smoke, Functional, Exploratory, Regression

Selamat datang di jantung operasi Quality Assurance! Jika pada bab sebelumnya kita telah belajar cara membaca *requirement* dan merancang strategi serta *test case* yang solid, kini saatnya kita beraksi. Chapter ini akan membawa Anda melalui tahapan eksekusi tes, yaitu momen di mana Anda benar-benar menguji aplikasi, menemukan *bug*, dan memvalidasi kualitas.

Eksekusi tes bukan sekadar "klik sana-sini" atau "menjalankan daftar tugas". Ini adalah proses strategis yang membutuhkan pemikiran kritis, observasi tajam, dan kemampuan untuk bertindak cepat. Kita akan membahas urutan eksekusi yang efisien, mengapa urutan tersebut penting, bagaimana menangani setiap jenis pengujian, hingga cara melaporkan hasil dan *bug* yang ditemukan. Tujuan utama kita adalah mengadopsi prinsip *fail-fast*: menemukan masalah secepat mungkin agar tim pengembangan dapat memperbaikinya tanpa menunda proses lebih lanjut.

Mari kita selami bagaimana seorang QA Engineer yang efektif menjalankan pengujian!

Mengapa Urutan Eksekusi Tes Itu Penting? — Konsep Fail-Fast

Bayangkan Anda sedang membangun sebuah rumah. Apakah Anda akan langsung memasang atap sebelum memastikan pondasinya kokoh? Tentu tidak. Anda akan memulai dengan pondasi,

lalu dinding, kemudian rangka atap, dan seterusnya. Setiap langkah dibangun di atas keberhasilan langkah sebelumnya.

Konsep *fail-fast* dalam pengujian memiliki filosofi yang sama. Kita ingin menemukan masalah paling fundamental dan kritis secepat mungkin. Mengapa? 1. **Menghemat Waktu dan Sumber Daya:** Jika ada masalah serius di fungsionalitas dasar aplikasi, melanjutkan pengujian fungsional yang lebih mendalam akan membuang-buang waktu. Lebih baik berhenti, melaporkan, dan menunggu perbaikan. 2. **Meningkatkan Efisiensi:** Dengan memastikan dasar-dasar berfungsi, kita dapat lebih percaya diri saat melangkah ke pengujian yang lebih kompleks, fokus pada detail tanpa khawatir fondasinya rapuh. 3. **Mempercepat Umpan Balik:** Semakin cepat *bug* ditemukan dan dilaporkan, semakin cepat pengembang dapat memperbaikinya, mengurangi *cycle time* pengembangan secara keseluruhan.

Untuk mencapai prinsip *fail-fast*, kita akan mengikuti urutan eksekusi tes yang telah terbukti efektif. Urutan ini dirancang untuk secara progresif meningkatkan kedalaman dan cakupan pengujian, dimulai dari pemeriksaan paling dasar hingga eksplorasi yang lebih luas dan verifikasi regresi.

Berikut adalah urutan eksekusi tes yang akan kita ikuti:

1. **Automation Smoke Testing:** Memastikan build dasar berfungsi.
2. **Functional Testing:** Memverifikasi setiap *user story* atau fitur.
3. **Automation Exploratory Testing:** Mencari *edge cases* dan perilaku tak terduga secara terprogram.
4. **Regression Testing:** Memastikan tidak ada fungsionalitas lama yang rusak akibat perubahan baru.

Mari kita bahas masing-masing tahapan ini secara detail.

1. Automation Smoke Testing: Penjaga Gerbang Pertama

Smoke Testing, kadang disebut juga *Build Verification Test (BVT)*, adalah jenis pengujian awal yang dilakukan untuk memastikan bahwa *build* perangkat lunak yang baru di-deploy stabil dan fungsionalitas utamanya bekerja. Anggap saja ini sebagai "penjaga gerbang" pertama yang menentukan apakah *build* tersebut layak untuk diuji lebih lanjut.

Apa yang Diperiksa? * Aplikasi dapat diakses (misalnya, halaman login terbuka). * Fungsionalitas inti (misalnya, login, registrasi, membuat item dasar) berfungsi. * Tidak ada *error* fatal yang menyebabkan aplikasi *crash*.

Tujuan Utama: * Mencegah tim QA membuang waktu menguji *build* yang rusak secara fundamental. * Memberikan umpan balik instan kepada tim *engineering* jika ada masalah kritis pada *build* terbaru.

Kapan Dijalankan? *Smoke Testing* idealnya dijalankan secara otomatis setiap kali ada *deploy* baru ke lingkungan pengembangan (misalnya, *staging* atau *dev*). Ini memastikan bahwa setiap perubahan kode yang masuk tidak merusak fungsionalitas dasar.

Contoh Otomatisasi Smoke Test Biasanya, *smoke test* adalah *suite* yang relatif kecil namun penting. Anda dapat menjalankannya menggunakan *framework* pengujian yang relevan dengan *stack* teknologi proyek Anda.

```
# Contoh untuk proyek Python dengan Pytest
pytest tests/smoke/ --maxfail=1 -q

# Contoh untuk proyek JavaScript/Node.js
npm run test:smoke

# Contoh lain jika menggunakan Playwright (akan dibahas lebih lanjut di Chapter 07)
npx playwright test --grep "@smoke"
```

Perintah `--maxfail=1` pada `pytest` berarti pengujian akan berhenti setelah kegagalan pertama, ini sangat cocok dengan prinsip *fail-fast* untuk *smoke test*.

Apa yang Harus Dilakukan Jika Smoke Test Gagal?

Ini adalah poin krusial yang harus Anda ingat:

- **JANGAN LANJUTKAN:** Jika *smoke test* gagal, itu berarti *build* tidak stabil untuk pengujian lebih lanjut. Semua pengujian lain yang Anda lakukan setelahnya kemungkinan besar akan menemukan lebih banyak *bug* yang disebabkan oleh masalah dasar ini, yang pada akhirnya membuang waktu Anda dan tim.
- **Laporkan Segera:** Prioritas utama Anda adalah memberi tahu tim *engineering* secepat mungkin. Komunikasi instan sangat penting di sini.

Langkah-langkah Pelaporan Kegagalan Smoke Test:

1. **Identifikasi Masalah:** Lihat log *smoke test* untuk mengetahui bagian mana yang gagal.

2. **Berhenti Total:** Hentikan semua aktivitas pengujian lainnya pada *build* tersebut.

3. **Kirim Pesan ke Saluran #engineering:** Gunakan Slack atau *platform* komunikasi tim Anda untuk mengirim pesan ke saluran khusus *engineering*.

4. **Tag Tech Lead:** Pastikan Anda me-mention *Tech Lead* atau individu yang bertanggung jawab atas *deploy* agar mereka segera menyadari masalahnya.

Sertakan Detail Penting:

- Sebutkan bahwa *smoke test* gagal.
- Tautkan ke *build* atau *commit* yang bermasalah.
- Sebutkan modul atau fungsionalitas yang terdampak.
- Berikan tautan ke log *CI/CD* atau laporan *smoke test* yang lebih detail.

Contoh Pesan Slack untuk Kegagalan Smoke Test:

Halo tim #engineering,

Mohon perhatiannya, **smoke test** untuk **deploy** terbaru ke lingkungan **staging** [Link ke Build/Commit]

Modul yang terdampak:

- [Login Pengguna]
- [Registrasi Akun]

Log selengkapnya dapat dilihat di sini: [Link ke Log CI/CD atau Laporan Otomatisasi]

Saya akan menunda semua pengujian fungsional hingga masalah ini teratasi. Mohon segera dicek. @[Nama T]

Dengan tindakan cepat ini, Anda telah menjaga kualitas *pipeline* pengembangan dan mencegah pemborosan waktu yang berharga bagi tim QA.

2. Functional Testing: Memverifikasi Setiap Cerita Pengguna

Setelah *smoke test* berhasil dan Anda yakin *build* dasar berfungsi, langkah selanjutnya adalah *Functional Testing*. Di tahap ini, Anda akan memverifikasi bahwa setiap fitur atau *user story* yang telah dikembangkan bekerja sesuai dengan spesifikasi yang telah disepakati di *Product Requirement Document (PRD)* dan *test case* yang Anda rancang di Chapter 02.

Apa yang Diperiksa? * Setiap fungsionalitas yang disebutkan dalam *user story* atau *requirement*.
* Apakah sistem menghasilkan *output* yang benar untuk *input* yang diberikan. * Apakah semua skenario positif, negatif, dan *edge case* (sesuai *test case* Anda) memberikan hasil yang diharapkan.

Kapan Dijalankan? * Setelah *smoke test* berhasil. * Ketika fitur baru selesai dikembangkan dan di-*deploy* ke lingkungan *staging* atau *QA*.

Proses Melakukan Functional Testing:

1. **Ambil Test Case:** Gunakan daftar *test case* yang sudah Anda siapkan dari Chapter 02. Idealnya, *test case* ini sudah dikelompokkan per fitur atau *user story*.
2. **Eksekusi Step-by-Step:** Jalankan setiap langkah dalam *test case* secara berurutan.
3. **Verifikasi Hasil:** Bandingkan *actual result* dengan *expected result*.

Catat Status: Tandai status setiap *test case*. Status umum meliputi:

- **Pass:** Hasil aktual sesuai dengan yang diharapkan.
- **Fail:** Hasil aktual tidak sesuai dengan yang diharapkan (ada *bug*).
- **Blocked:** Tidak dapat menjalankan *test case* karena ada *bug* di fungsionalitas sebelumnya atau *dependency* lain.
- **Skipped:** *Test case* tidak relevan untuk *build* saat ini atau tidak perlu dijalankan karena alasan tertentu.

Kumpulkan Bukti (untuk Kegagalan): Jika sebuah *test case* gagal, sangat penting untuk mengumpulkan bukti yang cukup agar pengembang dapat mereproduksi dan memperbaiki *bug* tersebut. Bukti yang harus dikumpulkan meliputi:

- **Screenshot/Video:** Rekam layar atau ambil tangkapan layar saat *bug* terjadi. Video seringkali lebih baik untuk *bug* yang melibatkan interaksi dinamis.
- **Console Logs:** Buka *developer tools* di browser (biasanya F12) dan tangkap log di tab "Console". Ini bisa menunjukkan *error JavaScript* atau masalah di sisi *frontend*.

- **Network Logs:** Di *developer tools*, tab "Network" dapat menunjukkan permintaan API yang gagal atau respons yang tidak terduga.
- **Langkah Reproduksi (Repro Steps):** Tuliskan dengan sangat jelas dan terperinci langkah-langkah yang Anda lakukan untuk menemukan *bug* tersebut. Ini adalah hal terpenting!

Alat untuk Melacak Functional Testing:

Untuk melacak *test case* dan status eksekusinya, Anda bisa menggunakan berbagai alat:

- **Spreadsheet (Google Sheets/Excel):** Sederhana dan mudah digunakan untuk proyek kecil. Anda bisa membuat kolom untuk ID *Test Case*, Deskripsi, Status, Bukti, dan Catatan *Bug*.
- **Test Management Tools (Jira, TestRail, Zephyr, qTest):** Alat khusus yang dirancang untuk manajemen *test case*, *test execution*, dan pelaporan. Mereka menawarkan fitur yang lebih canggih seperti pelacakan riwayat, integrasi dengan *bug tracking*, dan laporan yang komprehensif.
- **Notion:** Untuk tim yang menggunakan Notion sebagai *workspace* terpusat, Anda bisa membuat *sub-table* atau *database* khusus untuk *test case* dan eksekusinya.

Contoh Pelacakan Test Case di Notion atau Spreadsheet:

ID Test Case	Deskripsi Test Case	Status	Bukti (Link)	Catatan Bug (Link)
TC-001	Login dengan kredensial email dan password valid	■ Pass	-	-
TC-002	Login dengan email tidak terdaftar	■ Fail	[SS/Video TC-002]	[BUG-005]

TC-003	Login dengan password salah	■ Pass	-	-
TC-004	Registrasi akun baru dengan data lengkap	■ Pass	-	-
TC-005	Registrasi akun baru dengan email sudah terdaftar	■ Fail	[SS/Video TC-005]	[BUG-006]
TC-006	Melihat daftar produk di halaman utama	■ Pass	-	-
TC-007	Menambahkan produk ke keranjang belanja	■ Blocked	-	[BUG-007]
TC-008	Checkout dengan metode pembayaran kartu kredit	■■ Skipped	Tidak relevan untuk build ini	-

Kolom "Catatan Bug (Link)" akan mengarah ke tiket *bug* yang Anda buat di sistem pelacakan *bug* (misalnya Notion, Jira), yang akan kita bahas lebih lanjut di Chapter 04.

Dengan melakukan *functional testing* secara sistematis, Anda memastikan bahwa setiap bagian dari aplikasi bekerja sesuai tujuan, dan Anda memiliki bukti yang kuat untuk setiap masalah yang ditemukan.

3. Automation Exploratory Testing: Berburu Kasus Edge yang Tersembunyi

Setelah Anda memverifikasi fungsionalitas inti melalui *functional testing*, saatnya untuk berburu *bug* yang lebih sulit ditemukan: *edge cases*, perilaku tak terduga, atau masalah yang muncul dari kombinasi input yang tidak biasa. Ini adalah ranah *Exploratory Testing*.

Apa itu Exploratory Testing? Berbeda dengan *functional testing* yang mengikuti *test case* yang sudah ditentukan, *exploratory testing* adalah pendekatan pengujian yang tidak terstruktur, simultan dalam desain dan eksekusi. Artinya, Anda mendesain tes saat Anda menjalankannya, berdasarkan pengetahuan, pengalaman, dan intuisi Anda.

Penting: *Exploratory testing* **bukan** *box-ticking*. Ini adalah aktivitas "berburu". Anda tidak hanya menandai *pass/fail*, tetapi Anda menjelajahi, bereksperimen, dan mengamati perilaku sistem yang tidak terduga.

Tujuan Utama: * Menemukan *bug* yang tidak tercakup oleh *test case* formal. * Mengidentifikasi *edge cases* atau skenario yang tidak terpikirkan sebelumnya. * Memahami lebih dalam bagaimana aplikasi berperilaku di bawah kondisi yang berbeda. * Memberikan umpan balik tentang kegunaan (*usability*) dan pengalaman pengguna (*UX*).

Kapan Dijalankan? * Setelah *functional testing* selesai dan *build* dianggap cukup stabil. * Ketika Anda memiliki waktu luang setelah pengujian utama, atau ketika Anda ingin memahami area baru dari aplikasi.

Time-Box Exploratory Testing: Karena sifatnya yang tidak terstruktur, *exploratory testing* dapat menjadi *black hole* waktu jika tidak dibatasi. Sangat penting untuk menetapkan *time-box* atau batas waktu yang jelas. Misalnya: **4-8 jam per fitur atau area aplikasi.**

Dalam konteks "Automation Exploratory Testing", kita tidak hanya mengandalkan eksplorasi manual, tetapi juga memanfaatkan skrip otomatisasi untuk menghasilkan *input* secara terprogram dan mencari perilaku yang tidak terduga. Ini adalah perpaduan antara kecerdasan manusia dan kekuatan otomatisasi.

Teknik Automation Exploratory Testing:

Property-Based Testing:

- **Konsep:** Alih-alih menguji dengan *input* spesifik, Anda mendefinisikan "properti" yang harus selalu benar untuk fungsi Anda, terlepas dari *input* yang valid. *Framework* kemudian menghasilkan ribuan *input* acak (tetapi valid) untuk mencoba melanggar properti tersebut.
- **Contoh:** Jika Anda memiliki fungsi `tambah(a, b)`, properti bisa jadi `tambah(a, b) == tambah(b, a)` (komutatif) atau `tambah(a, 0) == a`. *Framework* akan mencoba berbagai angka `a` dan `b` untuk melihat apakah properti ini selalu berlaku.
- **Library:** Hypothesis (Python), QuickCheck (Haskell/Scala), js-jig (JavaScript).

Fuzzing Inputs to APIs:

- **Konsep:** *Fuzzing* melibatkan pengiriman *input* yang tidak valid, tidak terduga, atau semi-acak ke suatu sistem (misalnya, API, *form* web) untuk mencoba menyebabkan *crash*, *error*, atau perilaku yang tidak diinginkan.
- **Contoh:** Mengirim *string* yang sangat panjang ke kolom nama pengguna, karakter khusus ke kolom angka, JSON yang tidak valid ke *endpoint* API, atau nilai negatif ke kolom yang hanya menerima positif.
- **Tools:** Burp Suite (untuk aplikasi web), Postman (dengan koleksi yang dimodifikasi), atau skrip kustom menggunakan bahasa pemrograman favorit Anda.

Stress Sessions:

- **Konsep:** Mensimulasikan banyak pengguna bersamaan atau banyak permintaan ke sistem untuk menguji kinerja, stabilitas, dan bagaimana aplikasi menangani beban tinggi.
- **Tujuan:** Menemukan *bottleneck*, *memory leak*, atau *race conditions* yang hanya muncul di bawah tekanan.
- **Tools:** JMeter, k6, Locust, Gatling.

Contoh Skenario Exploratory Testing (Manual atau dengan Bantuan Script):

Form Input:

- Memasukkan karakter khusus (`!@#$%^&*()`) ke semua kolom teks.
- Memasukkan teks yang sangat panjang (ribuan karakter) ke kolom yang seharusnya singkat.

- Mencoba mengunggah file dengan ukuran sangat besar atau tipe yang tidak didukung.
- Mengubah *input* di *developer console* (misalnya, membuat *input* yang *disable* menjadi *enable*).

Alur Pengguna:

- Melakukan tindakan yang tidak biasa: misalnya, menambahkan item ke keranjang, lalu mengklik tombol *back* beberapa kali, lalu mencoba *checkout* lagi.
- Membuka banyak tab browser secara bersamaan dan melakukan tindakan yang berbeda di setiap tab.
- Menguji aplikasi di berbagai ukuran layar dan orientasi (mode *responsive*).
- Memutus koneksi internet di tengah-tengah transaksi penting.

Bagaimana Mencatat Hasil Exploratory Testing?

Karena sifatnya yang tidak terstruktur, Anda tidak akan memiliki *test case* yang *pass/fail*. Namun, Anda tetap harus mencatat apa yang Anda temukan:

- **Bug Baru:** Jika Anda menemukan *bug*, segera laporkan dengan format yang sama seperti *bug* dari *functional testing* (bukti, langkah reproduksi).
- **Ide Test Case Baru:** Eksplorasi sering kali mengungkapkan skenario yang belum Anda pikirkan. Catat ide-ide ini untuk dimasukkan ke dalam *test plan* Anda di masa depan.
- **Observasi:** Catat hal-hal menarik tentang perilaku sistem, area yang terasa rapuh, atau area yang membutuhkan pengujian lebih lanjut.
- **Sesi Jurnal:** Beberapa QA Engineer suka membuat jurnal singkat selama sesi eksplorasi, mencatat apa yang mereka lakukan, apa yang mereka amati, dan apa yang mereka pelajari.

Exploratory testing adalah salah satu area di mana kreativitas dan intuisi seorang QA Engineer sangat bersinar. Dengan menggabungkannya dengan otomatisasi, Anda dapat memperluas jangkauan pencarian *bug* Anda secara signifikan.

4. Regression Testing: Memastikan Fungsionalitas Lama Tetap Berjalan

Setelah *functional testing* dan *exploratory testing* untuk fitur baru selesai, ada satu tahap krusial lagi sebelum *build* siap rilis: *Regression Testing*.

Apa itu Regression Testing? *Regression Testing* adalah proses pengujian yang dilakukan untuk memastikan bahwa perubahan baru pada kode (misalnya, penambahan fitur baru, perbaikan *bug*, atau perubahan konfigurasi) tidak merusak fungsionalitas yang sudah ada dan sebelumnya bekerja dengan baik.

Mengapa Penting? Dalam pengembangan perangkat lunak, sangat mudah bagi perubahan di satu bagian kode untuk secara tidak sengaja memengaruhi bagian lain yang tampaknya tidak terkait. Ini disebut "regresi". Aplikasi modern sering kali memiliki banyak *shared code paths* (jalur kode yang digunakan oleh berbagai fitur). Fitur baru atau perbaikan *bug* dapat memperkenalkan *bug* baru di area lain, dan *regression testing* adalah jaring pengaman kita untuk menangkapnya.

Kapan Dijalankan? * Sebelum setiap rilis besar (major release). * Setelah implementasi fitur baru atau perbaikan *bug* yang signifikan. * Secara berkala, misalnya, setiap minggu atau setiap dua minggu, untuk memastikan stabilitas aplikasi secara keseluruhan.

Pendekatan untuk Regression Testing:

Ada dua pendekatan utama untuk *regression testing*: otomatis dan manual. Idealnya, sebagian besar *regression test* harus diotomatisasi untuk efisiensi.

a. Automated Regression Suite

Jika Anda memiliki *suite* pengujian otomatis yang komprehensif (seperti yang akan kita bahas di Chapter 07), ini adalah aset terbesar Anda untuk *regression testing*.

Cara Melakukan: Cukup jalankan *suite* otomatisasi regresi Anda.

```
# Contoh untuk proyek Python dengan Pytest
pytest tests/regression/ -n 4 --html=reports/regression.html

# Contoh untuk proyek JavaScript/Playwright
npx playwright test --grep "@regression" --project=chromium --output=reports/
```

- `pytest tests/regression/`: Menjalankan semua tes di folder `tests/regression/`.
- `-n 4`: Menjalankan tes secara paralel menggunakan 4 proses (mempercepat eksekusi).

- `--html=reports/regression.html`: Menghasilkan laporan dalam format HTML.
- `--grep "@regression"`: Menjalankan tes yang diberi tag `@regression`.

Keuntungan Otomatisasi: * **Cepat:** Dapat dijalankan dalam hitungan menit atau jam, bukan hari. * **Konsisten:** Selalu menjalankan langkah yang sama persis setiap kali. * **Dapat Diulang:** Dapat dijalankan berulang kali tanpa kelelahan. * **Skalabilitas:** Mudah untuk menambahkan lebih banyak tes tanpa peningkatan biaya linier.

b. Manual Regression Checklist

Untuk kasus di mana otomatisasi belum sepenuhnya diterapkan, atau untuk *flow* yang sangat kompleks yang sulit diotomatisasi, Anda dapat menggunakan *checklist* regresi manual.

Cara Membuat dan Menggunakan: 1. **Identifikasi Alur Kunci:** Buat daftar alur pengguna atau fungsionalitas inti yang harus selalu berfungsi. Contoh: * Login dan Logout pengguna. * Proses *checkout* (jika aplikasi *e-commerce*). * Membuat, mengedit, dan menghapus item dasar (jika aplikasi CRUD). * Pencarian dan filter. * Memuat halaman *dashboard* utama. * Integrasi pembayaran (jika ada). 2. **Dokumentasikan di Notion:** Simpan *checklist* ini di Notion atau alat manajemen proyek serupa dengan label khusus (misalnya, `Labels=["regression-checklist"]`). 3. **Jalankan Secara Periodik:** Secara manual ikuti setiap item di *checklist* dan verifikasi fungsionalitasnya. 4. **Catat Status:** Tandai *pass/fail* dan laporkan *bug* yang ditemukan seperti biasa.

Contoh Manual Regression Checklist di Notion:

```
# Regression Checklist: Aplikasi E-commerce Thomas
```

```
**Status:** [Dalam Proses / Selesai]
```

```
**Terakhir Dijalankan:** 2023-10-26 (v1.5.0-rc1)
```

```
---
```

```
## Fungsionalitas Akun Pengguna
```

```
- [ ] Login dengan kredensial valid
```

```
- [ ] Login dengan kredensial tidak valid (email/password salah)
```

```
- [ ] Registrasi akun baru
```

```
- [ ] Reset password melalui email
```

```
- [ ] Mengubah informasi profil (nama, alamat)
```

```
- [ ] Logout
```

```
## Fungsionalitas Produk
```

```
- [ ] Melihat daftar produk di halaman utama
- [ ] Melihat detail produk
- [ ] Menggunakan fitur pencarian produk
- [ ] Menggunakan filter dan sortir produk
|
## Fungsionalitas Keranjang & Checkout
- [ ] Menambahkan produk ke keranjang belanja
- [ ] Mengubah kuantitas produk di keranjang
- [ ] Menghapus produk dari keranjang
- [ ] Melakukan checkout dengan metode pembayaran kartu kredit
- [ ] Melakukan checkout dengan metode pembayaran transfer bank
- [ ] Melihat riwayat pesanan setelah checkout berhasil
|
## Fungsionalitas Umum
- [ ] Memuat halaman utama tanpa error
- [ ] Navigasi antar halaman (header, footer)
- [ ] Responsivitas di perangkat mobile (uji beberapa resolusi kunci)
- [ ] Tidak ada error di Console Browser
|
---
**Catatan:** Jika ada item yang gagal, buat tiket bug dan tautkan di sini.
```

Regression testing, baik otomatis maupun manual, adalah investasi waktu yang sangat penting untuk menjaga kualitas produk seiring waktu. Tanpa ini, kita berisiko memperkenalkan *bug* baru setiap kali ada perubahan, yang dapat merusak kepercayaan pengguna dan reputasi produk.

Pelaporan Eksekusi Tes: Transparansi Hasil

Setelah Anda selesai menjalankan *smoke*, *functional*, *exploratory*, dan *regression testing*, langkah selanjutnya adalah menyusun laporan eksekusi tes. Laporan ini adalah ringkasan dari semua aktivitas pengujian yang telah Anda lakukan, status kualitas aplikasi, dan *bug* yang ditemukan. Ini adalah cara Anda mengkomunikasikan hasil kerja keras Anda kepada tim proyek (PM, *engineering*, *stakeholder* lainnya).

Tujuan Laporan Eksekusi Tes:

- Memberikan gambaran yang jelas dan ringkas tentang status kualitas *build* atau fitur yang diuji.
- Mengidentifikasi area yang stabil dan area yang memerlukan perhatian lebih lanjut.
- Menjadi dasar untuk keputusan "Siap Rilis" atau "Tidak Siap Rilis".

- Mencatat *bug* yang ditemukan dan prioritasnya.

Struktur Laporan Standar:

Berikut adalah struktur laporan eksekusi tes yang efektif dan informatif, yang dapat Anda adaptasi sesuai kebutuhan proyek Anda. Anda bisa menyusunnya dalam format Markdown, dokumen Notion, atau bahkan email/pesan Slack yang lebih ringkas.

```
# Test Execution Report – [Nama Fitur/Build] – [Tanggal Eksekusi]
```

```
**Tanggal:** 2023-10-26
```

```
**Lingkungan:** Staging v1.5.0-rc1
```

```
**Reporter:** QA Thomas
```

Ringkasan Eksekusi Tes

Ringkasan ini memberikan gambaran cepat tentang keseluruhan status pengujian.

```
- **Total Test Cases:** 50
```

```
- **Passed:** 42 (84%)
```

```
- **Failed:** 5
```

```
- **Blocked:** 2
```

```
- **Skipped:** 1
```

```
**Catatan:** Fungsionalitas utama [Sebutkan Fungsionalitas] telah diverifikasi dan berfungsi dengan baik
```

Bug yang Ditemukan

Daftar **bug** yang ditemukan selama siklus eksekusi ini, dikelompokkan berdasarkan prioritas. Sertakan

```
- **P0 (Critical):** 1
```

```
* [BUG-001] Login button tidak responsif setelah percobaan gagal pertama - [Link ke Notion Bug]
```

```
- **P1 (High):** 1
```

```
* [BUG-002] Error 500 saat menambahkan item ke keranjang dari halaman produk - [Link ke Notion Bug]
```

```
- **P2 (Medium):** 2
```

```
* [BUG-003] Tampilan footer pecah di resolusi mobile tertentu - [Link ke Notion Bug]
```

```
* [BUG-004] Validasi input email tidak mendeteksi format email yang salah - [Link ke Notion Bug]
```

```
- **P3 (Low):** 1
```

```
* [BUG-005] Typo pada tooltip "Tambah ke Keranjang" - [Link ke Notion Bug]
```

Rekomendasi

Berdasarkan hasil eksekusi tes dan status **bug** yang ditemukan, berikan rekomendasi mengenai kesiapan

```
- [ ] Siap untuk rilis
```

```
- [ ] Siap secara kondisional (harus memperbaiki: [BUG-001], [BUG-002] sebelum rilis)
```

```
- [X] TIDAK siap untuk rilis
```

Risiko / Observasi Tambahan

Catat risiko potensial, area yang mungkin memerlukan pengujian lebih lanjut, atau observasi penting lainnya.

- Ditemukan penurunan kinerja ringan pada halaman daftar produk saat memuat lebih dari 100 item. Perlu diteliti lebih lanjut.
- Integrasi pihak ketiga [Nama Integrasi] belum diuji secara end-to-end karena *staging environment*.
- Beberapa *edge cases* terkait input form (misalnya, karakter khusus) belum sepenuhnya diuji karena keterbatasan waktu.

Kapan dan Bagaimana Mendistribusikan Laporan:

- **Setelah Setiap Siklus Eksekusi:** Idealnya, laporan ini dibuat setelah setiap siklus eksekusi yang signifikan (misalnya, setelah pengujian fitur utama selesai, atau di akhir hari jika pengujian berlangsung beberapa hari).

Saluran Komunikasi:

- **Slack #qa:** Posting ringkasan laporan ke saluran QA tim Anda. Ini menjaga semua orang tetap *aware* dan memberikan transparansi.
- **Centralized Report:** Simpan laporan lengkap di lokasi terpusat seperti Google Drive (untuk laporan HTML otomatisasi), Notion, atau *wiki* tim Anda. Berikan tautan ke laporan lengkap di pesan Slack Anda.
- **Meeting / Demo:** Siapkan ringkasan untuk dibahas dalam *daily stand-up*, *sprint review*, atau *demo* fitur.

Dengan laporan yang terstruktur dengan baik, Anda tidak hanya melaporkan angka, tetapi juga memberikan konteks, rekomendasi, dan wawasan yang berharga bagi tim proyek. Ini adalah salah satu cara utama QA Engineer menunjukkan nilai strategis mereka.

Routing Bug: Memastikan Bug Sampai ke Tangan yang Tepat

Menemukan *bug* adalah satu hal, tetapi memastikan *bug* tersebut ditangani dengan benar dan oleh orang yang tepat adalah hal lain. Di sinilah peran "Routing Bug" menjadi sangat penting. Sistem *routing* yang efektif memastikan bahwa *bug* dengan prioritas tinggi segera mendapatkan perhatian, sementara *bug* dengan prioritas lebih rendah masuk ke *backlog* untuk ditinjau.

Pentingnya Routing yang Benar:

- **Efisiensi:** Mencegah *bug* kritis terabaikan dan *bug* minor mengganggu alur kerja pengembang.
- **Prioritas:** Memastikan sumber daya tim *engineering* dialokasikan untuk masalah yang paling mendesak.
- **Transparansi:** Semua *stakeholder* tahu di mana mencari *bug* dan statusnya.

Sistem Prioritas Bug (P0-P3)

Kita akan menggunakan skala prioritas umum: P0, P1, P2, P3. Definisi ini mungkin sedikit bervariasi antar tim, tetapi intinya sama.

P0 (Critical / Blocker):

- **Definisi:** Menghentikan fungsionalitas inti aplikasi, sistem tidak dapat digunakan, atau terjadi kehilangan data yang signifikan. Tidak ada *workaround*.
- **Contoh:** Pengguna tidak bisa login sama sekali, aplikasi *crash* saat startup, proses pembayaran selalu gagal.

Routing:

1. Buat tiket *bug* di Notion (atau Jira).
2. Posting peringatan ke saluran Slack **#incidents** (saluran untuk masalah mendesak).
3. **DM langsung Tech Lead** atau pengembang yang bertanggung jawab. Ini adalah *bug* yang membutuhkan perhatian instan.

- **Contoh DM Slack ke Tech Lead:** @TechLeadName, ada P0 bug baru: [BUG-001] Login button not responding. Ini sangat kritis dan memblokir pengujian. Mohon segera lihat. Link tiket: <Notion Bug Link>

P1 (High):

- **Definisi:** Fungsionalitas inti terpengaruh secara signifikan, tetapi mungkin ada *workaround* atau tidak sepenuhnya memblokir seluruh sistem. Degradasi serius pada pengalaman pengguna.
- **Contoh:** Fitur utama tidak bekerja di browser tertentu, *error* saat upload gambar tetapi pengguna masih bisa upload gambar lain, sebagian data tidak ditampilkan.

Routing:

1. Buat tiket *bug* di Notion.
2. Posting peringatan ke saluran Slack **#incidents**.
3. **DM langsung Tech Lead** atau pengembang yang bertanggung jawab. Meskipun ada *workaround*, ini tetap prioritas tinggi.

P2 (Medium):

- **Definisi:** Fungsionalitas non-inti terpengaruh, *minor issue*, *UI/UX glitch*, atau masalah yang tidak menghalangi penggunaan utama aplikasi.
- **Contoh:** Tampilan *footer* pecah di resolusi tertentu, validasi *input* yang kurang sempurna, *error* di *console* yang tidak memengaruhi fungsionalitas utama.

Routing:

1. Buat tiket *bug* di Notion.
2. Posting ke saluran Slack **#qa** atau saluran umum tim pengembangan untuk visibilitas. Tidak perlu DM langsung.

P3 (Low):

- **Definisi:** Masalah kosmetik, *typo*, perbaikan kecil, saran perbaikan, atau masalah yang dampaknya minimal dan tidak memengaruhi fungsionalitas.
- **Contoh:** Typo pada label, warna tombol sedikit melenceng dari desain, *alignment* teks yang sedikit bergeser.

Routing:

1. Buat tiket *bug* di Notion **saja**. *Bug* ini akan terlihat saat *backlog review* atau *sprint planning*, dan dapat diprioritaskan jika ada waktu atau relevan dengan *sprint* mendatang. Tidak perlu notifikasi Slack.

Flow Routing Bug Otomatis vs. Manual

Tim yang lebih maju mungkin memiliki skrip atau integrasi yang mengotomatisasi *routing* ini.

Contoh Skrip Otomatisasi (Konseptual, dari Source Material):

```
# scripts/qa_report_bug.py
# Skrip ini mengimplementasikan routing berbasis severity
# P0/P1 → Notion bug + #incidents Slack + DM Tech Lead
# P2      → Notion bug + #qa Slack
# P3      → Notion bug only (visible in backlog review)

# Cara penggunaan (dari terminal):
# python scripts/qa_report_bug.py --title "[login] Login button unresponsive" --severity P1 --assignee
```

Ekuivalen Manual (Menggunakan Python untuk Ilustrasi Konsep):

Jika Anda belum memiliki skrip otomatisasi penuh, Anda akan melakukan langkah-langkah ini secara manual menggunakan *tool* yang relevan (misalnya, membuat tiket di Notion secara manual, lalu mengirim pesan di Slack secara manual).

```
# Ini adalah ilustrasi bagaimana sebuah skrip akan bekerja,
# Anda akan melakukan langkah-langkah ini secara manual
# menggunakan Notion dan Slack secara terpisah.

# Asumsikan Anda memiliki konektor untuk Notion dan Slack
# (Ini hanya pseudocode untuk menjelaskan konsep)
from notion_connector import NotionTicketing
from slack_connector import SlackConnector

# Inisialisasi konektor (asumsikan sudah dikonfigurasi)
nt = NotionTicketing.from_env()
slack = SlackConnector.from_env()

# Data bug yang ditemukan
bug_title = "[login] Login button unresponsive after first failed attempt"
bug_reporter = "qa-thomas"
bug_severity = "P1" # Bisa P0, P1, P2, P3
bug_steps = """
1. Navigate to /login
2. Enter valid email, wrong password, click Sign in
3. Wait for error message
4. Click Sign in again (without changing inputs)
"""
bug_expected = "Login attempt re-submitted"
bug_actual = "Button does not respond; no network request fires"
bug_environment = "staging v1.4.2-rc3, Chrome 124, macOS"
bug_screenshots = ["https://drive.google.com/link_ke_screenshot_1", "https://drive.google.com/link_ke_
```

```
# 1. Buat tiket bug di Notion
```

```
bug_ticket = nt.create_bug(
```

```
    title=bug_title,
```

```
    reporter=bug_reporter,
```

```
    severity=bug_severity,
```

```
    steps_to_reproduce=bug_steps,
```

```
    expected=bug_expected,
```

```
    actual=bug_actual,
```

```
    environment=bug_environment,
```

```
    screenshots=bug_screenshots,
```

```
)
```

```
print(f"Bug created in Notion: {bug_ticket.url}")
```

```
# 2. Lakukan routing berdasarkan severity
```

```
if bug_severity == "P0" or bug_severity == "P1":
```

```
    # Post ke #incidents channel
```

```
    slack.post_message(
```

```
        channel="#incidents",
```

```
        text=f":warning: **P{bug_severity} bug filed**: {bug_title}",
```

```
        ticket_url=bug_ticket.url,
```

```
        priority=bug_severity,
```

```
    )
```

```
print(f"Posted to #incidents Slack channel for P{bug_severity} bug.")
```

```
# DM Tech Lead
```

```
slack.dm_user(
```

```
    user_id="U123ABC", # ID Slack Tech Lead
```

```
    text=f":alert: **P{bug_severity} bug requires immediate attention**: {bug_title}. Link: {bug_ticket.url}"
```

```
)
```

```
print(f"DM'd Tech Lead for P{bug_severity} bug.")
```

```
elif bug_severity == "P2":
```

```
    # Post ke #qa channel
```

```
    slack.post_message(
```

```
        channel="#qa",
```

```
        text=f":bug: **P2 bug filed**: {bug_title}",
```

```
        ticket_url=bug_ticket.url,
```

```
        priority=bug_severity,
```

```
    )
```

```
print("Posted to #qa Slack channel for P2 bug.")
```

```
elif bug_severity == "P3":
```

```
    print("P3 bug filed in Notion only. Will be reviewed in backlog.")
```

Melalui sistem *routing* ini, Anda tidak hanya menemukan *bug*, tetapi juga memastikan bahwa setiap *bug* mendapatkan visibilitas dan penanganan yang sesuai dengan tingkat urgensinya. Ini adalah bagian integral dari menjaga alur kerja tim pengembangan tetap efisien dan menjaga kualitas produk secara keseluruhan.

Laporan Terpusat dan Sign-off: Akhir dari Siklus Eksekusi

Setelah semua siklus eksekusi tes (smoke, fungsional, eksplorasi, regresi) selesai, laporan eksekusi tes telah disusun, dan semua *bug* telah di-route, ada dua langkah terakhir yang penting: membuat "Laporan Terpusat" dan melakukan "Sign-off" untuk menyatakan kesiapan rilis.

Laporan Terpusat (Centralized Report)

Laporan terpusat adalah ringkasan progres harian atau ringkasan akhir siklus pengujian yang dibagikan kepada seluruh tim proyek, terutama Product Manager (PM) dan tim *engineering*. Ini biasanya berupa pesan singkat di saluran Slack yang berisi poin-poin penting dari laporan eksekusi tes yang lebih detail.

Tujuan Laporan Terpusat:

- Memberikan *update* reguler tentang status pengujian kepada semua *stakeholder*.
- Menjaga transparansi dan memastikan semua orang memiliki pemahaman yang sama tentang kualitas *build*.
- Membantu PM dan *Tech Lead* dalam mengambil keputusan terkait rilis.

Konten Laporan Terpusat (Biasanya Dikirim ke Saluran #qa):

- **Progres Tes:** Berapa banyak *test case* yang telah dijalankan dari total yang direncanakan (misalnya, X dari Y *test case* telah dijalankan).
- **Bug Baru yang Ditemukan:** Jumlah *bug* baru yang diajukan, dikelompokkan berdasarkan *severity*.
- **Bloker yang Terbuka:** Jika ada *bug* P0 atau P1 yang masih terbuka dan memblokir pengujian lebih lanjut.

- **Estimasi Kesiapan Rilis (ETA to "ready for release"):** Perkiraan kapan *build* akan siap untuk rilis, berdasarkan progres dan *bug* yang tersisa.
- **Tautan ke Laporan Lengkap:** Selalu sertakan tautan ke laporan eksekusi tes yang lebih detail (misalnya, dokumen Notion, laporan HTML otomatisasi di Google Drive).

Contoh Pesan Slack (Laporan Harian/Akhir Siklus):

Halo Tim #qa,

Berikut adalah update QA untuk [Nama Fitur/Build] (v1.5.0-rc1) per 2023-10-26:

****Progres Tes:****

- Total 50 test case, 45 sudah dijalankan (90%).
- Status: 42 Passed, 3 Failed, 0 Blocked, 0 Skipped.

****Bug Baru Ditemukan:****

- P0: 1 ([BUG-001] Login button unresponsive)
- P1: 1 ([BUG-002] Error 500 saat tambah item ke keranjang)
- P2: 2 ([BUG-003], [BUG-004])
- P3: 1 ([BUG-005])

****Blokir Terbuka:****

- [BUG-001] masih memblokir pengujian alur login lanjutan. Tim engineering sedang menanganinya.

****ETA Ready for Release:****

- Jika [BUG-001] dan [BUG-002] diperbaiki hari ini, saya perkirakan build ini bisa "Ready for Release"

Laporan eksekusi lengkap: [Link ke Notion Test Execution Report]

Laporan otomatisasi HTML: [Link ke Google Drive/S3 Bucket]

Mohon informasinya jika ada pertanyaan!

Sign-off: Pernyataan Kesiapan Rilis

"Sign-off" adalah momen formal di mana Anda, sebagai QA Engineer, secara resmi menyatakan bahwa *build* atau fitur telah memenuhi semua kriteria keluar (exit criteria) pengujian dan siap untuk rilis atau tahap selanjutnya (misalnya, *User Acceptance Testing - UAT*).

Kriteria Keluar (Exit Criteria) yang Umum:

- Semua *test case* kritis dan prioritas tinggi telah dijalankan dan *pass*.
- Semua *bug* P0 dan P1 telah diperbaiki dan diverifikasi.

- *Bug* P2 dan P3 yang tersisa telah ditinjau dan disepakati untuk ditunda ke *sprint* berikutnya atau diterima sebagai *known issue*.
- *Regression suite* telah dijalankan dan tidak ada regresi kritis yang ditemukan.
- Semua *stakeholder* yang relevan telah diberitahu dan setuju dengan status kesiapan.

Langkah-langkah Sign-off:

1. **Verifikasi Semua Kriteria Keluar:** Pastikan Anda telah memeriksa setiap poin dalam daftar kriteria keluar tim Anda.
2. **Perbarui Status Tiket Test Strategy:** Jika Anda melacak *test strategy* dalam sistem manajemen proyek (misalnya Notion), ubah status tiket tersebut menjadi **Done** atau **Ready for Release**.
3. **Hasilkan UAT Report (Opsional, tapi Direkomendasikan):** Jika proyek Anda melibatkan *User Acceptance Testing* (UAT), ini adalah waktu yang tepat untuk mulai menyusun UAT Report yang merangkum hasil pengujian Anda dan akan menjadi dasar bagi UAT oleh *end-user* atau PM. (Ini akan dibahas lebih detail di Chapter 05).
4. **Beritahu Product Manager (PM):** Kirim pesan eksplisit kepada PM (dan *Tech Lead*) yang menyatakan bahwa fitur atau *build* sudah "Ready for Release". Pesan ini harus jelas dan ringkas.

Contoh Pesan Slack ke PM untuk Sign-off:

Halo @PMThomas dan @TechLeadName,

Saya ingin menginformasikan bahwa fitur [Nama Fitur] (Link ke PRD/Epic) telah selesai melalui semua si

****Semua test case kritis dan prioritas tinggi telah PASS.****

****Semua bug P0 dan P1 telah diperbaiki dan diverifikasi.****

Bug P2 dan P3 yang tersisa telah disepakati untuk ditunda/diterima.

Berdasarkan hasil pengujian komprehensif, saya merekomendasikan fitur ini ****SIAP UNTUK RILIS.****

Detail laporan eksekusi lengkap dapat dilihat di sini: [Link ke Notion Test Execution Report]

Mohon informasinya jika ada pertanyaan lebih lanjut. Terima kasih!

Dengan melakukan pelaporan terpusat dan *sign-off* secara formal, Anda tidak hanya menyelesaikan tugas pengujian Anda, tetapi juga memainkan peran kunci dalam proses

pengambilan keputusan rilis, memastikan bahwa produk yang diluncurkan memiliki kualitas terbaik yang mungkin.

Rangkuman

Chapter ini telah membawa Anda melalui inti dari tugas seorang QA Engineer: eksekusi tes. Kita telah membahas bagaimana pendekatan yang terstruktur dan berprinsip *fail-fast* dapat menghemat waktu dan sumber daya tim.

Berikut poin-poin penting yang perlu Anda ingat:

- **Urutan Eksekusi Itu Penting:** Selalu ikuti urutan Smoke -> Functional -> Exploratory -> Regression untuk menemukan masalah paling kritis di awal dan mengoptimalkan efisiensi.
- **Smoke Test sebagai Penjaga Gerbang:** Jika *smoke test* gagal, segera hentikan pengujian, laporkan ke tim *engineering* (dan *Tech Lead*), dan jangan lanjutkan hingga masalah dasar teratasi.
- **Functional Testing:** Verifikasi setiap *user story* dengan *test case* yang telah dibuat, kumpulkan bukti yang solid untuk setiap kegagalan.
- **Exploratory Testing:** Lakukan "perburuan" *bug* yang tidak terduga dan *edge cases* dengan *time-box* yang jelas (4-8 jam), baik secara manual maupun dengan bantuan skrip otomatisasi. Ini bukan sekadar *box-ticking*.
- **Regression Testing:** Pastikan perubahan baru tidak merusak fungsionalitas lama, dengan memanfaatkan *suite* otomatisasi atau *checklist* manual.
- **Pelaporan yang Transparan:** Susun laporan eksekusi tes yang ringkas namun informatif, mencakup ringkasan, daftar *bug*, dan rekomendasi.
- **Routing Bug yang Efektif:** Tentukan prioritas *bug* (P0-P3) dan *route* ke saluran komunikasi yang tepat (DM *Tech Lead*, *#incidents*, *#qa*, atau hanya Notion) agar mendapatkan perhatian sesuai urgensinya.
- **Laporan Terpusat dan Sign-off:** Berikan *update* reguler kepada tim melalui laporan terpusat dan berikan *sign-off* formal ketika semua kriteria keluar terpenuhi, menyatakan bahwa *build* siap rilis.

Anda kini memiliki pemahaman yang kuat tentang bagaimana menjalankan pengujian secara strategis dan efisien. Di chapter berikutnya, kita akan menyelami lebih dalam tentang bagaimana melaporkan *bug* secara efektif, memastikan setiap *bug* dapat direproduksi dan ditangani.

Checklist Actionable

Berikut adalah daftar tindakan yang bisa Anda lakukan dan praktikkan setelah membaca chapter ini:

- [] **Pahami dan Terapkan Urutan Eksekusi:** Selalu mulai dengan *Smoke Testing*, lalu *Functional*, *Exploratory*, dan terakhir *Regression*.
- [] **Siapkan Skrip Smoke Test Otomatis:** Identifikasi fungsionalitas kunci aplikasi Anda dan buat skrip *smoke test* otomatis yang berjalan cepat.
- [] **Tentukan Protokol Kegagalan Smoke Test:** Buat *template* pesan Slack untuk melaporkan kegagalan *smoke test* ke `#engineering` dan pastikan Anda tahu siapa *Tech Lead* yang harus di-DM.
- [] **Gunakan Alat Pelacakan Tes:** Pilih *spreadsheet* atau *test management tool* (seperti Notion) untuk melacak status setiap *test case* selama *functional testing*.
- [] **Kumpulkan Bukti Komprehensif:** Saat menemukan *bug* di *functional testing*, selalu ambil *screenshot/video*, *console logs*, *network logs*, dan tulis *repro steps* yang jelas.
- [] **Alokasikan Waktu untuk Exploratory Testing:** Jadwalkan sesi *exploratory testing* dengan *time-box* (misalnya, 4-8 jam) setelah *functional testing* untuk setiap fitur baru.
- [] **Buat atau Perbarui Regression Checklist:** Identifikasi alur kunci di aplikasi Anda dan buat *checklist* regresi (manual atau otomatis) yang akan dijalankan secara berkala.
- [] **Adaptasi Template Laporan Eksekusi Tes:** Sesuaikan *template* laporan yang diberikan di chapter ini dengan kebutuhan proyek Anda dan gunakan secara konsisten.
- [] **Pahami Definisi Prioritas Bug (P0-P3):** Pastikan Anda dan tim Anda memiliki pemahaman yang sama tentang definisi setiap prioritas *bug*.
- [] **Tentukan Flow Routing Bug:** Latih diri Anda untuk merouting *bug* sesuai prioritasnya: P0/P1 ke `#incidents` + DM *Tech Lead*; P2 ke `#qa`; P3 ke Notion saja.

- [] **Siapkan Template Pesan Sign-off:** Buat *template* pesan Slack untuk memberitahu PM dan *Tech Lead* saat *build* atau fitur siap rilis.

Tentu, mari kita mulai menulis Chapter 04 dari buku "Journey QA Thomas".

Chapter 04: Bug Reporting — Severity, Template, dan Eskalasi

Selamat datang kembali dalam perjalanan Anda menjadi seorang Quality Assurance Engineer! Jika di bab sebelumnya kita sudah belajar bagaimana merancang strategi dan *test case* yang solid, maka di bab ini kita akan membahas salah satu momen paling krusial dan sering disalahpahami dalam siklus pengembangan: pelaporan *bug*.

Bagi sebagian orang, menemukan *bug* mungkin terasa seperti memenangkan lotre, tetapi bagi seorang QA Engineer profesional, menemukan *bug* hanyalah setengah dari pertempuran. Pertempuran sesungguhnya adalah bagaimana Anda mengomunikasikan *bug* tersebut secara efektif agar dapat dipahami, direproduksi, dan diperbaiki oleh tim *engineer*. Laporan *bug* yang baik bukan sekadar daftar masalah, melainkan peta jalan yang jelas bagi *developer* untuk menemukan akar masalah dan menyelesaikannya dengan efisien. Ingatlah *prime directive* QA Thomas: memverifikasi bahwa setiap alur yang diimplementasikan sesuai dengan *acceptance criteria user story*. Setiap penyimpangan dari kriteria tersebut, bahkan jika kodenya tidak *crash*, adalah sebuah *defect* yang harus dilaporkan.

Di bab ini, Anda akan belajar cara membuat laporan *bug* yang *actionable*, memahami skala prioritas *severity* (P0-P3), hingga alur eskalasi yang tepat agar *bug* Anda mendapatkan perhatian yang sesuai. Mari kita mulai!

Anatomi Laporan Bug yang Efektif: Lebih dari Sekadar "Ini Rusak"

Sebuah laporan *bug* yang baik adalah jembatan komunikasi antara QA dan *developer*. Ia harus ringkas, jelas, dan memberikan semua informasi yang dibutuhkan agar *developer* dapat mereproduksi masalah dan memperbaikinya tanpa harus bolak-balik bertanya. Di Thomas, kami memiliki standar *template* laporan *bug* yang memandu Anda untuk mengisi setiap detail penting.

Mari kita bedah setiap komponen dari *template* laporan *bug* standar:

```
# [<area>] <one-line summary>

**Severity**: P0 | P1 | P2 | P3
**Reporter**: qa-thomas
**Environment**: <build version, browser, OS, device>
**Reproducible**: Always | Intermittent | Once | Cannot reproduce
**Linked story**: <Notion task ID>

## Steps to Reproduce
1. ...
2. ...
3. ...

## Expected
<what should happen>

## Actual
<what actually happens>

## Evidence
- Screenshot: <Drive link>
- Video: <Drive link>
- Console log: <paste or link>
- Network log: <paste or link>

## Impact
- Users affected: <all | specific cohort | edge case>
- Workaround: <if any>
- Frequency: <X per Y users/sessions>

## Notes / hypothesis
<any pointers for the engineer who'll pick this up>
```

Mari kita lihat mengapa setiap bagian ini penting:

1. Judul (Title)

[<area>] <one-line summary>

Judul adalah kesan pertama. Ia harus singkat, deskriptif, dan langsung menunjukkan inti masalah serta area fungsional yang terpengaruh. * **<area>**: Bagian mana dari aplikasi yang terpengaruh (misalnya: [Login], [Search Page], [Checkout], [User Profile]). Ini membantu *developer* dengan cepat mengetahui tim atau bagian kode mana yang kemungkinan besar bertanggung jawab. * **<one-line summary>**: Ringkasan masalah dalam satu baris. Hindari kalimat yang terlalu umum seperti "Bug di halaman X". Lebih spesifik seperti "Validasi email tidak berfungsi" atau "Tombol 'Tambah ke Keranjang' tidak responsif".

Contoh Judul yang Baik: * [Login] Pengguna tidak bisa masuk dengan kredensial yang benar * [Search Page] Filter harga tidak mengembalikan hasil yang relevan * [Checkout] Total harga tidak terupdate setelah menghapus item * [User Profile] Avatar pengguna tidak tampil di profil

Contoh Judul yang Buruk: * [Bug] Halaman error (Terlalu umum) * Login rusak (Tidak ada area spesifik, terlalu informal) * Masalah di fitur baru (Tidak deskriptif sama sekali)

2. Severity

P0 | P1 | P2 | P3

Ini adalah indikator seberapa parah dampak *bug* terhadap pengguna atau sistem. Kita akan membahas rubrik *severity* ini secara mendalam di bagian selanjutnya, tetapi penting untuk menentukannya sejak awal.

3. Reporter

qa-thomas

Siapa yang melaporkan *bug* ini? Ini penting untuk komunikasi jika *developer* membutuhkan klarifikasi lebih lanjut. Di Thomas, kami menggunakan qa-thomas sebagai identitas standar.

4. Environment

<build version, browser, OS, device>

Informasi ini krusial untuk mereplikasi *bug*. Sebuah *bug* bisa saja muncul di satu *environment* (misalnya Chrome di Windows 10) tetapi tidak di *environment* lain (misalnya Firefox di macOS). *

build version: Versi *build* aplikasi yang Anda uji. Ini penting karena *bug* mungkin sudah diperbaiki di *build* yang lebih baru, atau baru muncul di *build* tertentu. * **browser:** Peramban web yang digunakan (Chrome, Firefox, Safari, Edge). * **os:** Sistem Operasi (Windows 10, macOS Ventura, Android 13, iOS 16). * **device:** Perangkat yang digunakan (Desktop, iPhone 14, Samsung Galaxy S23, iPad Pro).

Contoh: Staging v1.2.3, Chrome 115, macOS Ventura, Desktop atau Dev v2.0.0, Safari, iOS 16, iPhone 14 Pro

5. Reproducible

Always | Intermittent | Once | Cannot reproduce

Seberapa sering *bug* ini muncul? * **Always:** *Bug* selalu terjadi setiap kali Anda mengikuti langkah-langkah reproduksi. Ini adalah skenario terbaik bagi *developer*. * **Intermittent:** *Bug* terjadi kadang-kadang, tidak setiap saat, meskipun langkah-langkahnya sama. Ini lebih sulit untuk di-debug, jadi berikan detail sebanyak mungkin. * **Once:** *Bug* hanya terjadi sekali dan Anda tidak dapat mereproduksinya lagi. Laporkan ini dengan hati-hati dan berikan konteks penuh. * **Cannot reproduce:** Anda mencoba mereproduksi *bug* yang dilaporkan oleh orang lain atau yang Anda lihat sekilas, tetapi tidak berhasil. Ini penting untuk dicatat agar tidak membuang waktu.

6. Linked Story

<Notion task ID>

Setiap *test case* harus dapat dilacak kembali ke *user story* yang diujinya. Demikian pula, setiap *bug* idealnya harus dapat dilacak kembali ke *user story* yang gagal dipenuhi *acceptance criteria*-nya. Ini menunjukkan bahwa *bug* tersebut adalah penyimpangan dari perilaku yang diharapkan berdasarkan spesifikasi.

7. Steps to Reproduce (Langkah-langkah Reproduksi)

Ini adalah bagian paling vital dari laporan *bug*. Jelaskan langkah-langkah secara berurutan, jelas, dan spesifik mungkin agar siapa pun dapat mengikuti dan melihat *bug* yang sama. * Gunakan daftar bernomor. * Setiap langkah harus *actionable* (misalnya: "Klik tombol X", "Masukkan teks Y", "Navigasi ke halaman Z"). * Sertakan data *test* yang digunakan jika relevan (misalnya: "Login dengan user: thomas@example.com, password: thomas123").

Contoh Langkah-langkah Reproduksi yang Baik: 1. Buka aplikasi di browser Chrome (v115). 2. Navigasi ke halaman Login (<https://app.thomas.com/login>). 3. Masukkan email:

invalid@example.com. 4. Masukkan password: password123. 5. Klik tombol "Masuk".

8. Expected (Yang Diharapkan)

Jelaskan apa yang seharusnya terjadi jika aplikasi berfungsi dengan benar, sesuai dengan *acceptance criteria user story*.

Contoh: * *Expected*: Sistem menampilkan pesan error "Email atau password salah." di bawah kolom email. * *Expected*: Halaman *search result* menampilkan produk yang harganya antara Rp 50.000 - Rp 100.000, diurutkan dari harga terendah.

9. Actual (Yang Terjadi)

Jelaskan apa yang sebenarnya terjadi, yang merupakan penyimpangan dari perilaku yang diharapkan. Ini adalah deskripsi *bug* itu sendiri.

Contoh: * *Actual*: Sistem menampilkan pesan error "Terjadi kesalahan. Silakan coba lagi." di bagian atas halaman, dan tidak ada pesan spesifik di bawah kolom email. * *Actual*: Halaman *search result* menampilkan produk dengan harga di atas Rp 100.000, dan urutan harga tidak konsisten.

10. Evidence (Bukti)

Sertakan bukti visual atau log teknis yang mendukung laporan *bug* Anda. Ini sangat membantu *developer* untuk memahami konteks dan mempercepat proses *debugging*. * **Screenshot**: Untuk *bug* visual (UI/UX). * **Video**: Sangat berguna untuk *bug* yang melibatkan interaksi kompleks, animasi, atau yang *intermittent*. * **Console log**: Pesan error atau peringatan dari *browser console* atau log aplikasi. * **Network log**: Permintaan dan respons API yang gagal atau tidak sesuai (dari tab Network di *developer tools*).

Tips: Selalu simpan bukti di lokasi yang mudah diakses dan dibagikan, seperti Google Drive, dan pastikan tautannya dapat diakses oleh tim.

11. Impact (Dampak)

Bagian ini membantu tim memahami urgensi dan prioritas *bug* dari perspektif bisnis atau pengguna. * **Users affected**: Siapa yang terpengaruh (semua pengguna, kohort spesifik, *edge case*). * **Workaround**: Jika ada cara bagi pengguna untuk menghindari *bug* tersebut (misalnya, "pengguna bisa menggunakan fitur X sebagai gantinya"). * **Frequency**: Seberapa sering *bug* ini

terjadi pada pengguna (misalnya, "terjadi pada 1 dari 100 sesi pengguna").

12. Notes / Hypothesis (Catatan / Hipotesis)

Ruang untuk menambahkan konteks tambahan, pengamatan, atau dugaan awal Anda tentang penyebab *bug*. Ini bisa menjadi petunjuk berharga bagi *developer*.

Contoh Laporan Bug Lengkap:

```
# [Login] Pengguna tidak bisa masuk dengan kredensial yang benar

**Severity**: P1
**Reporter**: qa-thomas
**Environment**: Staging v1.2.3, Chrome 115.0.5790.170, macOS Ventura 13.5, Desktop
**Reproducible**: Always
**Linked story**: TSK-1234 (Sebagai pengguna, saya bisa login ke akun saya)

## Steps to Reproduce
1. Buka browser Chrome (v115).
2. Navigasi ke halaman Login (`https://staging.thomas.com/login`).
3. Masukkan email: `user@example.com` (kredensial yang valid).
4. Masukkan password: `password123` (kredensial yang valid).
5. Klik tombol "Masuk".

## Expected
Pengguna berhasil masuk ke aplikasi dan diarahkan ke halaman Dashboard.

## Actual
Setelah mengklik tombol "Masuk", halaman tidak merespons. Tidak ada pesan error yang muncul di UI. Di console browser, terlihat error 500 dari endpoint `/api/auth/login`.

## Evidence
- Screenshot: [https://drive.google.com/link-ke-screenshot-error.png](https://drive.google.com/link-ke-screenshot-error.png)
- Video: [https://drive.google.com/link-ke-video-repro.mp4](https://drive.google.com/link-ke-video-repro.mp4)
- Console log:
  ```
 POST https://staging.thomas.com/api/auth/login 500 (Internal Server Error)
 Error: Request failed with status code 500
 at createError (createError.js:16:1)
 at settle (settle.js:17:1)
 at XMLHttpRequest.onloadend (xhr.js:66:1)
  ```
```

- Network log: [https://drive.google.com/link-ke-network-log.har](https://drive.google.com/link-ke-net

Impact

- Users affected: Semua pengguna yang mencoba login di lingkungan Staging.
- Workaround: Tidak ada. Pengguna tidak bisa mengakses aplikasi.
- Frequency: Terjadi pada setiap percobaan login dengan kredensial valid.

Notes / hypothesis

Diduga ada masalah di sisi backend saat memproses permintaan login atau koneksi ke database autentikasi

Dengan laporan seperti ini, *developer* dapat dengan cepat memahami masalah, mereproduksinya, dan mulai mencari solusinya tanpa banyak pertanyaan.

Rubrik Severity: Mengukur Dampak Bug dan Menentukan Prioritas

Menentukan *severity* (tingkat keparahan) *bug* adalah salah satu tugas terpenting seorang QA. Ini bukan hanya tentang seberapa "rusak" sesuatu, tetapi lebih kepada seberapa besar *dampak* kerusakan tersebut terhadap pengguna, bisnis, dan keseluruhan sistem. Thomas menggunakan rubrik *severity* P0 hingga P3 untuk mengklasifikasikan *bug*:

Level	Kriteria	Contoh Nyata	Alur Eskalasi Slack
P0	Kritis / Blocker: Produksi <i>down</i> (tidak bisa diakses), kehilangan data, pelanggaran keamanan, memblokir semua pengguna dari penggunaan fitur inti atau keseluruhan aplikasi.	Login tidak berfungsi sama sekali, pembayaran gagal total, data pengguna terhapus, aplikasi <i>crash</i> terus-menerus.	#incidents + DM Tech Lead

P1	Tinggi: Fitur inti rusak untuk banyak pengguna, tidak ada <i>workaround</i> yang mudah. Meskipun aplikasi bisa diakses, fungsionalitas utama tidak berjalan.	Fitur pencarian mengembalikan error 500, proses pendaftaran pengguna <i>intermittent</i> (kadang berhasil, kadang tidak), fitur <i>upload</i> gambar rusak.	#incidents + DM Tech Lead
P2	Sedang: Fitur terganggu tetapi ada <i>workaround</i> atau hanya mempengaruhi sebagian kecil fungsionalitas/pengguna. Pengguna masih bisa menyelesaikan tugasnya meskipun dengan sedikit kesulitan.	Urutan <i>sort</i> hasil pencarian salah, <i>typo</i> besar di UI yang penting, notifikasi tidak muncul tepat waktu, <i>bug</i> di fitur non-inti.	#qa
P3	Rendah / Kosmetik: Masalah visual minor, ketidaknyamanan kecil, <i>bug</i> di <i>edge case</i> yang jarang terjadi. Tidak menghalangi fungsionalitas inti.	Piksel tidak sejajar, warna teks sedikit berbeda, <i>placeholder</i> yang tidak tepat, <i>bug</i> di <i>tooltip</i> yang jarang diakses.	Notion saja

Penting: * **P0 dan P1** biasanya memerlukan perhatian segera karena berdampak langsung pada pengalaman pengguna dan potensi kerugian bisnis. * **P2** masih penting untuk diperbaiki, tetapi tidak mendesak seperti P0/P1. * **P3** seringkali dapat ditempatkan di *backlog* dan diperbaiki di siklus pengembangan berikutnya atau saat ada waktu luang.

Pemahaman yang jelas tentang *severity* ini akan membantu Anda membuat keputusan yang tepat tentang seberapa cepat dan seberapa luas Anda perlu mengeskalisasi *bug* yang ditemukan.

Alur Eskalasi Bug: Dari Penemuan hingga Perbaikan

Menemukan *bug* dan menuliskannya dalam *ticket* hanyalah langkah pertama. Langkah berikutnya adalah memastikan *bug* tersebut sampai ke tangan yang tepat dengan urgensi yang sesuai. Di Thomas, kami memiliki alur eskalasi yang jelas berdasarkan tingkat *severity bug*:

1. **Semua Bug:** Setiap *bug* yang dilaporkan, tidak peduli *severity*-nya, akan dibuatkan *ticket* di Notion (database *bug* kami).
2. **Visibilitas Terpusat:** Selain di Notion, semua *bug* juga akan diposting ke *channel* Slack `#qa-reports` untuk visibilitas terpusat bagi seluruh tim. Ini memastikan semua *stakeholder* dapat melihat *bug* yang ditemukan.

Berikut adalah alur eskalasi spesifik berdasarkan *severity*:

P0 / P1 (Kritis & Tinggi):

- Buat *ticket bug* di Notion.
- Segera posting ke *channel* Slack `#incidents`. *Channel* ini khusus untuk masalah-masalah mendesak yang membutuhkan perhatian tim secepatnya.
- DM (Direct Message) Tech Lead tim terkait. Ini memastikan pemimpin teknis segera menyadari masalah dan dapat mengalokasikan sumber daya untuk perbaikan.

P2 (Sedang):

- Buat *ticket bug* di Notion.
- Posting ke *channel* Slack `#qa`. Ini adalah *channel* umum untuk diskusi QA dan laporan *bug* yang tidak terlalu mendesak.

P3 (Rendah / Kosmetik):

- Cukup buat *ticket bug* di Notion saja.
- *Bug* ini akan muncul dalam tinjauan *backlog* berikutnya dan akan dipertimbangkan untuk perbaikan di masa mendatang. Tidak perlu notifikasi Slack langsung.

Mengapa Alur Eskalasi Berbeda? Tujuan dari alur eskalasi yang berbeda ini adalah untuk: *

Meningkatkan Efisiensi: Hanya masalah yang paling mendesak yang mengganggu *workflow* tim dengan notifikasi langsung. * **Memastikan Prioritas:** *Bug* P0/P1 mendapatkan perhatian instan karena dampaknya yang besar. * **Mengurangi Kebisingan:** Tim *developer* tidak terbebani oleh notifikasi untuk *bug* minor, memungkinkan mereka fokus pada pekerjaan yang lebih penting.

Otomatisasi Routing dengan `qa_report_bug.py`

Untuk menyederhanakan proses pelaporan dan memastikan alur eskalasi diikuti dengan benar, Thomas menggunakan *script* otomatis `qa_report_bug.py`. *Script* ini terintegrasi dengan Notion dan Slack, sehingga Anda tidak perlu melakukan langkah-langkah manual untuk setiap *bug*.

Bagaimana Cara Kerjanya (Konseptual):

Ketika Anda menemukan *bug* dan ingin melaporkannya, alih-alih membuka Notion dan Slack secara manual, Anda akan menjalankan *script* ini dan memberikan informasi *bug* yang diperlukan.

```
# Contoh penggunaan script (konseptual)
python scripts/qa_report_bug.py \
  --title "[Login] Pengguna tidak bisa masuk" \
  --severity P1 \
  --environment "Staging v1.2.3, Chrome, macOS" \
  --reproducible Always \
  --steps "1. Buka login; 2. Masukkan kredensial; 3. Klik masuk" \
  --expected "Masuk dashboard" \
  --actual "Halaman tidak merespons, error 500" \
  --evidence "link-ke-bukti.png" \
  --linked_story TSK-1234
```

Script ini kemudian akan: 1. Membuat *ticket bug* baru di database Notion dengan semua detail yang Anda berikan. 2. Berdasarkan *severity* yang Anda tentukan, *script* akan secara otomatis: * Jika P0/P1: Memposting notifikasi ke `#incidents` Slack dan mengirim DM ke Tech Lead. * Jika P2: Memposting notifikasi ke `#qa` Slack. * Jika P3: Tidak melakukan notifikasi Slack tambahan (hanya Notion). 3. Memposting ringkasan *bug* ke *channel* Slack `#qa-reports` untuk visibilitas

umum.

Dengan otomatisasi ini, proses pelaporan menjadi lebih cepat, konsisten, dan meminimalkan kesalahan manusia dalam mengikuti alur eskalasi. Ini adalah contoh bagaimana seorang QA Engineer modern memanfaatkan *tool* untuk meningkatkan efisiensi.

Kapan TIDAK Perlu Melaporkan Bug?

Tidak semua masalah atau pengamatan yang Anda temukan harus dilaporkan sebagai *bug*. Terkadang, apa yang terlihat seperti *bug* sebenarnya adalah sesuatu yang lain. Melaporkan hal-hal yang bukan *bug* hanya akan menambah *noise* di *backlog* dan membuang waktu *developer*. Berikut adalah beberapa skenario kapan Anda sebaiknya tidak membuat *ticket bug*:

Itu adalah Permintaan Fitur Baru (Feature Request)

- **Skenario:** Anda melihat bahwa aplikasi tidak memiliki fitur "ekspor data ke Excel" dan Anda berpikir itu akan sangat berguna.
- **Mengapa Bukan Bug:** Aplikasi tidak memiliki fitur tersebut bukan berarti ada yang rusak. Itu adalah fitur yang belum ada.
- **Tindakan:** Komunikasikan ide ini kepada Product Manager (PM). PM akan mengevaluasinya sebagai potensi fitur baru untuk dipertimbangkan dalam *roadmap* produk. Jangan membuat *bug ticket*.

Kesalahpahaman Perilaku yang Diharapkan (Misunderstanding of Expected Behavior)

- **Skenario:** Anda berpikir bahwa mengklik tombol "Simpan" harusnya mengarahkan Anda ke halaman lain, tetapi ternyata hanya menyimpan data di halaman yang sama. Setelah bertanya ke PM atau *developer*, ternyata memang begitulah desainnya.
- **Mengapa Bukan Bug:** Perilaku yang Anda harapkan berbeda dengan desain yang sebenarnya. Tidak ada yang rusak, hanya ada perbedaan pemahaman.
- **Tindakan:** Klarifikasi dan selesaikan kesalahpahaman ini di Slack atau dalam rapat tim. Update pemahaman Anda tentang *acceptance criteria*. Jika perlu, update *test case* Anda.

Tidak Dapat Direproduksi Setelah 2 Kali Percobaan (Can't Reproduce 2x)

- **Skenario:** Anda melihat sekilas sebuah anomali (misalnya, teks berkedip sebentar atau halaman *loading* sangat lama), tetapi setelah mencoba mereproduksinya dua kali dengan

langkah yang sama, *bug* tersebut tidak muncul lagi.

- **Mengapa Bukan Bug (untuk dilaporkan sebagai P0/P1):** *Bug* yang tidak dapat direproduksi sangat sulit, bahkan mustahil, untuk di-debug. Melaporkannya sebagai *bug* prioritas tinggi akan membuang waktu *developer*.
- **Tindakan:** Jika Anda yakin ada sesuatu yang terjadi tetapi tidak dapat mereproduksinya, Anda bisa membuat *ticket bug* dengan `Reproducible = Cannot reproduce` atau `Once`. Namun, pastikan untuk memberi catatan di judul atau deskripsi bahwa ini mungkin *noise* atau *edge case* yang sangat langka. Jangan mengeskalasikannya sebagai *bug* P0/P1 yang mendesak.

Dengan memahami kapan tidak perlu melaporkan *bug*, Anda membantu menjaga *backlog* tetap bersih dan relevan, sehingga tim dapat fokus pada masalah nyata yang membutuhkan perhatian.

Anti-Pattern: Batch Filing di Akhir Siklus vs. File as You Find

Salah satu kesalahan umum yang sering dilakukan oleh QA yang kurang berpengalaman adalah menunda pelaporan *bug*. Mereka mungkin menunggu hingga akhir siklus *testing* atau hingga mereka menemukan "cukup banyak" *bug* sebelum mulai membuat *ticket*. Ini dikenal sebagai **Batch Filing di Akhir Siklus** dan merupakan *anti-pattern* yang merugikan.

Anti-Pattern: Batch Filing di Akhir Siklus

- **Apa itu:** Menemukan *bug* sepanjang hari/minggu, mencatatnya secara pribadi (misalnya di notepad), dan baru membuat *ticket bug* untuk semuanya sekaligus di akhir hari atau akhir siklus.

Dampak Negatif:

- **Feedback Loop Lambat:** *Developer* tidak mendapatkan *feedback* segera. *Bug* yang ditemukan pagi hari baru diketahui *developer* sore hari atau keesokan harinya, menunda perbaikan.
- **Context Switching Tinggi:** *Developer* mungkin sudah beralih ke tugas lain. Ketika mereka mendapatkan laporan *bug* dari fitur yang mereka kerjakan kemarin, mereka harus mengalihkan konteks kembali ke pekerjaan sebelumnya.

- **Informasi Hilang:** Semakin lama Anda menunda pelaporan, semakin besar kemungkinan Anda melupakan detail penting, konteks, atau langkah-langkah reproduksi yang spesifik.
- **Crunch di Akhir:** Anda akan terburu-buru membuat banyak *ticket bug* di akhir, meningkatkan risiko kesalahan ketik, detail yang kurang, atau *bug* yang tidak lengkap.
- **Kelelahan Developer:** Menerima "banjir" *bug* sekaligus bisa membuat *developer* kewalahan dan frustrasi.

Best Practice: File as You Find

- **Apa itu:** Segera setelah Anda menemukan *bug* yang dapat direproduksi, buat *ticket bug* sesegera mungkin.

Manfaat:

- **Feedback Loop Cepat:** *Developer* mendapatkan notifikasi *bug* hampir secara *real-time*. Mereka dapat memperbaiki *bug* saat konteks fitur masih segar di pikiran mereka.
- **Efisiensi Developer:** Mengurangi *context switching*. *Developer* bisa memperbaiki *bug* kecil dengan cepat sebelum beralih ke tugas berikutnya.
- **Akurasi Tinggi:** Anda melaporkan *bug* saat detail dan konteksnya masih sangat jelas di kepala Anda, memastikan laporan yang lengkap dan akurat.
- **Distribusi Beban Merata:** *Bug* masuk secara bertahap, bukan membanjiri tim sekaligus.
- **Mencegah Regresi:** Perbaikan *bug* lebih awal dapat mencegah *bug* tersebut menyebabkan masalah lebih lanjut di bagian lain aplikasi.

Sebagai QA Thomas, Anda harus menganut prinsip "**File as You Find**". Gunakan *template* standar dan otomatisasi `qa_report_bug.py` untuk membuat proses ini secepat dan semudah mungkin. Ingat, tujuan Anda adalah menjadi *quality gate* yang efisien, bukan *bottleneck*.

Rangkuman

Di bab ini, Anda telah mempelajari seluk-beluk pelaporan *bug* yang efektif, sebuah keterampilan fundamental bagi setiap QA Engineer. Kita telah membahas bahwa laporan *bug* yang baik adalah alat komunikasi yang jelas, ringkas, dan lengkap, yang memandu *developer* menuju perbaikan.

Anda kini memahami setiap komponen dari *template* laporan *bug* standar Thomas, mulai dari judul yang deskriptif, detail *environment*, langkah-langkah reproduksi yang presisi, hingga pentingnya bukti visual dan log teknis. Kita juga telah mendalami rubrik *severity* P0-P3, yang membantu Anda menilai dampak *bug* dan menentukan urgensinya.

Selain itu, Anda telah melihat bagaimana alur eskalasi Thomas bekerja, memastikan *bug* mendapatkan perhatian yang sesuai berdasarkan *severity*-nya, dan bagaimana *script* otomatis `qa_report_bug.py` menyederhanakan proses ini. Terakhir, kita membahas kapan tidak perlu melaporkan *bug* (misalnya, untuk *feature request* atau kesalahpahaman) dan mengapa praktik "File as You Find" jauh lebih unggul daripada "Batch Filing" dalam menjaga efisiensi tim.

Dengan menguasai seni pelaporan *bug*, Anda tidak hanya menemukan masalah, tetapi juga menjadi fasilitator solusi yang efektif, memastikan *software* yang dirilis memenuhi standar kualitas tertinggi.

Checklist Actionable

Berikut adalah daftar tindakan yang bisa Anda terapkan segera setelah membaca bab ini:

- **Pahami Template Bug:** Hafalkan atau simpan *template* laporan *bug* standar. Pastikan Anda tahu setiap bagiannya dan mengapa itu penting.
- **Latih Penulisan Judul:** Latih diri Anda untuk menulis judul *bug* yang ringkas, informatif, dan spesifik ([Area] Ringkasan Masalah).
- **Detailkan Langkah Reproduksi:** Saat menemukan *bug*, tulis langkah-langkah reproduksi secara berurutan, bernomor, dan se-eksplisit mungkin.
- **Sertakan Bukti Kuat:** Selalu sertakan *screenshot*, video, atau log (console/network) untuk mendukung laporan *bug* Anda, terutama untuk *bug* visual atau *intermittent*.
- **Tentukan Severity dengan Tepat:** Gunakan rubrik P0-P3 secara konsisten. Pertimbangkan dampak *bug* terhadap pengguna dan bisnis, bukan hanya kesulitan teknisnya.
- **Ikuti Alur Eskalasi:** Pastikan Anda mengikuti alur eskalasi Slack yang benar berdasarkan *severity bug* (P0/P1 ke `#incidents` + DM Tech Lead, P2 ke `#qa`, P3 hanya Notion).

- **Manfaatkan Otomatisasi:** Jika tersedia, gunakan *script* `qa_report_bug.py` untuk mengotomatisasi proses pelaporan dan eskalasi.
 - **Terapkan "File as You Find":** Laporkan *bug* segera setelah Anda menemukannya dan berhasil mereproduksinya. Hindari menumpuk *bug* untuk dilaporkan nanti.
 - **Hindari "Noise":** Sebelum melaporkan *bug*, pertimbangkan apakah itu benar-benar *bug*, *feature request*, atau kesalahpahaman. Jika tidak dapat direproduksi, tandai dengan jelas.
 - **Verifikasi Perbaikan:** Setelah *bug* diperbaiki, pastikan Anda memverifikasi perbaikan tersebut di *build* yang baru. Ini akan dibahas lebih lanjut di bab *Test Execution*.
-

Chapter 05: Reporting & Documentation — UAT, Release Notes, Knowledge Base

Selamat datang kembali dalam perjalanan Anda sebagai Quality Assurance Engineer! Jika di bab-bab sebelumnya kita sudah membahas bagaimana cara menganalisis *requirement*, merancang strategi pengujian, hingga melaksanakan eksekusi pengujian, kini saatnya kita berbicara tentang bagaimana semua kerja keras itu dikomunikasikan.

Sebagai seorang QA Engineer, tugas Anda tidak berhenti pada menemukan *bug* atau memastikan fitur bekerja. Bagian krusial lainnya adalah mendokumentasikan dan melaporkan temuan serta status kualitas produk kepada berbagai pihak yang berkepentingan. Bayangkan Anda telah menemukan *bug* kritis atau memastikan sebuah fitur siap dirilis, tetapi informasi ini tidak sampai ke telinga yang tepat. Tentu saja, itu akan menjadi sia-sia, bukan?

Bab ini akan memandu Anda melalui empat jenis dokumen utama yang akan Anda hasilkan sebagai QA Engineer, serta bagaimana cara mendistribusikannya secara efektif. Kita akan membahas Laporan UAT (User Acceptance Testing) yang krusial untuk persetujuan *stakeholder*, Catatan Rilis (Release Notes) yang berhadapan langsung dengan pengguna, Dokumentasi Pengujian sebagai basis pengetahuan jangka panjang, dan Laporan Eksekusi Pengujian yang

menjadi catatan harian Anda. Mari kita mulai!

Mengapa Dokumentasi Penting dalam Peran QA?

Sebelum kita menyelami jenis-jenis dokumen, mari kita pahami mengapa aspek pelaporan dan dokumentasi ini sangat vital:

1. **Transparansi dan Akuntabilitas:** Dokumentasi memberikan gambaran jelas tentang apa yang telah diuji, bagaimana hasilnya, dan apa saja risiko yang mungkin ada. Ini membangun kepercayaan dan akuntabilitas.
2. **Pengambilan Keputusan:** Laporan yang baik membantu *stakeholder* (Product Manager, Tech Lead, bahkan jajaran direksi) membuat keputusan yang tepat, misalnya apakah sebuah fitur sudah siap dirilis, atau apakah ada *bug* yang harus segera diperbaiki.
3. **Basis Pengetahuan Jangka Panjang:** Dokumentasi pengujian berfungsi sebagai *knowledge base* yang dapat dirujuk di masa depan. Ini sangat berharga untuk *onboarding* anggota tim baru, *regresi*, atau untuk memahami mengapa sebuah keputusan pengujian diambil di masa lalu.
4. **Komunikasi Efektif:** Tidak semua orang dalam tim atau perusahaan memiliki pemahaman teknis yang sama. Dokumentasi yang terstruktur membantu Anda mengkomunikasikan informasi kompleks dalam format yang mudah dicerna oleh audiens yang berbeda.
5. **Audit dan Kepatuhan:** Dalam beberapa industri, dokumentasi pengujian adalah persyaratan wajib untuk tujuan audit dan kepatuhan regulasi.

Singkatnya, dokumentasi adalah jembatan antara aktivitas pengujian Anda dengan pengambilan keputusan bisnis. Ini adalah cara Anda memastikan bahwa kualitas tidak hanya *tercapai*, tetapi juga *terkomunikasikan* dengan baik.

1. Laporan UAT (User Acceptance Testing)

Laporan UAT adalah salah satu dokumen terpenting yang akan Anda hasilkan, terutama setelah fase UAT selesai. Tujuan utamanya adalah untuk memberikan gambaran komprehensif kepada *stakeholder* tentang status fitur atau produk dari sudut pandang pengguna akhir, dan yang terpenting, mendapatkan persetujuan resmi (sign-off) untuk rilis.

Siapa yang membaca ini? Laporan UAT ditujukan untuk audiens tingkat tinggi: Leadership, Product Manager, Tech Lead, dan *stakeholder* bisnis lainnya yang memiliki kepentingan dalam produk.

Struktur Laporan UAT

Laporan UAT harus jelas, ringkas, dan fokus pada informasi yang relevan untuk pengambilan keputusan. Berikut adalah struktur yang disarankan, berdasarkan template yang sering digunakan di "Journey QA Thomas":

```
# UAT Report – [Nama Fitur/Proyek] – [Tanggal]
```

```
**Doc type**: UAT Report
```

```
**Feature/Release**: [Nama Fitur atau Versi Rilis]
```

```
**Author**: qa-thomas
```

```
**Status**: Draft | Final
```

```
**Last updated**: YYYY-MM-DD
```

```
**Related**:
```

```
- PRD: [Link ke Product Requirement Document]
```

```
- Test Strategy: [Link ke Dokumen Strategi Pengujian]
```

```
- Linked Epic: [Link ke Epic di Jira/Notion]
```

```
---
```

```
## Executive Summary
```

```
<1 paragraf. Apakah kita mencapai target kualitas? Temuan penting? Rekomendasi?>
```

```
## Scope Tested
```

```
- Stories: <Daftar User Story yang diuji dengan ID/link>
```

```
- Out of scope: <Daftar hal yang tidak termasuk dalam pengujian>
```

```
## Pass/Fail by User Story
```

```
| Story ID | Story | Status | Notes |
```

```
|---|---|---|---|
```

```
| S-01 | Pengguna dapat mendaftar akun baru | ■ Pass | |
```

```
| S-02 | Pengguna dapat masuk dengan email/password | ■■ Pass with notes | Ada sedikit inkonsistensi U
```

```
| S-03 | Pengguna dapat mengubah profil | ■ Fail | Fitur upload foto profil gagal saat ukuran file > 2
```

```
## Bug Summary
```

```
| Severity | Count | Status | Tickets |
```

```
|---|---|---|---|
```

```
| P0 | 0 | – | |
```

```
| P1 | 2 | 1 fixed, 1 open | [BUG-005], [BUG-007] |
```

```
| P2 | 5 | 3 fixed, 2 deferred | [BUG-001], [BUG-002], [BUG-003], [BUG-004], [BUG-006] |
```

```
| P3 | 8 | logged for backlog | [BUG-008] - [BUG-015] |
```

UAT Participants

Name	Role	Sessions	Findings filed
Thomas	Product Manager	3	2 bugs, 1 improvement
Sarah	Business Analyst	2	1 bug
Budi	Marketing Lead	1	0

Performance Observations

- Response time: <rata-rata 300ms vs target 500ms>
- Error rate: <0.1% vs target 0.5%>
- Load test: <Tidak dilakukan untuk rilis ini>

Accessibility Observations

- Screen reader: <Pass, semua elemen interaktif dapat diakses>
- Keyboard navigation: <Pass, semua fungsi dapat diakses via keyboard>
- Color contrast: <Ada beberapa isu pada elemen secondary, tidak blocking>

Risks for Release

1. ****Bug P1 yang masih terbuka:**** [BUG-007] (Error saat mengubah email) - ****Mitigasi:**** Akan dirilis h
2. ****Inkonsistensi UI kecil:**** Tidak blocking, akan diperbaiki di sprint berikutnya.

Recommendation

- [] Ready for release
- [x] Conditionally ready (membutuhkan perbaikan [BUG-007] sebagai *hotfix*)
- [] NOT ready (reason: ...)

Sign-offs

Role	Name	Signed	Date
QA	qa-thomas	■	2023-10-26
PM	pm-thomas	■	
Tech Lead	tech-lead-thomas	■	
Business stakeholder	...	■	

Penjelasan Bagian-bagian Penting:

- **Executive Summary:** Ini adalah bagian terpenting bagi *stakeholder* tingkat tinggi. Rangkum temuan kunci dan rekomendasi dalam satu paragraf yang mudah dicerna.
- **Scope Tested:** Jelaskan dengan jelas apa saja yang diuji dan apa yang sengaja tidak diuji. Ini menghindari kesalahpahaman.
- **Pass/Fail by User Story:** Ini memberikan detail granular tentang status setiap *user story* yang diuji selama UAT. Gunakan status visual (■, ■■■, ■) untuk kejelasan.

- **Bug Summary:** Ringkasan *bug* berdasarkan tingkat keparahan (P0-P3, seperti yang dibahas di Bab 04). Ini menunjukkan gambaran risiko yang ada.

- **UAT Participants:** Mencatat siapa saja yang terlibat dalam UAT, berapa sesi yang mereka ikuti, dan berapa banyak temuan yang mereka laporkan. Ini menunjukkan partisipasi dan validasi.

- **Performance & Accessibility Observations:** Meskipun bukan fokus utama UAT, observasi ini penting untuk memberikan gambaran kualitas yang lebih holistik.

- **Risks for Release:** Identifikasi risiko yang masih ada dan bagaimana tim berencana untuk mengatasinya. Ini menunjukkan proaktivitas Anda.

- **Recommendation:** Rekomendasi Anda sebagai QA Engineer: apakah produk siap dirilis, siap dengan syarat, atau belum siap sama sekali. Ini adalah hasil dari penilaian Anda.

- **Sign-offs:** Bagian ini **sangat krusial**. Ini adalah tempat *stakeholder* secara resmi menyetujui atau menolak rilis berdasarkan laporan Anda. Tanda tangan (atau centang digital) dari PM, Tech Lead, dan *stakeholder* bisnis lainnya adalah bentuk komitmen. Tanpa ini, rilis bisa berisiko karena tidak ada persetujuan yang jelas.

Penyimpanan dan Distribusi: Di "Journey QA Thomas", laporan ini disimpan sebagai tugas di Notion dan diekspor ke PDF untuk *stakeholder* yang mungkin lebih memilih lampiran email. Pastikan format PDF tetap rapi dan mudah dibaca.

2. Catatan Rilis (Release Notes)

Berbeda dengan Laporan UAT yang ditujukan untuk internal dan *stakeholder*, Catatan Rilis (Release Notes) adalah dokumen yang berhadapan langsung dengan **pengguna akhir**. Tujuannya adalah untuk menginformasikan kepada pengguna tentang perubahan, fitur baru, perbaikan, atau isu yang diketahui dalam versi produk yang baru dirilis.

Siapa yang membaca ini? Pengguna akhir produk, tim Marketing (untuk kampanye), dan tim Support (untuk membantu pengguna).

Struktur Catatan Rilis

Catatan rilis harus ditulis dengan bahasa yang mudah dipahami oleh non-teknisi, fokus pada manfaat bagi pengguna, dan menghindari jargon teknis.

Release v1.2.0 – 2023-10-26

****Doc type**:** Release Notes

****Feature/Release**:** v1.2.0 - Peningkatan Fitur Profil

****Author**:** qa-thomas

****Status**:** Final

****Last updated**:** 2023-10-26

****Related**:**

- Linked Epic: [Link ke Epic "Peningkatan Profil Pengguna"]

Halo Pengguna Atmoscheck!

Kami sangat senang untuk mengumumkan pembaruan terbaru kami, v1.2.0, yang membawa peningkatan signifikan.

■ Baru

- ****Upload Foto Profil yang Lebih Mudah:**** Kini Anda bisa mengunggah foto profil langsung dari perangkat.
- ****Pengaturan Notifikasi yang Lebih Detail:**** Kontrol penuh ada di tangan Anda! Sesuaikan jenis notifikasi.

■ Diperbaiki

- Memperbaiki masalah di mana beberapa pengguna tidak dapat menyimpan perubahan pada nama pengguna mereka.
- Mengatasi **bug** yang menyebabkan aplikasi **crash** saat mencoba melihat riwayat data dari tanggal tertentu.

■ Ditingkatkan

- Peningkatan kinerja pada halaman **dashboard**, membuat pemuatan data cuaca Anda lebih cepat dan responsif.
- Tampilan antarmuka pengguna (UI) yang lebih bersih dan modern pada bagian pengaturan, memberikan pengalaman yang lebih baik.

■■ Isu yang Diketahui

- Saat ini, kami menemukan isu kecil di mana pratinjau foto profil mungkin terlihat sedikit terdistorsi.

Terima kasih atas dukungan Anda!

Tim Atmoscheck

Penjelasan Bagian-bagian Penting:

- **Judul dan Versi:** Jelas menunjukkan versi rilis dan tanggalnya.
- **Pengantar:** Sapa pengguna dan berikan sedikit konteks tentang pembaruan ini.
- **■ New (Baru):** Daftar fitur baru. Fokus pada manfaat yang akan didapatkan pengguna, bukan detail teknis implementasinya.
- **■ Fixed (Diperbaiki):** Daftar *bug* yang telah diperbaiki. Lagi-lagi, jelaskan masalah dari sudut pandang pengguna, bukan ID *bug* atau *stack trace*.
- **■ Improved (Ditingkatkan):** Peningkatan pada kinerja, UX, atau fitur yang sudah ada.

- ■■ **Known Issues (Isu yang Diketahui):** Jika ada *bug* non-kritis yang disadari tetapi belum bisa diperbaiki di rilis ini, jujur sampaikan di sini. Berikan *workaround* jika ada. Ini membangun kepercayaan dan mengurangi tiket *support*.

Penyimpanan dan Distribusi: Catatan rilis disimpan di Notion (dengan label `release-notes`), di folder Drive (agar tim Marketing/Support dapat mengambilnya dengan mudah), dan diringkas kemudian diposting di saluran Slack `#announcements`. Marketing juga mungkin akan menggunakannya untuk *email blast* kepada pengguna.

3. Dokumentasi Pengujian Jangka Panjang (Test Documentation)

Selain laporan spesifik per rilis, ada kebutuhan untuk menjaga basis pengetahuan jangka panjang mengenai pengujian. Ini adalah *knowledge base* yang terus berkembang dan menjadi referensi utama bagi tim QA (dan kadang-kadang tim lain) untuk memahami *setup*, data, dan proses pengujian secara keseluruhan.

Siapa yang membaca ini? Tim QA (utama), Developer (untuk memahami lingkungan pengujian atau *bug patterns*), dan tim *onboarding* (untuk anggota baru).

Kategori Utama dalam Test Documentation (di Notion "QA Knowledge Base"):

Test Environments (Lingkungan Pengujian):

- **Deskripsi:** Detail tentang berbagai lingkungan pengujian yang tersedia (misalnya, `staging`, `dev`, `QA`, `pre-prod`).

Apa yang disertakan:

- URL untuk setiap lingkungan.
- Kredensial akses (username/password atau cara mendapatkannya).
- Detail konfigurasi unik atau "gotchas" (misalnya, "lingkungan staging tidak memiliki integrasi pembayaran langsung, gunakan mode sandbox").
- Status lingkungan (misalnya, kapan terakhir di-refresh, siapa yang bertanggung jawab).

Contoh: ``` ### Lingkungan Staging

- **URL:** `https://staging.atmoscheck.com`

- **Akses:** Login SSO via Google. Jika tidak bisa akses, hubungi #ops-team.

- **Data:** Refresh otomatis setiap hari Minggu pukul 03:00 WIB dari database produksi yang di-masking. Data pengguna bersifat anonim.

- **Catatan:** Pembayaran real tidak aktif. Gunakan kartu kredit test yang disediakan Stripe. ``

Test Data Inventory (Inventaris Data Pengujian):

- **Deskripsi:** Daftar pengguna, *fixture*, atau data spesifik yang tersedia untuk pengujian.

Apa yang disertakan:

- Jenis data (misalnya, pengguna admin, pengguna premium, pengguna tanpa langganan).
- Kredensial atau cara mengakses data tersebut.
- Status data (misalnya, "pengguna admin_thomas memiliki akses ke semua fitur").
- Cara me-refresh atau membuat data baru.

Contoh: ``` ### Data Pengguna untuk Testing

- **Admin User:** `admin_test@atmoscheck.com` / `password123` (akses penuh)
- **Premium User:** `premium_test@atmoscheck.com` / `password123` (langganan aktif)
- **Free User:** `free_test@atmoscheck.com` / `password123` (langganan dasar)

Pengguna dengan Data Cuaca Spesifik:

- `user_jakarta@atmoscheck.com` (memiliki data historis Jakarta 5 tahun)
- `user_bandung@atmoscheck.com` (memiliki data historis Bandung 3 tahun)

- **Cara Membuat Data Baru:** Gunakan script `generate_test_data.sh` di repo `atmoscheck-backend/scripts`. ``

Automation Infrastructure (Infrastruktur Otomasi):

- **Deskripsi:** Informasi tentang *framework* otomasi, cara menjalankannya, dan di mana menemukan hasilnya.

Apa yang disertakan:

- *Framework* yang digunakan (misalnya, Playwright, Selenium).
- Perintah untuk menjalankan *test suite* (lokal atau CI/CD).
- Lokasi laporan pengujian (misalnya, Allure Reports, dashboard CI).
- Panduan tentang cara menambahkan *test case* baru.

Contoh: ``` ### Panduan Automation Testing (Playwright)

- **Repo:** `git@github.com:atmoscheck/qa-automation.git`
- **Setup Lokal:** `npm install && npx playwright install`
- **Menjalankan Semua Test:** `npm test`
- **Menjalankan Test Spesifik:** `npm test -- --project=chromium --grep "login feature"`
- **Laporan HTML Lokal:** `npx playwright show-report` (setelah `npm test`)
- **Laporan CI/CD:** Terintegrasi dengan Jenkins. Laporan Allure dapat diakses via [Link ke Jenkins Job].
- **Cara Menambahkan Test Baru:** Lihat `docs/HOW_TO_ADD_TESTS.md` di repo. ```

Regression Checklist (Daftar Periksa Regresi):

- **Deskripsi:** Daftar *test case* atau skenario kritis yang **harus selalu diuji** sebelum setiap rilis utama, terlepas dari fitur yang berubah. Ini memastikan fungsi inti tidak rusak.

Apa yang disertakan:

- Daftar skenario kritis (misalnya, login, pendaftaran, melihat *dashboard* utama, melakukan pembayaran).
- Link ke *test case* yang relevan jika ada.
- Prioritas atau frekuensi pengujian.

Contoh: ``` ### Daftar Periksa Regresi Wajib (Setiap Rilis)

Login & Logout:

- [TC-001] Login sukses dengan kredensial valid.
- [TC-002] Logout sukses.
- [TC-003] Login gagal dengan kredensial tidak valid.

Dashboard Utama:

- [TC-005] Data cuaca saat ini ditampilkan dengan benar.
- [TC-006] Navigasi ke halaman lain berfungsi.

Pengaturan Profil:

- [TC-010] Mengubah nama pengguna berhasil.
- [TC-011] Mengubah foto profil berhasil.

Pembayaran (jika ada):

- [TC-007] Proses pembayaran langganan berhasil. ``

Bug Patterns (Pola Bug):

- **Deskripsi:** Dokumentasi tentang jenis *bug* yang sering muncul, penyebab umumnya, dan contoh tiket terkait. Ini membantu tim belajar dari kesalahan masa lalu dan mengidentifikasi *bug* lebih cepat.

Apa yang disertakan:

- Kategori *bug* (misalnya, "Off-by-one errors", "Race conditions", "Cross-browser UI issues").
- Deskripsi singkat tentang pola tersebut.
- Contoh *bug report* (dengan link tiket).
- Saran mitigasi atau cara menguji untuk pola tersebut.

Contoh: `` ### Pola Bug Umum Atmoscheck

Inkonsistensi Data Historis:

- **Deskripsi:** Sering terjadi ketika ada perubahan pada API data cuaca eksternal atau saat sinkronisasi data. Data yang ditampilkan tidak sesuai dengan data asli atau memiliki gap.
- **Contoh Tiket:** [BUG-201: Data historis Jakarta hilang untuk 2022], [BUG-205: Perbedaan suhu historis antara staging dan prod]
- **Tips Uji:** Selalu verifikasi data historis dari sumber yang terpercaya setelah setiap perubahan pada integrasi API data.

UI/UX Browser-Specific:

- **Deskripsi:** Tampilan atau interaksi UI yang berbeda/rusak pada browser tertentu (terutama Safari dan Firefox) dibandingkan Chrome.
- **Contoh Tiket:** [BUG-310: Tombol "Lihat Detail" tidak rata di Safari], [BUG-315: Modal notifikasi terpotong di Firefox]
- **Tips Uji:** Selalu lakukan pengujian visual cepat di setidaknya 3 browser utama. ``

Pemeliharaan: Dokumentasi ini harus menjadi *living document*. Di "Journey QA Thomas", disarankan untuk mengalokasikan "documentation hour" setiap bulan untuk meninjau dan memperbarui *knowledge base* ini. Ini memastikan informasi tetap relevan dan akurat.

4. Laporan Eksekusi Pengujian (Test Execution Report)

Laporan Eksekusi Pengujian adalah dokumen yang dihasilkan setelah setiap siklus pengujian selesai. Ini adalah catatan harian atau mingguan dari aktivitas pengujian Anda. Meskipun sudah dibahas secara detail di Bab 03, penting untuk memahami perannya dalam konteks pelaporan dan dokumentasi secara keseluruhan.

Siapa yang membaca ini? Product Manager dan Tech Lead, untuk mendapatkan *update* harian atau mingguan tentang progres pengujian.

Fungsi Utama:

- **Progress Tracking:** Menunjukkan berapa banyak *test case* yang telah dieksekusi, berapa yang lulus, gagal, atau diblokir.

- **Identifikasi Masalah:** Menyoroti *bug* yang ditemukan selama siklus pengujian tersebut.

- **Input untuk Laporan UAT:** Untuk rilis besar, beberapa laporan eksekusi pengujian dapat diagregasikan menjadi ringkasan pengujian tingkat rilis yang kemudian dimasukkan ke dalam Laporan UAT.

Penyimpanan dan Distribusi: Setiap laporan eksekusi pengujian biasanya diajukan sebagai tugas di Notion dan ringkasannya diposting di saluran Slack [#qa](#) setiap hari atau pada akhir siklus pengujian. Ini memastikan tim inti selalu mendapatkan informasi terbaru.

Konvensi Header Dokumen QA

Untuk menjaga konsistensi dan memudahkan pencarian informasi, semua dokumen yang dihasilkan oleh tim QA Thomas menggunakan konvensi header yang seragam. Ini membantu pembaca segera memahami konteks dokumen.

```
**Doc type**: <UAT Report | Release Notes | Test Doc | Test Execution Report>
**Feature/Release**: <nama fitur atau versi rilis>
**Author**: qa-thomas
**Status**: Draft | Final | Archived
**Last updated**: YYYY-MM-DD
**Related**:
- PRD: <link ke Product Requirement Document>
- Test Strategy: <link ke Dokumen Strategi Pengujian>
- Linked Epic: <link ke Epic di Jira/Notion>
```

Penjelasan Field Header:

- **Doc type:** Mengidentifikasi jenis dokumen (misalnya, "UAT Report", "Release Notes"). Ini membantu dalam kategorisasi dan pencarian.
- **Feature/Release:** Nama fitur atau nomor versi rilis yang menjadi fokus dokumen.
- **Author:** Nama penulis dokumen (misalnya, qa-thomas).
- **Status:** Status terkini dokumen (misalnya, Draft, Final, Archived).
- **Last updated:** Tanggal terakhir dokumen diperbarui.

Related: Bagian ini sangat penting untuk keterlacakan (traceability). Ini menyediakan tautan ke dokumen atau entitas terkait lainnya, seperti:

- **PRD (Product Requirement Document):** Dokumen yang menjelaskan persyaratan produk.
- **Test Strategy:** Dokumen yang menjelaskan strategi pengujian secara keseluruhan.
- **Linked Epic:** Tautan ke *epic* di *project management tool* (misalnya, Notion, Jira) yang terkait dengan fitur atau rilis ini.

Menerapkan konvensi header ini memastikan bahwa setiap dokumen memiliki identitas yang jelas dan mudah dihubungkan dengan konteks proyek yang lebih luas.

Distribusi Dokumen: Siapa Membaca Apa?

Membuat dokumen yang hebat tidak cukup; Anda juga harus memastikan bahwa dokumen tersebut sampai ke audiens yang tepat melalui saluran yang sesuai. Berikut adalah panduan distribusi dokumen di "Journey QA Thomas":

Dokumen	Audiens	Saluran	Mengapa?
Laporan UAT	Leadership, PM, Tech Lead, Stakeholder Bisnis	Notion + PDF (lampiran email)	<i>Stakeholder</i> membutuhkan ringkasan komprehensif untuk pengambilan keputusan rilis. Notion sebagai <i>source of truth</i> internal, PDF untuk formalitas dan kemudahan distribusi ke pihak eksternal atau yang prefer email.

Catatan Rilis

Pengguna,
Marketing,
Support

Notion + Slack
`#announcements`
+ Email Blast
(oleh Marketing)

Pengguna perlu tahu apa yang baru/berubah. Marketing menggunakannya untuk promosi. Support menggunakannya untuk menjawab pertanyaan pengguna. Slack untuk pengumuman cepat, Notion sebagai arsip, Email Blast untuk jangkauan luas.

**Dokumentasi
Pengujian
Jangka Panjang**

Tim Internal (QA,
Dev)

Notion (QA
Knowledge Base)

Ini adalah *knowledge base* internal yang terus diperbarui. Notion adalah platform kolaboratif yang ideal untuk dokumen hidup seperti ini. Tidak perlu distribusi luas ke seluruh perusahaan.

Laporan
Eksekusi
Pengujian

PM, Tech Lead

Notion + Slack
#qa (ringkasan
harian)

PM dan Tech
Lead perlu
update
harian/mingguan
tentang progres
pengujian. Slack
untuk komunikasi
cepat dan
informal, Notion
untuk detail dan
arsip.

Pentingnya Memilih Saluran yang Tepat:

- **Notion:** Berfungsi sebagai *source of truth* terpusat dan kolaboratif untuk sebagian besar dokumen QA. Ini memungkinkan *link* antar dokumen dan pencarian yang efisien.
- **PDF/Email:** Digunakan untuk dokumen formal yang membutuhkan persetujuan atau distribusi ke audiens yang mungkin tidak aktif di Notion.
- **Slack #announcements:** Untuk pengumuman rilis yang cepat dan menjangkau seluruh tim internal.
- **Slack #qa:** Untuk *update* harian atau ringkasan yang lebih teknis khusus untuk tim pengembangan dan produk.
- **Drive Folder:** Untuk menyimpan aset yang mungkin dibutuhkan oleh tim lain, seperti gambar untuk marketing atau versi final yang mudah diunduh.

Memahami audiens dan saluran distribusi adalah kunci untuk memastikan informasi Anda tidak hanya *ada*, tetapi juga *digunakan* dan *dipahami* oleh pihak yang tepat.

Rangkuman

Di bab ini, Anda telah mempelajari bahwa peran QA Engineer jauh melampaui sekadar eksekusi pengujian. Kemampuan untuk mendokumentasikan dan mengkomunikasikan hasil pengujian secara efektif adalah keterampilan yang sangat berharga. Kita telah membahas empat jenis dokumen utama:

1. **Laporan UAT:** Dokumen krusial untuk *stakeholder* yang merangkum hasil pengujian penerimaan pengguna dan mendapatkan *sign-off* resmi untuk rilis.
2. **Catatan Rilis:** Dokumen yang berhadapan dengan pengguna akhir, menginformasikan tentang fitur baru, perbaikan, dan isu yang diketahui dalam rilis terbaru.
3. **Dokumentasi Pengujian Jangka Panjang:** *Knowledge base* internal yang vital untuk tim QA, mencakup lingkungan, data, infrastruktur otomatisasi, *checklist regresi*, dan pola *bug*.
4. **Laporan Eksekusi Pengujian:** Catatan harian/mingguan tentang progres pengujian yang juga menjadi input untuk Laporan UAT.

Anda juga telah memahami pentingnya menggunakan konvensi header yang seragam untuk semua dokumen dan bagaimana mendistribusikan setiap dokumen ke audiens yang tepat melalui saluran yang paling efektif. Dengan menguasai aspek pelaporan dan dokumentasi ini, Anda tidak hanya menjadi *tester* yang handal, tetapi juga *quality gate* yang mampu menjembatani celah antara teknis dan bisnis, memastikan bahwa kualitas produk tidak hanya tercapai, tetapi juga dipahami dan dihargai oleh semua orang.

Checklist Actionable

Berikut adalah beberapa langkah yang dapat Anda terapkan segera setelah membaca bab ini:

- [] **Identifikasi *stakeholder* utama** untuk proyek Anda saat ini dan pahami kebutuhan informasi mereka.
- [] **Biasakan diri Anda dengan template Laporan UAT.** Coba isi bagian-bagiannya dengan skenario fiktif untuk melatih pemahaman Anda.
- [] **Buat draf Catatan Rilis** untuk fitur yang sedang Anda kerjakan, fokus pada manfaat pengguna dan hindari jargon teknis.
- [] **Mulai membangun atau mengisi "QA Knowledge Base" di Notion Anda.** Setidaknya, dokumentasikan lingkungan pengujian dan data pengujian yang Anda gunakan.
- [] **Jadwalkan "documentation hour" mingguan atau bulanan** untuk meninjau dan memperbarui dokumentasi pengujian jangka panjang Anda.
- [] **Pastikan setiap dokumen yang Anda hasilkan memiliki header yang konsisten** sesuai konvensi yang dibahas.

- [] **Verifikasi saluran distribusi** yang Anda gunakan untuk setiap jenis dokumen, apakah sudah tepat untuk audiensnya.
 - [] **Berlatih menulis ringkasan eksekutif** yang ringkas dan informatif untuk laporan Anda.
 - [] **Diskusikan dengan tim Anda** tentang pentingnya *sign-off* pada Laporan UAT dan bagaimana prosesnya akan dilakukan.
-

Chapter 06: Stakeholder Engagement — Demo, Meeting, UAT Coordination

Selamat datang kembali, petualang QA! Hingga chapter ini, kita telah menyelami berbagai aspek teknis peran Quality Assurance: dari menganalisis *requirement* yang kompleks, merancang *test case* yang komprehensif, mengeksekusi pengujian dengan berbagai metode, hingga melaporkan *bug* yang *actionable* dan mendokumentasikan hasil pengujian. Semua itu adalah fondasi yang sangat kuat untuk seorang QA Engineer.

Namun, pekerjaan QA tidak berhenti di meja Anda sendiri. Mungkin Anda sudah mulai menyadari bahwa *output* dari pekerjaan Anda — seperti laporan *bug*, hasil eksekusi *test case*, atau laporan UAT — tidak hanya dikonsumsi oleh Anda atau tim teknis semata. Sebaliknya, informasi tersebut menjadi jembatan komunikasi yang menghubungkan berbagai tim dan pemangku kepentingan.

Inilah inti dari *stakeholder engagement*: kemampuan seorang QA untuk berinteraksi, berkomunikasi, dan berkolaborasi secara efektif dengan berbagai pihak yang terlibat dalam siklus pengembangan produk. Dari Product Manager (PM), Developer, Tech Lead, hingga tim Marketing, bahkan *end-user* atau *business stakeholder* yang akan menggunakan fitur yang Anda uji. QA bukan sekadar 'tukang tes' yang bekerja di balik layar, melainkan seorang *quality gate* dan *communicator* yang memastikan semua orang memiliki pemahaman yang sama tentang kualitas produk.

Di chapter ini, kita akan membahas bagaimana Anda dapat menjadi QA Engineer yang tidak hanya andal secara teknis, tetapi juga mahir dalam berinteraksi dengan *stakeholder*. Kita akan melihat cara mendemonstrasikan fitur dengan percaya diri dan aman, bagaimana berkontribusi secara berbobot dalam berbagai jenis *meeting*, dan bagaimana mengoordinasikan *User Acceptance Testing* (UAT) yang efektif. Mari kita mulai perjalanan ini!

1. Demo Fitur: Strategi Aman untuk Memamerkan Kualitas

Mendemonstrasikan fitur yang baru dikembangkan kepada para pemangku kepentingan adalah momen krusial bagi tim produk dan engineering. Ini adalah kesempatan untuk menunjukkan kerja keras yang telah dilakukan, mengumpulkan *feedback*, dan mendapatkan persetujuan untuk langkah selanjutnya. Bagi seorang QA, momen demo juga merupakan validasi akhir bahwa fitur tersebut telah memenuhi standar kualitas dan siap untuk dipertontonkan.

Namun, ada satu fakta pahit tentang demo: **demo langsung (live demo) itu berbahaya**. Betul, Anda tidak salah dengar. Seringkali, saat demo langsung, hal-hal yang tidak terduga bisa terjadi. Koneksi internet tiba-tiba putus, data demo tidak sengaja terhapus, *environment* demo mendadak *down*, atau bahkan *bug* yang luput dari perhatian muncul di detik-detik terakhir. Ini bisa sangat memalukan dan merusak reputasi tim, apalagi jika terjadi di depan *stakeholder* penting.

Sebagai Quality Assurance, Anda memiliki peran penting untuk memastikan demo berjalan lancar dan tanpa drama. Bagaimana strateginya?

Strategi Demo: Pra-Rekaman + Skrip Live + Rekaman Cadangan

Strategi terbaik untuk demo yang aman dan efektif adalah kombinasi dari persiapan matang dan rencana cadangan yang kuat:

Rekaman Otomatis (Pre-recorded Automation Run):

- Buatlah rekaman video dari skrip otomatisasi Anda yang berjalan, yang secara naratif menjelaskan alur fitur dari awal hingga akhir.
- Rekaman ini tidak hanya menunjukkan bahwa fitur berfungsi, tetapi juga memamerkan kemampuan otomatisasi Anda.

- Narasi dalam video harus jelas, ringkas, dan fokus pada *value* yang diberikan fitur.

Demo Langsung Berbasis Skrip (Scripted Live Demo):

- Idealnya, Anda tetap akan melakukan demo secara langsung. Namun, demo langsung ini harus **berbasis skrip** dan menggunakan **skrip otomatisasi yang sama** dengan yang Anda rekam.
- Pastikan Anda menjalankan skrip otomatisasi ini dalam kondisi *environment* demo yang stabil dan data yang terkontrol.
- Dengan skrip otomatisasi, Anda meminimalkan interaksi manual yang bisa memicu kesalahan dan memastikan setiap langkah demo konsisten.

Rekaman Cadangan Siap Akses (Backup Recording):

- Ini adalah jaring pengaman Anda. Rekaman otomatis yang Anda buat sebelumnya harus **diunggah ke Drive** atau *platform cloud* lain yang mudah diakses.
- Siapkan tautan *Slack-ready* ke rekaman ini. Jika terjadi kegagalan tak terduga saat demo langsung (misalnya, *environment* tiba-tiba *down*), Anda bisa langsung beralih ke rekaman cadangan dengan cepat dan mulus.

Dengan strategi ini, Anda tidak hanya memastikan kelancaran demo, tetapi juga menunjukkan profesionalisme dan kesiapan tim.

Checklist Persiapan Demo

Sebelum demo, pastikan Anda telah melewati *checklist* ini:

[] Data Demo dalam Kondisi Baik di *Environment* Demo:

- Pastikan *environment* demo menggunakan data yang relevan, realistis, dan bersih. Hindari data yang aneh atau tidak lengkap yang bisa membingungkan *stakeholder*.
- Jika perlu, siapkan data khusus untuk demo yang menyoroti kasus penggunaan kunci.
- Contoh: Untuk demo fitur *checkout*, pastikan ada produk di keranjang, alamat pengiriman yang valid, dan metode pembayaran yang aktif.

[] Skrip Otomatisasi Diuji 24 Jam Sebelum Demo:

- Jangan pernah berasumsi skrip otomatisasi Anda akan selalu berfungsi. Jalankan skrip setidaknya sehari sebelum demo untuk memastikan tidak ada perubahan mendadak di *environment* atau kode yang memecah skrip Anda.

- Ini memberi Anda waktu untuk memperbaiki masalah kecil jika ada.

[] Rekaman Cadangan Diunggah ke Drive:

- Setelah rekaman otomatisasi selesai, segera unggah ke *platform cloud* seperti Google Drive, OneDrive, atau Dropbox.

- Pastikan izin aksesnya sudah diatur dengan benar agar *stakeholder* bisa melihatnya jika diperlukan.

[] Tautan Rekaman Siap di Slack untuk Cadangan Instan:

- Siapkan tautan langsung ke rekaman cadangan Anda. Simpan di *notes* pribadi atau di saluran Slack yang relevan.

- Jika terjadi masalah saat demo langsung, Anda bisa segera membagikan tautan ini dan melanjutkan presentasi tanpa banyak jeda.

Contoh Skenario Demo: Thomas Menyelamatkan Hari

Thomas, seorang QA Engineer di Atmoscheck, akan mendemonstrasikan fitur "Integrasi Laporan Cuaca Kustom" kepada tim manajemen. Dia telah menyiapkan skrip otomatisasi Playwright untuk melewati alur fitur tersebut.

Persiapan Thomas:

- Thomas memastikan *environment staging* memiliki akun pengguna dengan langganan premium yang memungkinkan integrasi kustom.
- Dia menjalankan skrip otomatisasi sehari sebelumnya dan merekamnya, lalu mengunggah video ke Google Drive dengan tautan yang dapat dibagikan. Tautan ini disimpan di *notes* dan juga di *draft* pesan Slack.

Saat Demo:

- Thomas memulai demo langsung, menjalankan skrip otomatisasi Playwright. Semua berjalan lancar, dan *stakeholder* terkesan dengan fluiditas demonya.

- Namun, di tengah-tengah demo, koneksi internet di ruang rapat tiba-tiba terputus. Panik mulai menyelimuti.

- **Tindakan Thomas:** Dengan tenang, Thomas berkata, "Mohon maaf atas gangguan koneksi ini. Sebagai rencana cadangan, saya telah menyiapkan rekaman demo lengkap. Saya akan membagikan tautannya di chat sekarang, dan kita bisa melanjutkan presentasi dengan video tersebut." Dia langsung menempelkan tautan Drive yang sudah disiapkan di *meeting chat*.

- Demo dapat dilanjutkan tanpa hambatan berarti, dan *stakeholder* menghargai kesiapan Thomas.

Ini menunjukkan bahwa persiapan yang matang dan strategi cadangan adalah kunci untuk demo yang sukses dan bebas stres.

2. Berpartisipasi dalam Meeting: Kontribusi Berbobot dari QA

Sebagai QA Engineer, Anda akan menghabiskan sebagian waktu Anda di berbagai jenis *meeting*. Ini bukan hanya tentang 'hadir', tetapi tentang 'berkontribusi'. Peran QA dalam *meeting* sangat vital karena Anda membawa perspektif kualitas, potensi risiko, dan *estimasi* pengujian yang tidak dimiliki oleh peran lain.

Berikut adalah beberapa jenis *meeting* umum yang akan Anda hadiri, beserta peran dan kontribusi Anda di dalamnya:

Jenis-jenis Meeting dan Peran QA

Meeting	Frekuensi	Peran QA
Sprint Planning	Mingguan/Dua Mingguan	Menyuarakan estimasi pengujian, mengidentifikasi risiko

Daily Standup	Harian	Berbagi status pengujian, mengidentifikasi <i>blocker</i>
Backlog Grooming	Mingguan	Mengidentifikasi isu <i>testability</i> pada pekerjaan yang akan datang
Sprint Review	Akhir Sprint	Demo fitur bersama PM, berbagi laporan QA
Sprint Retrospective	Akhir Sprint	Mengemukakan peningkatan proses QA
Bug Triage	Sesuai Kebutuhan	Memimpin prioritisasi <i>bug</i> dengan rubrik <i>severity</i>
Pre-release Sync	Per Rilis	Pemilik tanda tangan persetujuan rilis (<i>sign-off owner</i>)

Mari kita bahas lebih detail peran dan kontribusi Anda di setiap *meeting*:

a. Sprint Planning

- **Tujuan:** Merencanakan pekerjaan yang akan dilakukan dalam *sprint* berikutnya.

Kontribusi QA:

- **Memberikan Estimasi Pengujian:** Berikan estimasi waktu yang realistis untuk pengujian setiap *user story* atau tugas. Ini mencakup waktu untuk analisis, desain *test case*, eksekusi, dan pelaporan.
- **Mengidentifikasi Risiko Kualitas:** Soroti potensi risiko yang mungkin muncul selama pengujian, seperti ketergantungan pada sistem lain, kompleksitas fitur, atau kurangnya data uji.
- **Memastikan *Definition of Done* (DoD) Mencakup Pengujian:** Pastikan bahwa "selesai" untuk setiap tugas berarti juga telah melewati pengujian QA yang memadai.

- **Menanyakan Klarifikasi:** Jika ada *user story* yang kurang jelas atau memiliki *acceptance criteria* yang ambigu, inilah saatnya untuk menanyakan klarifikasi kepada PM atau Developer.

Contoh: "Untuk *user story* 'Integrasi Pembayaran Baru', saya estimasi butuh 2 hari penuh untuk pengujian fungsional dan *negative testing*, mengingat ada banyak skenario *error* dari pihak ketiga. Kita juga perlu memastikan ada *test account* yang valid untuk setiap metode pembayaran."

b. Daily Standup

- **Tujuan:** Sinkronisasi harian tentang kemajuan, *blocker*, dan rencana kerja.

Kontribusi QA:

- **Bagikan Status Pengujian:** Beri tahu tim apa yang sedang Anda uji, apa yang sudah selesai, dan apa yang akan Anda lakukan selanjutnya.
- **Laporkan Blocker:** Segera sampaikan jika ada masalah yang menghambat pekerjaan Anda (misalnya, *bug* kritis, *environment* tidak stabil, ketergantungan yang belum terpenuhi).
- **Sebutkan Potensi Risiko:** Jika Anda melihat *bug* yang berpotensi memengaruhi *user story* lain, sampaikan di sini.

Contoh: "Kemarin saya selesai menguji modul *login* baru, dan semua *test case* positif lolos. Hari ini saya akan mulai *negative testing* dan *edge case*. Saya menemukan *bug* di mana *password* yang terlalu panjang tidak divalidasi dengan benar, sudah saya laporkan."

c. Backlog Grooming (atau Refinement)

- **Tujuan:** Memastikan *item backlog* jelas, terdefinisi dengan baik, dan siap untuk *sprint planning* berikutnya.

Kontribusi QA:

- **Identifikasi Isu Testability:** Tinjau *user story* yang akan datang dan identifikasi apakah ada aspek yang sulit atau tidak mungkin diuji, atau membutuhkan persiapan khusus.
- **Sarankan Acceptance Criteria Tambahan:** Bantu PM untuk menyempurnakan *acceptance criteria* agar lebih spesifik dan terukur dari perspektif pengujian.

- **Estimasi Awal:** Berikan perkiraan kasar tentang upaya pengujian yang diperlukan untuk setiap *item*.

Contoh: "Untuk fitur 'Notifikasi Push Personalisasi', *acceptance criteria* saat ini hanya menyebut 'notifikasi terkirim'. Kita perlu spesifikasi lebih lanjut tentang *trigger*, konten notifikasi untuk berbagai skenario pengguna, dan bagaimana kita akan memverifikasi pengiriman notifikasi di berbagai perangkat."

d. Sprint Review

- **Tujuan:** Mendemonstrasikan pekerjaan yang telah selesai kepada *stakeholder* dan mengumpulkan *feedback*.

Kontribusi QA:

- **Demo Bersama PM:** Anda dapat membantu PM dalam mendemonstrasikan fitur, atau bahkan memimpin bagian demo yang menyoroti kualitas atau skenario *edge case* tertentu.
- **Berbagi Laporan QA:** Sajikan ringkasan singkat tentang status kualitas *sprint* tersebut, termasuk jumlah *test case* yang dieksekusi, *bug* yang ditemukan dan diperbaiki, serta hasil UAT jika sudah dilakukan.
- **Menjawab Pertanyaan Teknis:** Siap menjawab pertanyaan *stakeholder* terkait stabilitas, performa, atau masalah kualitas yang mungkin mereka miliki.

Contoh: "Selain demo fitur yang sudah ditampilkan PM, saya ingin menambahkan bahwa kami telah menjalankan 150 *test case* untuk *sprint* ini, dengan tingkat keberhasilan 98%. Dua *bug* kritis yang ditemukan telah diperbaiki dan diverifikasi. Kita juga sudah mendapatkan *sign-off* dari UAT untuk modul *dashboard* baru."

e. Sprint Retrospective

- **Tujuan:** Merefleksikan *sprint* yang baru saja berakhir dan mengidentifikasi area untuk perbaikan proses.

Kontribusi QA:

- **Mengemukakan Peningkatan Proses QA:** Bagikan observasi tentang apa yang berjalan baik dan apa yang bisa ditingkatkan dari perspektif QA (misalnya, *bug* yang terlambat ditemukan, kesulitan dalam setup *environment*, komunikasi yang kurang efektif).

- **Mencari Solusi Bersama:** Berpartisipasi aktif dalam diskusi untuk menemukan solusi atas masalah yang diangkat.

- **Fokus pada Proses, Bukan Individu:** Pastikan *feedback* konstruktif dan berfokus pada perbaikan sistem atau proses, bukan menyalahkan individu.

Contoh: "Saya perhatikan ada beberapa *bug* yang baru muncul di akhir *sprint* karena integrasi antar modul belum dilakukan lebih awal. Mungkin kita bisa mencoba strategi *integration testing* yang lebih agresif di tengah *sprint* agar *bug* ini bisa terdeteksi lebih cepat."

f. Bug Triage

- **Tujuan:** Meninjau *bug* yang dilaporkan, menentukan *severity*, *priority*, dan rencana tindakan.

Kontribusi QA:

- **Memimpin Prioritisasi dengan Rubrik *Severity*:** Sebagai pelapor *bug* dan penilai dampak, Anda adalah orang terbaik untuk memimpin diskusi tentang *severity* dan *priority bug*. Gunakan rubrik P0-P3 yang telah disepakati (lihat Chapter 04).

- **Memberikan Konteks Mendalam:** Jelaskan dampak *bug* dari perspektif pengguna dan bisnis.

- **Verifikasi Reproduksi:** Pastikan *bug* masih dapat direproduksi dan berikan langkah-langkah yang jelas.

Contoh: "Berdasarkan rubrik kita, *bug* 'Pembayaran Gagal Tanpa Pesan Error' adalah P0 karena menghentikan pengguna menyelesaikan transaksi inti dan tidak ada *workaround*. Ini harus segera diperbaiki."

g. Pre-release Sync

- **Tujuan:** Sinkronisasi terakhir sebelum rilis produk, memastikan semua siap.

Kontribusi QA:

- **Pemilik Tanda Tangan Persetujuan Rilis (*Sign-off Owner*):** Ini adalah salah satu peran paling krusial Anda. Anda adalah *gatekeeper* terakhir. Anda harus memberikan persetujuan (atau menolak) rilis berdasarkan status kualitas saat ini.

- **Sajikan Laporan Kualitas Final:** Siapkan ringkasan tentang status *bug* yang diketahui, hasil pengujian regresi, dan persetujuan UAT.

- **Identifikasi Risiko Rilis:** Sampaikan risiko yang tersisa (jika ada) dan dampak potensialnya.

Contoh: "Berdasarkan semua pengujian yang telah kami lakukan, termasuk regresi otomatis dan UAT yang telah disetujui, saya memberikan *sign-off* untuk rilis versi 2.3. Semua *bug* kritis telah diperbaiki dan diverifikasi. Ada dua *bug* minor yang diketahui, namun dampaknya minimal dan tidak memblokir fungsionalitas utama."

Dengan berpartisipasi secara aktif dan berbobot dalam *meeting-meeting* ini, Anda tidak hanya meningkatkan visibilitas peran QA, tetapi juga berkontribusi secara signifikan terhadap keberhasilan tim dan kualitas produk secara keseluruhan.

3. Koordinasi UAT (User Acceptance Testing): Menjembatani Bisnis dan Teknis

User Acceptance Testing (UAT) adalah tahap pengujian di mana *business stakeholder* atau pengguna aktual memvalidasi bahwa fitur atau produk yang dikembangkan telah memenuhi kebutuhan bisnis mereka dan siap untuk digunakan. Ini adalah "jembatan" antara tim teknis dan pengguna akhir.

Peran Anda sebagai QA dalam UAT adalah sebagai **koordinator, dokumentator, dan fasilitator**. Anda memastikan proses UAT berjalan lancar, *feedback* terkumpul dengan baik, dan keputusan dapat diambil berdasarkan hasil UAT.

Tahapan Koordinasi UAT

a. Pra-UAT (Pre-UAT)

Tahap ini adalah tentang persiapan agar UAT berjalan terstruktur.

[] Buat Rencana UAT:

- **Siapa yang Menguji Apa?** Identifikasi partisipan UAT (misalnya, PM, *Business Analyst*, *Key Users*). Tetapkan modul atau fitur spesifik yang akan diuji oleh masing-masing partisipan.

- **Kapan?** Tentukan jadwal UAT, durasi, dan *deadline* untuk pengumpulan *feedback*.

- **Kriteria Keberhasilan (Success Criteria):** Apa yang dianggap sebagai "lulus" UAT? Misalnya, "Semua alur utama dapat diselesaikan tanpa *bug* kritis," atau "80% dari skenario UAT berhasil diselesaikan."

- **Lingkungan UAT:** Pastikan ada *environment* khusus untuk UAT yang stabil dan terpisah dari *staging* atau *production*.

[] Siapkan Skrip UAT:

- Skrip UAT lebih sederhana daripada *test case* QA. Ditulis untuk orang non-QA, dengan bahasa yang mudah dipahami.
- Fokus pada alur bisnis utama dan skenario pengguna yang realistis.
- Sertakan langkah-langkah jelas, data yang harus digunakan, dan hasil yang diharapkan.
- Contoh: "Langkah 1: Buka halaman produk X. Langkah 2: Tambahkan produk ke keranjang. Hasil yang diharapkan: Produk X muncul di keranjang dengan harga yang benar."

[] Isi *Environment* UAT dengan Data Realistis:

- Data uji yang realistis sangat penting. Hindari menggunakan data *dummy* yang tidak masuk akal.
- Jika memungkinkan, gunakan salinan data *production* yang sudah di-*anonymize* atau buat data yang merepresentasikan skenario bisnis nyata.
- Pastikan data tersebut mencakup berbagai kasus (misalnya, pengguna dengan langganan berbeda, produk dengan stok kosong, dll.).

[] Latih Partisipan (Sesi Orientasi 10 Menit):

- Jangan berasumsi partisipan UAT akan langsung mengerti cara menggunakan *environment* atau mengisi *feedback*.
- Adakan sesi singkat (sekitar 10-15 menit) untuk:
- Menjelaskan tujuan UAT.
- Menunjukkan cara mengakses *environment* UAT.
- Menjelaskan cara menggunakan skrip UAT.

- Menjelaskan bagaimana dan di mana mereka harus melaporkan *feedback* atau *bug*.
- Menjawab pertanyaan awal.

Contoh Dialog Pelatihan UAT: "Selamat pagi Bapak/Ibu, terima kasih sudah meluangkan waktu untuk UAT fitur 'Manajemen Pesanan Baru'. Dalam sesi singkat ini, saya akan menunjukkan cara mengakses *environment* UAT kita di `uat.atmoscheck.com`, bagaimana menjalankan skrip UAT yang sudah saya siapkan, dan yang paling penting, bagaimana melaporkan temuan Anda di Notion. Fokus kita adalah memastikan fitur ini memenuhi kebutuhan bisnis kita. Jika ada *bug* atau *feedback* terkait pengalaman pengguna, mohon dicatat dengan detail."

b. Selama UAT (During UAT)

Selama UAT berlangsung, Anda adalah titik kontak utama dan fasilitator.

[] Siap Menjawab Pertanyaan:

- Dedikasikan saluran Slack (misalnya, #uat-manajemen-pesanan) atau *channel* komunikasi lain di mana partisipan dapat mengajukan pertanyaan secara *real-time*.
- Respon dengan cepat dan bantu memecahkan masalah kecil (misalnya, "Saya tidak bisa *login*," "Data ini kenapa kosong?").
- Ini membantu menjaga momentum UAT dan mengurangi frustrasi partisipan.

[] Tangkap *Feedback* Secara *Real-time*:

- Gunakan *tool* seperti Notion, Jira, atau Google Sheets untuk mencatat semua temuan.

Buat satu tugas/item per temuan (*finding*). Pastikan setiap entri mencakup:

- Deskripsi jelas tentang masalah.
- Langkah-langkah untuk mereproduksi (jika *bug*).
- *Screenshot* atau video (jika relevan).
- Nama pelapor.
- *Severity* awal (jika *bug*).
- Jenis temuan (bug, misunderstanding, new request).

[] Triage Temuan:

Saat *feedback* masuk, segera *triage* (pilah) temuan tersebut ke dalam kategori:

- **Bug Aktual:** Masalah yang memang merupakan *defect* dalam kode atau implementasi. Laporkan ini ke tim Developer.
- **Kesalahpahaman (Misunderstanding):** Partisipan UAT mungkin salah memahami fungsionalitas atau cara penggunaan. Berikan klarifikasi.
- **Permintaan Baru (New Request):** Partisipan mungkin menyarankan fitur baru atau peningkatan yang tidak termasuk dalam *scope* awal. Catat ini sebagai *feedback* untuk *backlog* di masa mendatang, bukan *bug*.

c. Pasca-UAT (Post-UAT)

Setelah UAT selesai, ada beberapa langkah penting untuk merangkum dan menindaklanjuti.

[] Buat Laporan UAT (UAT Report):

- Rangkum semua temuan, statusnya (diperbaiki, ditunda, bukan *bug*), dan keputusan yang diambil.
- Sertakan daftar partisipan, tanggal UAT, dan kriteria keberhasilan yang tercapai/tidak tercapai.
- (Lihat Chapter 05 untuk detail lebih lanjut tentang pembuatan laporan).

[] Kumpulkan *Sign-off*:

- Dapatkan persetujuan resmi dari PM, Tech Lead, dan *Business Stakeholder* bahwa fitur telah memenuhi harapan mereka dan siap untuk rilis.
- *Sign-off* ini adalah dokumen penting yang menunjukkan bahwa semua pihak telah menyetujui kualitas dan fungsionalitas fitur.

[] Jika *Sign-off* Terhambat:

- Jika ada *bug* kritis yang ditemukan selama UAT dan belum diperbaiki, atau jika *stakeholder* merasa fitur belum memenuhi kebutuhan, *sign-off* mungkin tidak diberikan.

- Dalam kasus ini, Anda perlu:
- Melaporkan *bug* yang menghambat *sign-off*.
- Bekerja sama dengan tim *engineering* untuk memperbaikinya.
- Mengulang sebagian atau seluruh proses UAT setelah perbaikan.

[] Jika *Sign-off* Diterima:

- Segera beri tahu PM dan tim *engineering* bahwa fitur "Siap untuk Rilis". Ini adalah sinyal bahwa semua pintu kualitas telah dibuka.

Koordinasi UAT yang baik memastikan bahwa produk yang dirilis tidak hanya berfungsi dengan benar secara teknis, tetapi juga memenuhi ekspektasi dan kebutuhan bisnis pengguna.

4. Handoff: Notifikasi PM saat Siklus QA Selesai

Setelah semua pengujian selesai, *bug* kritis diperbaiki, dan UAT telah disetujui, tiba saatnya untuk secara resmi memberi tahu Product Manager bahwa siklus QA untuk fitur tersebut telah selesai. Ini adalah **handoff** yang penting, menandakan bahwa tanggung jawab selanjutnya ada di tangan PM untuk merencanakan rilis.

Handoff ini harus jelas, ringkas, dan berisi semua informasi yang relevan. Ini bukan sekadar "sudah selesai ya", tetapi sebuah notifikasi formal yang berisi rangkuman status dan artefak penting.

```
from handoff import handoff_to_pm
[
    handoff_to_pm(
        from_agent="qa-thomas",
        title=f"QA complete: {feature_name}",
        priority="P1",
        epic_id=epic_id,
        summary="UAT signed off. Test report attached. Ready for release.",
        body_markdown=f"""
## Status
```

Siklus QA selesai. Semua kriteria **exit** telah terpenuhi.

Artefak

- Strategi Pengujian: {strategy_url}
- Test Case: {test_cases_url}
- Laporan Eksekusi: {report_url}
- Laporan UAT: {uat_url}
- Draf Release Notes: {notes_url}

Langkah Selanjutnya yang Direkomendasikan

- PM: Konfirmasi jendela rilis
 - Marketing: Handoff peluncuran (sesuai arahan Pimpinan Thomas tentang koordinasi Marketing)
 - Tech Lead: Rencana **deployment**
- "" ,
)

Mari kita bedah template `handoff_to_pm` ini:

- **from_agent="qa-thomas"**: Menunjukkan siapa yang melakukan *handoff*. Ini penting untuk akuntabilitas.
- **title=f"QA complete: {feature_name}"**: Judul yang jelas dan langsung menunjukkan fitur mana yang telah selesai diuji.
- **priority="P1"**: Menunjukkan bahwa ini adalah notifikasi penting yang membutuhkan perhatian segera dari PM.
- **epic_id=epic_id**: Mengaitkan *handoff* ini dengan *epic* atau proyek yang lebih besar.
- **summary="UAT signed off. Test report attached. Ready for release."**: Ringkasan singkat yang memberikan gambaran cepat tentang status.

body_markdown: Ini adalah bagian paling detail, berisi informasi penting dalam format Markdown.

- **## Status**: Konfirmasi bahwa siklus QA telah selesai dan semua kriteria *exit* (seperti semua *test case* kritis lolos, *bug* P0/P1 ditutup, UAT selesai) telah terpenuhi.

Artefak: Daftar tautan ke semua dokumen penting yang dihasilkan selama siklus QA. Ini memudahkan PM untuk mengakses bukti pengujian dan laporan.

- **strategy_url**: Tautan ke dokumen strategi pengujian.
- **test_cases_url**: Tautan ke kumpulan *test case*.
- **report_url**: Tautan ke laporan eksekusi pengujian.

- `uat_url`: Tautan ke laporan UAT.

- `notes_url`: Tautan ke draf *release notes* (yang mungkin sudah Anda bantu siapkan, seperti dibahas di Chapter 05).

Langkah Selanjutnya yang Direkomendasikan: Ini adalah bagian proaktif. Anda tidak hanya melaporkan status, tetapi juga menyarankan tindakan selanjutnya untuk PM dan tim lain. Ini menunjukkan bahwa Anda berpikir ke depan dan membantu melancarkan proses rilis.

- Untuk PM: Konfirmasi jadwal rilis.
- Untuk Marketing: Koordinasi peluncuran (jika ada).
- Untuk Tech Lead: Rencana *deployment*.

Melakukan *handoff* seperti ini menunjukkan profesionalisme, efisiensi, dan memastikan tidak ada miskomunikasi tentang status kualitas fitur. Ini adalah momen Anda untuk menyerahkan tongkat estafet ke PM dengan percaya diri.

5. Perilaku Buruk yang Harus Dihindari oleh QA

Dalam perjalanan Anda sebagai QA Engineer, akan ada godaan atau kebiasaan buruk yang bisa merusak kredibilitas Anda dan kualitas produk. Mengenali dan menghindari perilaku ini sangat penting untuk menjadi QA yang efektif dan dihormati.

Berikut adalah beberapa perilaku buruk yang harus Anda hindari:

a. Menyetujui Rilis di Bawah Tekanan padahal Kriteria Belum Terpenuhi

- **Perilaku Buruk:** Anda tahu ada *bug* kritis yang belum diperbaiki, atau UAT belum selesai, tetapi tim ingin segera rilis. Anda merasa tertekan untuk memberikan *sign-off* agar tidak menghambat jadwal.
- **Mengapa Buruk:** Anda adalah *gatekeeper* kualitas. Jika Anda menyetujui rilis di bawah tekanan, Anda mengorbankan kualitas produk dan berpotensi menyebabkan masalah serius bagi pengguna dan reputasi perusahaan. Ini juga merusak kredibilitas Anda sebagai QA.

Yang Seharusnya: Tegastlah pada kriteria *exit* yang telah disepakati. Jelaskan risiko yang ada dan dampak potensialnya jika rilis dilakukan tanpa memenuhi kriteria. Tawarkan solusi alternatif, seperti menunda rilis, melakukan *patch* cepat, atau merilis sebagian fitur.

Contoh Dialog: PM: "Thomas, kita harus rilis besok pagi. Bisnis sudah menunggu. Tolong *sign-off* ya." Thomas: "Maaf, [Nama PM]. Saya belum bisa memberikan *sign-off*. Bug P0 'Pembayaran Gagal' masih terbuka, dan itu akan memblokir pengguna utama kita. Jika kita rilis dengan *bug* ini, kita berisiko kehilangan banyak transaksi dan merusak pengalaman pengguna." PM: "Tapi kita sudah janji!" Thomas: "Saya mengerti urgensinya. Alternatifnya, kita bisa menunda rilis sehari untuk memperbaiki *bug* ini, atau kita bisa melakukan *rollback* fitur pembayaran ini dan merilis sisanya terlebih dahulu. Mana yang lebih baik menurut Anda?"

b. Melaporkan *Bug* Sekaligus di Akhir Siklus Pengujian

- **Perilaku Buruk:** Anda menemukan *bug* saat pengujian, tetapi Anda menunda pelaporannya dan mengumpulkannya untuk dilaporkan sekaligus di akhir siklus.
- **Mengapa Buruk:** Ini melanggar prinsip "fail fast". *Bug* yang dilaporkan terlambat memberikan waktu yang lebih sedikit bagi *developer* untuk memperbaikinya, meningkatkan *pressure*, dan berpotensi menyebabkan *rework* yang lebih besar karena *developer* mungkin sudah membangun fitur lain di atas *bug* tersebut.
- **Yang Seharusnya:** Laporkan *bug* segera setelah Anda menemukannya. Berikan detail yang jelas agar *developer* dapat mereproduksinya dan memperbaikinya sesegera mungkin. Ini menghemat waktu, mengurangi biaya perbaikan, dan menjaga kualitas tetap tinggi sepanjang siklus pengembangan.

c. Melewatkan UAT untuk Fitur "Sederhana"

- **Perilaku Buruk:** Anda berpikir fitur tertentu terlalu sederhana dan tidak perlu UAT, atau PM mengatakan tidak ada waktu untuk UAT.
- **Mengapa Buruk:** Jika seorang pengguna akan menyentuh fitur tersebut, UAT diperlukan. Fitur yang terlihat sederhana di mata teknis bisa jadi memiliki implikasi bisnis yang kompleks atau alur pengguna yang tidak intuitif. Melewatkan UAT berisiko merilis fitur yang secara teknis berfungsi tetapi tidak memenuhi harapan atau kebutuhan bisnis pengguna.
- **Yang Seharusnya:** Tekankan pentingnya UAT untuk setiap fitur yang akan digunakan oleh *end-user* atau *business stakeholder*. Jika ada batasan waktu, usulkan UAT yang lebih ringkas

atau fokus pada skenario kritis saja, daripada melewatkannya sama sekali.

d. Melakukan Demo Tanpa Latihan

- **Perilaku Buruk:** Anda langsung melakukan demo tanpa persiapan, tanpa latihan, atau tanpa rencana cadangan.
- **Mengapa Buruk:** Demo yang tidak siap bisa menjadi bencana. Masalah teknis, kebingungan dalam navigasi, atau lupa alur bisa membuat Anda terlihat tidak profesional dan merusak kepercayaan *stakeholder* terhadap kualitas produk.
- **Yang Seharusnya:** Ikuti strategi demo yang telah kita bahas di awal chapter ini: pra-rekaman, demo langsung berbasis skrip, dan siapkan rekaman cadangan. Selalu latihan setidaknya sekali sebelum demo sebenarnya. Pastikan *environment* dan data demo dalam kondisi prima.

Menghindari perilaku buruk ini akan membantu Anda membangun reputasi sebagai QA Engineer yang profesional, dapat diandalkan, dan menjadi aset berharga bagi tim dan perusahaan.

Rangkuman

Chapter ini telah membawa kita melampaui aspek teknis QA dan masuk ke ranah yang sama pentingnya: interaksi dengan pemangku kepentingan (*stakeholder engagement*). Kita telah melihat bahwa seorang QA Engineer yang efektif tidak bekerja dalam isolasi, melainkan sebagai pusat komunikasi yang menghubungkan berbagai tim dan memastikan semua orang selaras dalam tujuan kualitas.

Berikut adalah poin-poin penting yang telah kita bahas:

- **Strategi Demo yang Aman:** Demo langsung bisa berbahaya, oleh karena itu, kombinasi rekaman otomatis, demo langsung berbasis skrip, dan rekaman cadangan adalah pendekatan terbaik untuk memastikan demo berjalan lancar dan profesional.
- **Kontribusi Berbobot dalam Meeting:** QA memiliki peran krusial dalam berbagai *meeting* tim, dari *Sprint Planning* hingga *Pre-release Sync*. Kontribusi Anda berupa estimasi pengujian, identifikasi risiko, laporan status, hingga *sign-off* rilis, sangat menentukan arah dan kualitas

produk.

- **Koordinasi UAT yang Efektif:** Sebagai koordinator, Anda memfasilitasi *User Acceptance Testing* dari tahap persiapan (Pra-UAT), pelaksanaan (Selama UAT), hingga tindak lanjut (Pasca-UAT), memastikan produk memenuhi kebutuhan bisnis dan mendapatkan persetujuan dari *stakeholder*.
- **Handoff Formal ke PM:** Setelah siklus QA selesai, penting untuk melakukan *handoff* yang jelas dan terstruktur kepada Product Manager, berisi status, artefak, dan rekomendasi langkah selanjutnya, menggunakan template yang konsisten.
- **Menghindari Perilaku Buruk:** Seorang QA yang kredibel akan menghindari menyetujui rilis di bawah tekanan, menunda pelaporan *bug*, melewatkan UAT untuk fitur "sederhana", atau melakukan demo tanpa persiapan.

Dengan menguasai aspek *stakeholder engagement* ini, Anda tidak hanya akan menjadi seorang QA Engineer yang ahli dalam pengujian, tetapi juga pemimpin yang efektif, komunikator yang ulung, dan jembatan yang vital antara tim teknis dan tujuan bisnis.

Checklist Actionable

Berikut adalah daftar tindakan yang bisa Anda terapkan segera untuk meningkatkan *stakeholder engagement* Anda:

- **[] Siapkan Strategi Demo Lengkap:** Untuk demo fitur berikutnya, buat rekaman otomatis, siapkan skrip untuk demo langsung, dan unggah rekaman cadangan ke Drive dengan tautan yang siap dibagikan.
- **[] Tinjau Daftar Meeting Mingguan Anda:** Identifikasi jenis *meeting* yang Anda hadiri dan siapkan 3-5 poin kontribusi berbobot dari perspektif QA untuk setiap *meeting* tersebut.
- **[] Latih Diri Anda untuk Sprint Planning:** Saat *sprint planning* berikutnya, aktiflah dalam memberikan estimasi pengujian dan mengidentifikasi potensi risiko kualitas untuk setiap *user story*.
- **[] Buat Rencana UAT untuk Fitur Mendatang:** Identifikasi fitur yang akan datang dan mulailah menyusun draf rencana UAT (siapa, apa, kapan, kriteria sukses) serta draf skrip UAT yang sederhana.

- [] **Siapkan Saluran Komunikasi UAT:** Jika Anda akan mengoordinasikan UAT, buat saluran Slack atau *platform* komunikasi khusus untuk pertanyaan *real-time* dari partisipan UAT.
- [] **Gunakan Template Handoff ke PM:** Setelah menyelesaikan siklus QA untuk fitur berikutnya, gunakan template `handoff_to_pm` untuk memberi tahu PM secara formal.
- [] **Evaluasi Perilaku Anda:** Refleksikan apakah Anda pernah melakukan salah satu "perilaku buruk" yang dibahas. Jika ya, rencanakan bagaimana Anda akan menghindarinya di masa depan dan berkomitmen untuk menjadi *gatekeeper* kualitas yang tegas.
- [] **Proaktif dalam Bug Triage:** Saat *bug triage*, siaplah untuk memimpin diskusi prioritas menggunakan rubrik *severity* yang telah disepakati dan berikan konteks dampak *bug* dari sisi bisnis.
- [] **Selalu Latihan Demo:** Sebelum setiap demo, luangkan waktu 15-30 menit untuk berlatih alur, bahkan jika itu hanya demo singkat untuk tim internal.

Selamat mempraktikkan keterampilan *stakeholder engagement* Anda!

Chapter 07: Automation Testing dengan Playwright

Selamat, Anda telah melangkah lebih jauh dalam perjalanan menjadi Quality Assurance Engineer yang andal! Setelah memahami pentingnya analisis *requirement* yang kritis, merancang *test case* yang komprehensif, melaksanakan *testing* secara strategis, melaporkan *bug* dengan efektif, hingga mendokumentasikan dan berinteraksi dengan *stakeholder*, kini saatnya kita membahas pilar berikutnya yang krusial: *Automation Testing*.

Di dunia pengembangan perangkat lunak yang serba cepat, mengandalkan *manual testing* sepenuhnya bisa menjadi hambatan. Proses *testing* yang berulang, terutama untuk *regression testing*, memakan waktu dan rentan terhadap *human error*. Di sinilah *automation testing* berperan. Dengan mengotomatiskan *test case* tertentu, Anda dapat menjalankan *test* lebih cepat, lebih sering, dan dengan konsistensi yang lebih tinggi, membebaskan waktu Anda untuk *exploratory*

testing atau fokus pada skenario yang lebih kompleks.

Dalam bab ini, kita akan menyelami Playwright, sebuah *framework automation testing* modern yang kuat dan fleksibel. Kita akan belajar cara menyiapkan proyek Playwright, memahami struktur yang rapi menggunakan *Page Object Model*, mengeksplorasi *data-driven testing*, dan bagaimana menjalankan *test* serta menganalisis laporannya. Bersiaplah untuk meningkatkan kemampuan QA Anda ke level berikutnya!

Mengapa Playwright? Pilihan di Antara Para Raksasa

Ketika berbicara tentang *automation testing* untuk aplikasi web, ada banyak pilihan *tool* di luar sana. Mungkin Anda pernah mendengar tentang Selenium, Cypress, atau Katalon. Di Fairatmos, kami telah beralih sepenuhnya ke Playwright. Mengapa demikian? Mari kita bedah keunggulan Playwright dibandingkan kompetitornya.

Playwright vs. Selenium

Selenium adalah pelopor di dunia *web automation* dan merupakan *framework* yang sangat matang dengan dukungan komunitas yang besar. Selenium WebDriver memungkinkan Anda mengontrol *browser* dari berbagai bahasa pemrograman (Java, Python, C#, Ruby, JavaScript).

Namun, Selenium memiliki beberapa tantangan: * **Setup yang Kompleks**: Membutuhkan konfigurasi WebDriver terpisah untuk setiap *browser*, yang seringkali memakan waktu dan rentan terhadap masalah kompatibilitas versi. * **Flakiness**: Tes Selenium seringkali *flaky* (tidak konsisten) karena masalah *timing* atau *race condition*. Anda harus menambahkan *explicit waits* atau *sleep* manual, yang memperlambat eksekusi dan membuat *script* kurang stabil. * **Debugging yang Sulit**: Laporan *error* terkadang kurang informatif, dan *debugging* bisa menjadi proses yang rumit.

Playwright hadir untuk mengatasi banyak masalah ini. Playwright dibangun dengan arsitektur yang lebih modern, memungkinkan interaksi langsung dengan *browser* tanpa perantara WebDriver.

Playwright vs. Cypress

Cypress adalah *framework* yang relatif baru dan sangat populer di kalangan *developer* karena kemudahan penggunaannya dan pengalaman *developer* yang luar biasa. Cypress berjalan di dalam *browser*, yang membuatnya sangat cepat dalam eksekusi dan *debugging*.

Meskipun demikian, Cypress memiliki beberapa batasan: * **Dukungan Browser Terbatas:** Secara historis, Cypress hanya mendukung *browser* berbasis Chromium dan Firefox. Dukungan WebKit (Safari) baru ditambahkan belakangan dan masih dalam pengembangan. * **Single-Origin Policy:** Cypress terikat pada *single-origin policy browser*, yang menyulitkan pengujian skenario yang melibatkan interaksi dengan *domain* atau *tab* yang berbeda. * **Tidak Ada Akses API Langsung:** Meskipun bisa melakukan *network requests*, Cypress tidak dirancang untuk *API testing first-class* seperti Playwright.

Playwright, di sisi lain, menawarkan dukungan penuh untuk Chromium, Firefox, dan WebKit (Safari) secara *out-of-the-box*. Playwright juga dapat menguji skenario multi-*tab* dan multi-*domain* dengan mudah, serta memiliki dukungan *API testing* yang sangat baik.

Playwright vs. Katalon

Katalon Studio adalah *tool automation testing* yang *low-code/no-code*, yang sangat cocok untuk pemula atau tim yang ingin memulai *automation* dengan cepat tanpa harus menulis banyak kode. Katalon menyediakan *recorder* untuk merekam interaksi pengguna dan menghasilkan *script* secara otomatis.

Kelemahan Katalon muncul ketika Anda membutuhkan fleksibilitas tinggi atau skenario *testing* yang sangat kompleks: * **Keterbatasan Kustomisasi:** Meskipun memungkinkan penulisan *script* Groovy, Katalon bisa menjadi kurang fleksibel dibandingkan *framework* berbasis kode murni seperti Playwright saat menghadapi logika yang rumit atau integrasi khusus. * **Performa:** Eksekusi *test* di Katalon bisa lebih lambat dibandingkan *framework* modern yang dioptimalkan untuk kecepatan. * **Vendor Lock-in:** Ketergantungan pada ekosistem Katalon bisa menjadi masalah jika Anda ingin beralih ke *tool* lain di masa mendatang.

Di Fairatmos, kami juga pernah menggunakan Katalon, tetapi seiring dengan kebutuhan *testing* yang semakin kompleks dan cepat, kami memutuskan untuk beralih ke Playwright. Migrasi ini memungkinkan kami untuk memiliki kontrol penuh atas *script*, mengoptimalkan performa, dan memanfaatkan kekuatan TypeScript untuk *type safety*.

Keunggulan Playwright

Jadi, mengapa Playwright menjadi pilihan kami? Berikut adalah beberapa karakteristik utama yang membuatnya menonjol:

1. **Cross-Browser Penuh:** Playwright mendukung Chromium (Chrome, Edge), Firefox, dan WebKit (Safari) secara *native* hanya dengan satu API. Anda dapat menguji aplikasi Anda di ketiga mesin *browser* utama, memastikan kompatibilitas yang luas.

2. **Auto-Waiting:** Playwright secara cerdas menunggu elemen siap sebelum berinteraksi dengannya. Ini secara signifikan mengurangi *flakiness test* dan menghilangkan kebutuhan akan *explicit waits* manual, membuat *script* Anda lebih stabil dan mudah ditulis.

3. **API Testing First-Class:** Playwright memiliki API bawaan untuk melakukan *network requests* (HTTP/HTTPS). Ini berarti Anda dapat melakukan *API testing* di samping *UI testing* dalam *framework* yang sama, bahkan menggunakannya untuk menyiapkan *test data* tanpa harus membuka *browser*.

4. **Trace Viewer, Screenshots, dan Video Otomatis:** Untuk setiap eksekusi *test* yang gagal, Playwright dapat secara otomatis merekam *trace* (langkah-langkah interaksi), mengambil *screenshot*, dan merekam video. Fitur ini sangat membantu dalam *debugging* dan memahami mengapa suatu *test* gagal.

5. **Parallelization:** Playwright dirancang untuk menjalankan *test* secara paralel secara *default*, baik di *browser* yang berbeda maupun di *worker* yang berbeda dalam *browser* yang sama. Ini mempercepat waktu eksekusi *test suite* secara drastis.

6. **TypeScript Support:** Playwright memiliki dukungan TypeScript yang sangat baik, memungkinkan Anda menulis *test* dengan *type safety*, *autocompletion*, dan *refactoring* yang lebih mudah.

Dengan kombinasi fitur-fitur ini, Playwright memberikan pengalaman *automation testing* yang modern, efisien, dan andal.

Struktur Proyek Automation Playwright

Proyek *automation testing* yang baik harus memiliki struktur folder yang terorganisir dengan rapi. Ini meningkatkan keterbacaan, kemudahan pemeliharaan, dan skalabilitas. Di Fairatmos, proyek Playwright kami diatur sebagai berikut:

```
automation-playwright/
■ playwright.config.ts      # Konfigurasi runner & browser Playwright
■ tsconfig.json             # Konfigurasi TypeScript
■ package.json              # Metadata proyek & daftar dependensi
■ .env.example              # Template variabel lingkungan (environment variables)
■ api/                     # Modul untuk interaksi API
■ ■ auth.api.ts             # Contoh: wrapper untuk POST /api/v1/user/sign-in
```

```

■■ helpers/                                # Fungsi-fungsi utilitas umum
■ ■■ customize.ts                          # Contoh: waitForText, fill by testId
■■ pages/                                  # Implementasi Page Object Model (POM)
■ ■■ login.page.ts                         # Objek halaman untuk alur login
■ ■■ register.page.ts                      # Objek halaman untuk registrasi user baru
■ ■■ forgot-password.page.ts              # Objek halaman untuk alur forgot password
■ ■■ profile.page.ts                       # Objek halaman untuk profil user, logout
■ ■■ dashboard.page.ts                    # Objek halaman untuk dashboard
■ ■■ forestry.page.ts                     # Objek halaman untuk pertanyaan Forestry
■ ■■ shapefile.page.ts                    # Objek halaman untuk upload shapefile
■ ■■ review.page.ts                       # Objek halaman untuk halaman review proyek
■ ■■ result.page.ts                       # Objek halaman untuk halaman hasil proyek
■■ data/                                   # Data uji (test data) untuk skenario data-driven
■ ■■ shapefile.data.ts                    # Contoh: data untuk upload shapefile
■■ test-data/                              # File data biner yang dibutuhkan test
■ ■■ Shapefile/atmostalk/                 # Contoh: file .shp/.shx/.dbf/.prj
■■ qa-docs/                                # Dokumentasi QA terkait (test design, UAT, bug list)
■■ scripts/                                # Script utilitas (misal: integrasi Slack)
■ ■■ slack-connector.ts
■ ■■ bug-parser.ts
■ ■■ report-bugs.ts
■■ tests/                                  # File test spec (skenario pengujian)
■ ■■ smoke.spec.ts                        # Test smoke untuk sanity check
■■ login/                                  # Kumpulan test untuk alur login
■ ■■ login-valid.spec.ts
■ ■■ login-incorrect-email.spec.ts
■ ■■ login-incorrect-password.spec.ts
■ ■■ login-api.spec.ts
■■ forgot-password/                        # Kumpulan test untuk alur forgot password
■ ■■ forgot-password.spec.ts
■ ■■ forgot-password-email-not-exist.spec.ts
■■ homepage/                              # Kumpulan test untuk halaman utama
■ ■■ register.spec.ts
■ ■■ logout.spec.ts
■ ■■ new-dashboard.spec.ts
■■ forestry/                              # Kumpulan test untuk modul forestry
■ ■■ upload-shapefile.spec.ts

```

Mari kita jelaskan setiap bagian penting dari struktur ini:

- **playwright.config.ts**: Ini adalah jantung konfigurasi Playwright Anda. Di sini Anda menentukan *browser* apa yang akan digunakan, berapa banyak *worker* yang akan dijalankan secara paralel, *timeout*, *base URL*, dan opsi *reporting* seperti perekaman video atau *screenshot*.
- **tsconfig.json**: File konfigurasi untuk TypeScript. Mengatur bagaimana kode TypeScript Anda akan dikompilasi menjadi JavaScript.

- **package.json**: File standar Node.js yang berisi metadata proyek, seperti nama, versi, *script* yang dapat dijalankan (`npm test`, `npm run report`), dan daftar semua dependensi (paket *library* pihak ketiga) yang dibutuhkan proyek.
- **.env.example**: Contoh file untuk variabel lingkungan. Anda akan menyalinnya menjadi `.env` dan mengisi kredensial sensitif seperti `TEST_EMAIL` dan `TEST_PASSWORD` yang tidak boleh di-commit ke *repository* kode.
- **api/**: Folder ini berisi fungsi-fungsi *wrapper* untuk interaksi dengan *API backend* aplikasi Anda. Mengapa penting? Anda dapat menggunakannya untuk menyiapkan data uji (misalnya, membuat pengguna baru via API) atau bahkan menguji *endpoint* API secara langsung tanpa perlu membuka *browser*, yang jauh lebih cepat.
- **helpers/**: Berisi fungsi-fungsi utilitas atau *custom commands* yang dapat digunakan kembali di berbagai *test case* atau *Page Object*. Contohnya bisa berupa fungsi untuk menunggu teks tertentu muncul, mengisi *field* berdasarkan `testId`, atau *assertion* yang sering digunakan.
- **pages/**: Ini adalah implementasi dari pola desain *Page Object Model* (POM). Setiap file di sini mewakili sebuah halaman atau komponen utama dalam aplikasi Anda. Kita akan membahas ini lebih detail di bagian selanjutnya.
- **data/**: Folder ini dikhususkan untuk menyimpan data uji. Daripada menanamkan data langsung ke dalam *test script*, memisahkannya ke dalam file terpisah memungkinkan Anda mengelola data dengan lebih baik, terutama untuk skenario *data-driven testing*.
- **test-data/**: Untuk menyimpan file data biner yang dibutuhkan oleh *test*, seperti *shapefile* (`.shp`, `.shx`, `.dbf`, `.prj`) yang digunakan dalam *test Forestry*. Memisahkannya dari kode memastikan *repository* tetap rapi.
- **qa-docs/**: Meskipun bukan bagian dari *automation script* itu sendiri, folder ini menunjukkan integrasi dengan dokumentasi QA. Ini bisa berisi *test design*, *execution reports*, *bug list*, dan *UAT reports*.
- **scripts/**: Folder untuk *script* utilitas yang tidak langsung terkait dengan *testing*, seperti *script* untuk integrasi Slack atau *parsing bug report*.
- **tests/**: Ini adalah tempat semua *test spec* (file yang berisi skenario pengujian) Anda berada. Biasanya diatur dalam sub-folder berdasarkan fitur atau modul aplikasi, seperti `login/`, `forgot-password/`, atau `forestry/`.

Struktur ini memastikan bahwa kode *automation* Anda modular, mudah dinavigasi, dan siap untuk dikembangkan seiring bertambahnya kompleksitas aplikasi Anda.

Page Object Model (POM): Membangun Abstraksi Halaman

Page Object Model (POM) adalah pola desain yang sangat populer dalam *automation testing* untuk aplikasi web. Tujuannya adalah untuk membuat abstraksi dari halaman-halaman antarmuka pengguna (UI) aplikasi Anda menjadi objek-objek kode.

Mengapa Menggunakan POM?

1. **Keterbacaan (Readability):** *Test case* menjadi lebih mudah dibaca karena tidak lagi berisi detail implementasi UI. Sebagai gantinya, *test case* akan memanggil metode pada *Page Object* yang merepresentasikan tindakan pengguna.
2. **Pemeliharaan (Maintainability):** Jika UI aplikasi berubah (misalnya, *locator* elemen berubah), Anda hanya perlu memperbarui *Page Object* yang relevan, bukan setiap *test case* yang menggunakan elemen tersebut. Ini mengurangi upaya pemeliharaan secara drastis.
3. **Penggunaan Kembali (Reusability):** Logika interaksi dengan halaman dapat digunakan kembali di berbagai *test case*. Misalnya, *flow* login dapat digunakan sebagai prasyarat untuk banyak *test case* lainnya.

Cara Kerja POM di Playwright

Di Playwright, Anda akan membuat sebuah kelas (class) untuk setiap halaman atau komponen utama aplikasi Anda. Kelas ini akan berisi:

- **Locators:** Properti yang mendefinisikan cara menemukan elemen UI di halaman (misalnya, tombol, *input field*, *dropdown*). Playwright memiliki *locator* yang sangat kuat, seperti `getByRole`, `getByText`, `getByLabel`, `getByPlaceholder`, `getByTestId`, dan *locator* umum.
- **Methods:** Fungsi-fungsi yang merepresentasikan tindakan pengguna atau interaksi dengan elemen di halaman tersebut. Misalnya, `fillEmail`, `clickLoginButton`, `verifyErrorMessage`.

Mari kita lihat contoh implementasi POM untuk alur login di Fairatmos.

Contoh Nyata: Login Flow

Anggap kita memiliki halaman login dengan *field* email, *password*, dan tombol login.

1. Membuat LoginPage Page Object (pages/login.page.ts)

```
// automation-playwright/pages/login.page.ts

import { expect, Locator, Page } from '@playwright/test';
import { CustomHelper } from '../helpers/customize'; // Asumsi ada helper

export class LoginPage {
  readonly page: Page;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;
  readonly errorMessageToast: Locator;
  readonly helper: CustomHelper;

  constructor(page: Page) {
    this.page = page;
    this.helper = new CustomHelper(page); // Inisialisasi helper
    this.emailInput = page.getByPlaceholder('Masukkan email Anda');
    this.passwordInput = page.getByPlaceholder('Masukkan password Anda');
    // Menggunakan getByRole untuk tombol Login
    this.loginButton = page.getByRole('button', { name: 'Login' });
    // Menggunakan locator yang lebih spesifik jika ada toast message
    this.errorMessageToast = page.locator('.Toastify__toast--error');
  }

  async navigateToLogin() {
    await this.page.goto('/login'); // Asumsi base URL sudah dikonfigurasi
  }

  async login(email: string, password_input: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password_input);
    await this.loginButton.click();
  }

  async verifyLoginSuccess() {
    // Asumsi setelah login berhasil akan diarahkan ke dashboard
    await expect(this.page).toHaveURL(/dashboard/);
    // Atau bisa juga menunggu elemen tertentu di dashboard muncul
    await expect(this.page.getByText('Selamat Datang di Fairatmos')).toBeVisible();
  }

  async verifyLoginFailedMessage(message: string) {
    await expect(this.errorMessageToast).toBeVisible();
    await expect(this.errorMessageToast).toContainText(message);
  }
}
```

```

    }
    // Contoh metode lain menggunakan helper (jika ada)
    async fillEmailByTestId(email: string) {
        await this.helper.fillByTestId('email-field', email);
    }
}

```

Penjelasan:

- `constructor(page: Page)`: Mengambil objek `page` dari Playwright, yang merupakan representasi halaman *browser*. Semua interaksi akan melalui objek `page` ini.
- `readonly emailInput: Locator`; Mendefinisikan *locator* untuk *field* email. Kita menggunakan `page.getByPlaceholder()` yang merupakan salah satu *smart locator* Playwright.
- `async navigateToLogin()`: Metode untuk menavigasi ke halaman login.
- `async login(email: string, password_input: string)`: Metode utama untuk melakukan alur login. Ini mengabstraksi detail pengisian *field* dan klik tombol.
- `async verifyLoginSuccess()`: Metode untuk melakukan *assertion* setelah login berhasil.
- `async verifyLoginFailedMessage(message: string)`: Metode untuk memverifikasi pesan *error* jika login gagal.

2. Menggunakan LoginPage di Test Spec (tests/login/login-valid.spec.ts)

Sekarang, mari kita lihat bagaimana *Page Object* ini digunakan dalam *test spec* untuk skenario login yang valid.

```

// automation-playwright/tests/login/login-valid.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../../pages/login.page'; // Import Page Object
import * as dotenv from 'dotenv';

dotenv.config(); // Memuat variabel lingkungan dari file .env

test.describe('Login Flow', () => {
    let loginPage: LoginPage; // Deklarasikan variabel LoginPage
}

```

```

test.beforeEach(async ({ page }) => {
  loginPage = new LoginPage(page); // Inisialisasi LoginPage sebelum setiap test
  await loginPage.navigateToLogin(); // Navigasi ke halaman login
});

test('should allow a user to log in with valid credentials', async () => {
  // Mengambil kredensial dari environment variables
  const email = process.env.TEST_EMAIL || 'default@example.com';
  const password = process.env.TEST_PASSWORD || 'password123';

  // Panggil metode login dari Page Object
  await loginPage.login(email, password);

  // Panggil metode verifikasi dari Page Object
  await loginPage.verifyLoginSuccess();

  console.log(`Login berhasil untuk user: ${email}`);
});

test('should show error message for incorrect password', async () => {
  const email = process.env.TEST_EMAIL || 'default@example.com';
  const incorrectPassword = 'wrongpassword';

  await loginPage.login(email, incorrectPassword);
  await loginPage.verifyLoginFailedMessage('Email atau password salah');
});

test('should show error message for non-existent email', async () => {
  const nonExistentEmail = 'nonexistent@example.com';
  const anyPassword = 'anypassword';

  await loginPage.login(nonExistentEmail, anyPassword);
  await loginPage.verifyLoginFailedMessage('Email atau password salah');
});
});

```

Penjelasan:

- `import { LoginPage } from '../pages/login.page';`: Mengimpor *Page Object* yang telah kita buat.
- `test.beforeEach`: Sebelum setiap *test* dijalankan, kita membuat instance `LoginPage` baru dan menavigasi ke halaman login. Ini memastikan setiap *test* dimulai dari kondisi yang bersih.
- `await loginPage.login(email, password);`: *Test case* menjadi sangat ringkas dan mudah dibaca. Ia hanya memanggil metode `login` pada objek `loginPage`, tanpa perlu tahu *locator* atau langkah-langkah detail di baliknya.

- `await loginPage.verifyLoginSuccess();`; Demikian pula, verifikasi juga dilakukan melalui metode pada *Page Object*.

Dengan POM, jika *locator* untuk *field* email berubah dari `getByPlaceholder('Masukkan email Anda')` menjadi `getByLabel('Email Address')`, Anda hanya perlu mengubahnya di satu tempat: yaitu di file `login.page.ts`. Semua *test case* yang menggunakan `loginPage.emailInput` akan otomatis terbaru tanpa perubahan pada *test spec* itu sendiri. Ini adalah kekuatan utama dari *Page Object Model*.

Data-Driven Testing: Menguji dengan Berbagai Data

Dalam *automation testing*, seringkali Anda perlu menjalankan *test case* yang sama berulang kali dengan *input data* yang berbeda. Misalnya, menguji formulir dengan berbagai kombinasi data valid/invalid, atau mengunggah berbagai jenis file. Ini disebut *Data-Driven Testing*.

Data-driven testing membantu Anda: * **Meningkatkan Cakupan (Coverage)**: Menguji lebih banyak skenario dengan data yang bervariasi. * **Mengurangi Duplikasi Kode**: Logika *test* tetap sama, hanya datanya yang berubah. * **Memudahkan Pemeliharaan Data**: Data uji disimpan secara terpisah dari logika *test*.

Di Playwright dengan TypeScript, kita dapat mengimplementasikan *data-driven testing* dengan menyimpan data dalam file TypeScript terpisah dan mengulanginya dalam *test spec*.

Contoh Nyata: Upload Shapefile

Di Fairatmos, salah satu fitur penting adalah kemampuan untuk mengunggah *shapefile* untuk proyek kehutanan. Proses ini melibatkan pengunggahan empat jenis file (`.shp`, `.shx`, `.dbf`, `.prj`) secara bersamaan. Kita perlu menguji ini dengan berbagai set *shapefile* yang berbeda (misalnya, dari lokasi yang berbeda, atau yang mungkin memiliki karakteristik khusus).

1. Menyimpan Data Uji (`data/shapefile.data.ts`)

Kita akan membuat sebuah *array* objek yang setiap objeknya merepresentasikan satu set data *shapefile* yang akan diunggah.

```
// automation-playwright/data/shapefile.data.ts

export interface ShapefileData {
  id: string; // ID unik untuk setiap set data
  name: string; // Nama untuk identifikasi dalam laporan
  folder: string; // Nama folder tempat file shapefile berada
  projectName: string; // Nama proyek yang akan diinput
  expectedCarbonCredit?: string; // Opsional: nilai kredit karbon yang diharapkan
  expectedScore?: string; // Opsional: skor yang diharapkan
}

export const shapefileDatasets: ShapefileData[] = [
  {
    id: 'cilacap_srn',
    name: 'Cilacap SRN',
    folder: 'cilacap',
    projectName: 'Proyek Cilacap SRN Otomatis',
    expectedCarbonCredit: '1000', // Contoh nilai yang diharapkan
    expectedScore: '85',
  },
  {
    id: 'bug1_verra',
    name: 'Bug Fix Test Verra',
    folder: 'bug1',
    projectName: 'Proyek Bug Fix 1 Verra Otomatis',
    expectedCarbonCredit: '500',
    expectedScore: '70',
  },
  {
    id: 'bug2_srn',
    name: 'Bug Fix Test SRN',
    folder: 'bug2',
    projectName: 'Proyek Bug Fix 2 SRN Otomatis',
    expectedCarbonCredit: '1200',
    expectedScore: '90',
  },
  // Anda bisa menambahkan lebih banyak set data di sini
];
```

Penjelasan:

- `export interface ShapefileData`: Mendefinisikan *interface* TypeScript untuk struktur data setiap *dataset*. Ini memberikan *type safety* dan *autocompletion*.
- `export const shapefileDatasets: ShapefileData[]`: Sebuah *array* dari objek `ShapefileData`. Setiap objek adalah satu set data yang akan digunakan untuk satu eksekusi *test*.

- **folder**: Ini akan digunakan untuk menemukan file *shapefile* yang sebenarnya di `test-data/Shapefile/atmostalk/`. Misalnya, `test-data/Shapefile/atmostalk/cilacap/` akan berisi `.shp`, `.shx`, `.dbf`, `.prj` untuk *dataset* 'cilacap_srn'.

2. Menggunakan Data di Test Spec (`tests/forestry/upload-shapefile.spec.ts`)

Sekarang, kita akan membuat *test spec* yang akan mengulang melalui `shapefileDatasets` dan menjalankan logika *upload shapefile* untuk setiap *dataset*.

```
// automation-playwright/tests/forestry/upload-shapefile.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../../pages/login.page';
import { DashboardPage } from '../../pages/dashboard.page';
import { ForestryPage } from '../../pages/forestry.page';
import { ShapefilePage } from '../../pages/shapefile.page';
import { ReviewPage } from '../../pages/review.page';
import { ResultPage } from '../../pages/result.page';
import { shapefileDatasets, ShapefileData } from '../../data/shapefile.data'; // Import data
import * as path from 'path';
import * as dotenv from 'dotenv';

dotenv.config();

// Definisikan path dasar ke folder shapefile
const SHAPEFILE_BASE_DIR = process.env.SHAPEFILE_DIR || path.resolve(__dirname, '../../test-data/Shapefile');

test.describe('Forestry Project - Upload Shapefile (Data-Driven)', () => {
  let loginPage: LoginPage;
  let dashboardPage: DashboardPage;
  let forestryPage: ForestryPage;
  let shapefilePage: ShapefilePage;
  let reviewPage: ReviewPage;
  let resultPage: ResultPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    dashboardPage = new DashboardPage(page);
    forestryPage = new ForestryPage(page);
    shapefilePage = new ShapefilePage(page);
    reviewPage = new ReviewPage(page);
    resultPage = new ResultPage(page);

    // Prasyarat: Login sebagai user test
    await loginPage.navigateToLogin();
    await loginPage.login(process.env.TEST_EMAIL!, process.env.TEST_PASSWORD!);
```

```

    await loginPage.verifyLoginSuccess();
    await dashboardPage.closeWelcomeModalIfVisible(); // Tutup modal selamat datang jika ada
  });
}

// Iterasi melalui setiap dataset shapefile
for (const dataset of shapefileDatasets) {
  test(`should successfully upload shapefile for ${dataset.name}`, async () => {
    await test.step('Mulai proyek Forestry baru', async () => {
      await dashboardPage.clickNewProjectButton();
      await dashboardPage.selectForestryProject();
    });

    await test.step('Jawab pertanyaan Forestry', async () => {
      // Asumsi ada metode untuk menjawab pertanyaan di ForestryPage
      await forestryPage.answerAllQuestions();
      await forestryPage.clickNextButton();
    });

    await test.step(`Upload shapefile dari folder ${dataset.folder}`, async () => {
      const folderPath = path.join(SHAPEFILE_BASE_DIR, dataset.folder);
      const shpPath = path.join(folderPath, `${dataset.folder}.shp`);
      const shxPath = path.join(folderPath, `${dataset.folder}.shx`);
      const dbfPath = path.join(folderPath, `${dataset.folder}.dbf`);
      const prjPath = path.join(folderPath, `${dataset.folder}.prj`);

      await shapefilePage.uploadShapefiles([shpPath, shxPath, dbfPath, prjPath]);
      await shapefilePage.waitForUploadProgressToComplete();
      await shapefilePage.clickNextButton();
    });

    await test.step(`Review dan beri nama proyek: ${dataset.projectName}`, async () => {
      await reviewPage.inputProjectName(dataset.projectName);
      await reviewPage.clickContinueButton();
    });

    await test.step('Verifikasi halaman hasil', async () => {
      await resultPage.waitForResultPageLoad();
      await resultPage.verifyProjectName(dataset.projectName);
      // Lakukan assertion berdasarkan data yang diharapkan dari dataset
      if (dataset.expectedCarbonCredit) {
        await resultPage.verifyCarbonCredit(dataset.expectedCarbonCredit);
      }
      if (dataset.expectedScore) {
        await resultPage.verifyProjectScore(dataset.expectedScore);
      }
      // Tambahkan assertion lain sesuai kebutuhan
    });
  });
}

```



```
}  
});
```

Penjelasan:

- `import { shapefileDatasets, ShapefileData } from`
`'../../data/shapefile.data';` Mengimpor data uji yang telah kita definisikan.
- `for (const dataset of shapefileDatasets):` Ini adalah inti dari *data-driven testing*. Kita menggunakan perulangan `for...of` untuk mengiterasi setiap objek `dataset` dalam `array shapefileDatasets`.
- `test(should successfully upload shapefile for ${dataset.name}, async () => { ... });`
Untuk setiap `dataset`, kita mendefinisikan sebuah *test case* baru. Nama *test case* dibuat dinamis menggunakan `dataset.name` agar mudah diidentifikasi dalam laporan.
- `const folderPath = path.join(SHAPEFILE_BASE_DIR, dataset.folder);` Kita menggunakan modul `path` dari Node.js untuk menggabungkan jalur file. `SHAPEFILE_BASE_DIR` diambil dari variabel lingkungan atau *path* default.
- `await shapefilePage.uploadShapefiles([...]);` Metode ini (yang akan ada di `pages/shapefile.page.ts`) akan bertanggung jawab untuk memilih dan mengunggah file-file tersebut.

Dengan pendekatan ini, Anda dapat dengan mudah menambahkan *dataset* baru ke `shapefile.data.ts` tanpa perlu mengubah logika *test* di `upload-shapefile.spec.ts`. Setiap penambahan *dataset* akan otomatis membuat *test case* baru yang akan dieksekusi, memastikan cakupan yang luas dengan kode yang efisien.

Menjalankan Test Otomatis & Melihat Laporan

Setelah Anda menulis *test case*, langkah selanjutnya adalah menjalankannya dan menganalisis hasilnya. Playwright menyediakan berbagai perintah untuk menjalankan *test* Anda, dari eksekusi *headless* cepat hingga mode UI interaktif untuk *debugging*.

Setup Pertama Kali

Sebelum menjalankan *test*, pastikan Anda sudah melakukan *setup* proyek:

```
cd automation-playwright
npm install # Menginstal semua dependensi
npx playwright install --with-deps chromium # Menginstal browser Chromium Playwright
copy .env.example .env # Salin template env
# Lalu isi file .env dengan kredensial TEST_EMAIL, TEST_PASSWORD, dll.
```

Perintah Penting untuk Menjalankan Test

| Perintah | Fungsi

| `npm test` | Menjalankan semua tes secara *headless* (tanpa membuka GUI *browser*). Ini adalah mode default untuk eksekusi cepat di CI/CD. | | `npm run test:headed` | Sama seperti `npm test`, tetapi *browser* akan terlihat saat *test* berjalan. Berguna untuk melihat interaksi secara visual saat *debugging*. | | `npm run test:ui` | Membuka Playwright UI. Ini adalah mode interaktif yang sangat direkomendasikan saat Anda mengembangkan atau *debugging test*. Anda bisa melihat *trace*, *screenshot*, menjalankan *test* satu per satu, dan bahkan melakukan *step-by-step debugging*. | | `npm run test:debug` | Memulai *debugger* Playwright, memungkinkan Anda untuk mengatur *breakpoint* dan melangkah melalui kode *test* Anda. Berguna untuk menganalisis perilaku *test* secara mendalam. | | `npm run codegen` | Perintah yang sangat ampuh! Ini membuka *browser* dan merekam interaksi Anda, lalu secara otomatis menghasilkan kode *test* Playwright. Sangat berguna untuk membuat *boilerplate* awal untuk *test* baru atau untuk mengidentifikasi *locator*. | | `npm run report` | Membuka laporan HTML interaktif dari hasil eksekusi *test* terakhir. Ini adalah *tool* penting untuk menganalisis kegagalan dan mendapatkan *insight* tentang performa *test*. |

Memfilter Test Tertentu

Anda tidak selalu ingin menjalankan seluruh *test suite*. Playwright memungkinkan Anda untuk memfilter *test* berdasarkan *path* file, *nama test*, atau *tag*.

Menjalankan semua *test* dalam folder tertentu: `bash npx playwright test tests/login`
Perintah ini akan menjalankan semua file `.spec.ts` yang ada di dalam folder `tests/login/`.

Menjalankan *test* dengan nama tertentu (menggunakan *substring* atau *regex*): `bash npx playwright test -g "incorrect_email"` Ini akan menjalankan semua *test case* yang namanya mengandung string `"incorrect_email"`. Anda bisa menggunakan `-g` (atau `--grep`) untuk mencocokkan sebagian nama *test* atau deskripsi *suite*.

Menjalankan *test API* saja (tanpa membuka *browser*): `bash npx playwright test tests/login/login-api.spec.ts` Jika Anda memiliki *test* yang hanya berinteraksi dengan

API (`request.post()`, `request.get()`, dll.), Anda bisa menjalankannya secara terpisah. Ini jauh lebih cepat karena tidak perlu meluncurkan *browser* UI.

Menjalankan *smoke test* cepat sebelum *deploy*: `bash npx playwright test tests/smoke.spec.ts tests/login/login-valid.spec.ts tests/login/login-api.spec.ts` Anda bisa menentukan beberapa file *spec* untuk dijalankan bersamaan. Ini ideal untuk *smoke test* yang cepat dan penting.

Parallelization (Eksekusi Paralel)

Salah satu keunggulan Playwright adalah kemampuannya untuk menjalankan *test* secara paralel secara *default*. Ini berarti jika Anda memiliki banyak *test case*, Playwright akan membaginya ke beberapa *worker* (proses) dan menjalankannya secara bersamaan, mempercepat total waktu eksekusi. Anda dapat mengkonfigurasi jumlah *worker* di `playwright.config.ts`.

Laporan HTML dan Fitur Debugging

Setelah *test* selesai dijalankan (terutama jika ada kegagalan), Anda dapat membuka laporan HTML interaktif dengan `npm run report`. Laporan ini adalah *tool* yang sangat berharga:

- **Ringkasan Hasil:** Menunjukkan berapa banyak *test* yang lulus, gagal, atau di-*skip*.
- **Detail Setiap Test:** Untuk setiap *test*, Anda bisa melihat langkah-langkah yang dieksekusi, *assertion* yang dilakukan, dan *error message* jika gagal.
- **Trace Viewer:** Ini adalah fitur unggulan Playwright. Untuk setiap *test* yang terekam *trace*, Anda bisa melihat *timeline* eksekusi, *screenshot* di setiap langkah, status jaringan, dan bahkan video singkat dari interaksi *browser*. Ini seperti memiliki kamera pengawas yang merekam setiap detail saat *test* dijalankan.
- **Screenshots dan Video:** Playwright dapat dikonfigurasi untuk mengambil *screenshot* saat *test* gagal atau merekam video penuh dari setiap eksekusi *test*. Ini sangat membantu dalam mereproduksi *bug* atau memahami konteks kegagalan.

Dengan fitur-fitur ini, menganalisis kegagalan *test* menjadi jauh lebih mudah dan cepat, memungkinkan Anda mengidentifikasi akar masalah dengan efisien.

Rangkuman

Selamat! Anda telah menyelesaikan bab penting tentang *automation testing* dengan Playwright. Kita telah belajar bahwa *automation testing* bukan lagi pilihan, melainkan keharusan untuk memastikan kualitas perangkat lunak di tengah siklus pengembangan yang cepat. Playwright menonjol sebagai *tool* modern yang kuat, mengatasi banyak keterbatasan *framework* lama dengan fitur-fitur seperti *auto-waiting*, dukungan *cross-browser* yang luas, *API testing first-class*, dan kemampuan *debugging* yang luar biasa.

Anda kini memahami pentingnya struktur proyek yang rapi, yang memastikan kode *automation* Anda mudah dikelola dan diskalakan. Kita juga telah mendalami *Page Object Model* (POM) sebagai pola desain fundamental untuk membuat *test case* yang mudah dibaca dan dipelihara, serta bagaimana menerapkan *data-driven testing* untuk menguji berbagai skenario dengan data yang bervariasi secara efisien. Terakhir, Anda telah mempelajari berbagai cara untuk menjalankan *test* dan memanfaatkan laporan HTML serta fitur *trace viewer* Playwright untuk *debugging* yang efektif.

Menguasai *automation testing* dengan Playwright akan secara signifikan meningkatkan efisiensi dan cakupan *testing* Anda, membebaskan Anda untuk fokus pada aspek kualitas yang lebih kompleks. Ini adalah langkah maju yang besar dalam perjalanan Anda menjadi Quality Assurance Engineer yang kompeten dan berharga.

Checklist Actionable

Berikut adalah langkah-langkah yang dapat Anda lakukan untuk mulai menerapkan *automation testing* dengan Playwright:

- [] **Inisialisasi Proyek Playwright:** Buat proyek baru atau navigasi ke folder `automation-playwright/` jika Anda menggunakan *template* yang ada. Jalankan `npm install` dan `npx playwright install --with-deps chromium`.
- [] **Konfigurasi Lingkungan:** Salin `.env.example` menjadi `.env` dan isi variabel lingkungan yang dibutuhkan (misalnya, `TEST_EMAIL`, `TEST_PASSWORD`).
- [] **Buat Page Object Sederhana:** Pilih salah satu halaman di aplikasi Anda (misalnya, halaman login atau halaman beranda). Buat file `.page.ts` baru di folder `pages/` dan definisikan *locator* serta metode dasar untuk berinteraksi dengan halaman tersebut.

- [] **Tulis Test Case Pertama Anda:** Buat file `.spec.ts` baru di folder `tests/` yang menggunakan *Page Object* yang baru Anda buat. Tulis *test case* sederhana untuk menguji fungsionalitas dasar halaman tersebut.
 - [] **Jalankan Test Anda:** Gunakan `npm run test:headed` untuk melihat *test* Anda berjalan di *browser*.
 - [] **Eksplorasi Playwright UI:** Gunakan `npm run test:ui` untuk membuka mode UI interaktif. Coba jalankan *test* satu per satu, gunakan fitur *step-by-step debugging*, dan lihat *trace viewer* untuk *test* yang gagal.
 - [] **Buat Laporan HTML:** Setelah menjalankan *test*, buka laporan HTML dengan `npm run report` dan analisis hasilnya, terutama untuk *test* yang gagal.
 - [] **Pertimbangkan Data-Driven Testing:** Identifikasi *test case* yang perlu diulang dengan data berbeda. Pindahkan data tersebut ke file di folder `data/` dan modifikasi *test spec* Anda untuk mengiterasi data tersebut.
 - [] **Integrasikan API Testing:** Jika ada kebutuhan untuk menyiapkan *test data* atau menguji *endpoint* API, mulai buat *wrapper* di folder `api/` dan gunakan di *test* Anda.
 - [] **Terus Berlatih:** Semakin banyak Anda menulis dan menjalankan *test* otomatis, semakin mahir Anda akan menjadi. Jangan ragu untuk bereksperimen dengan berbagai fitur Playwright!
-

Chapter 08: Tools & Workflow — Notion, Slack, Drive, Handoff

Selamat datang di chapter yang akan membuka wawasan Anda tentang bagaimana seorang QA Engineer tidak hanya bekerja dengan *code* dan *test case*, tetapi juga dengan ekosistem alat dan alur kerja yang terintegrasi. Di dunia QA Thomas, kualitas bukan hanya tentang menemukan *bug*, tetapi juga tentang bagaimana informasi disebarakan, bagaimana tim berkolaborasi, dan bagaimana setiap langkah didokumentasikan dengan rapi.

Bayangkan departemen QA sebagai sebuah *quality gate* yang tidak hanya memeriksa, tetapi juga mengelola alur informasi yang kompleks. Untuk memastikan semua berjalan lancar, QA Thomas mengandalkan serangkaian *tools* yang saling terhubung—mulai dari platform manajemen proyek, alat komunikasi, hingga *connector* untuk berinteraksi langsung dengan *backend*. Dalam chapter ini, kita akan menyelami setiap *tool* ini, memahami peran spesifiknya, dan bagaimana mereka membentuk alur kerja sehari-hari seorang QA Engineer yang efektif. Mari kita mulai perjalanan Anda untuk menguasai ekosistem kerja QA Thomas!

1. Notion: Pusat Komando Dokumentasi dan Pelaporan Bug

Notion adalah jantung dari ekosistem kerja QA Thomas. Sebagai platform *workspace* yang fleksibel, Notion menjadi tempat terpusat untuk segala hal, mulai dari dokumentasi perencanaan *testing* hingga pelacakan *bug*. Ini adalah "pusat komando" Anda, tempat di mana semua informasi penting dikelola secara terstruktur dan transparan.

Database Task (Test Documentation)

Database Task di Notion adalah tempat Anda menyimpan semua dokumentasi terkait *testing*. Ini termasuk *Test Plan*, *Test Strategy*, dan tentu saja, *Test Case* yang Anda buat. Tujuan utamanya adalah memastikan setiap aktivitas *testing* terdokumentasi dengan baik dan dapat ditelusuri.

Konvensi dan Properti Penting:

- **Linked Task (User Story):** Ini adalah properti krusial yang memastikan setiap *Test Case* atau dokumentasi *testing* memiliki keterkaitan langsung dengan *User Story* atau *task* pengembangan yang sedang diuji. Ini adalah implementasi dari **Aturan 1: Lacak setiap tes kembali ke *user story***. Jika sebuah *test case* tidak bisa dihubungkan ke *user story*, berarti Anda mungkin sedang menguji hal yang salah.
- **Status:** Menunjukkan progres *Test Case* atau *Test Plan* (e.g., Draft, In Progress, Ready for Review, Completed).
- **Owner:** Siapa yang bertanggung jawab atas *Test Case* atau *Test Plan* tersebut.
- **Type:** Kategori *testing* (e.g., Functional, Regression, Smoke, Exploratory).

- **Priority:** Seberapa penting *Test Case* tersebut dieksekusi.
- **Last Run Date:** Kapan terakhir *Test Case* ini dieksekusi.

Contoh Entri Test Case di Notion:

Bayangkan Anda baru saja menganalisis sebuah *User Story* untuk fitur "Login Pengguna" dan membuat *Test Case* untuk itu. Berikut adalah bagaimana entri *Test Case* Anda mungkin terlihat di Notion:

Properti	Nilai	Keterangan
Nama	TC-001: Login dengan Kredensial Valid	Judul <i>Test Case</i> yang deskriptif.
Linked Task	US-005: Sebagai Pengguna, Saya Ingin Bisa Login ke Aplikasi	Wajib! Menghubungkan ke <i>User Story</i> yang relevan.
Status	Ready for Execution	Siap untuk dijalankan.
Owner	QA Thomas	Penanggung jawab <i>Test Case</i> .
Type	Functional	Kategori <i>testing</i> .
Priority	High	Penting untuk diuji.
Preconditions	Pengguna terdaftar di sistem.	Kondisi yang harus dipenuhi sebelum <i>test</i> dijalankan.

Steps	1. Buka halaman login. 2. Masukkan email valid. 3. Masukkan password valid. 4. Klik tombol "Login".	Langkah-langkah yang jelas dan berurutan.
Expected Result	Pengguna berhasil login dan diarahkan ke halaman <i>dashboard</i> .	Hasil yang diharapkan dari eksekusi <i>Test Case</i> .
Actual Result	(Kosong, diisi setelah eksekusi)	Hasil aktual setelah <i>test</i> dijalankan.
Last Run Date	2023-10-26	Tanggal terakhir <i>Test Case</i> ini dieksekusi.
Notes	(Opsional)	Catatan tambahan, misalnya terkait data <i>test</i> yang digunakan.

Tips: Gunakan `notion_connector.NotionTicketing` (seperti yang disebutkan dalam *skill set* QA Thomas) untuk mengintegrasikan Notion dengan *tool* lain jika Anda memiliki skrip otomatisasi atau *tool* eksternal yang perlu membuat entri *Test Case* secara terprogram.

Database Bug (Bug Tracking)

Selain dokumentasi *testing*, Notion juga menjadi rumah bagi Database Bug. Ini adalah tempat terpusat untuk melaporkan, melacak, dan mengelola semua *bug* yang ditemukan. Database ini dirancang untuk memastikan setiap *bug* memiliki informasi yang lengkap, akurat, dan dapat ditindaklanjuti.

Konvensi dan Properti Penting (Aturan 2: Standar Pelaporan Bug):

- **Title:** Format [Area] Deskripsi singkat. (e.g., [Login Page] Tombol "Forgot Password" tidak berfungsi).
- **Steps to Reproduce:** Langkah-langkah bernomor yang jelas dan dapat ditindaklanjuti untuk mereproduksi *bug*.

- **Expected vs Actual:** Perbandingan antara hasil yang diharapkan dan hasil yang sebenarnya.
- **Environment:** Detail lingkungan di mana *bug* ditemukan (e.g., staging, production, dev + Chrome, Firefox, Android, iOS + *build version*).
- **Severity:** Tingkat keparahan *bug* (P0-P3). Ini sangat penting untuk prioritas perbaikan.
- **Screenshots or Video:** Bukti visual untuk *bug*, terutama yang terkait dengan UI/UX.
- **Reproducible:** Menunjukkan seberapa konsisten *bug* dapat direproduksi (Always, Intermittent, Once, Cannot reproduce).
- **Assignee:** Developer yang ditugaskan untuk memperbaiki *bug*.
- **Status:** Progres *bug* (e.g., Open, In Progress, Resolved, Closed, Reopened).
- **Linked Task:** (Opsional, tapi direkomendasikan) Menghubungkan ke *User Story* atau *Test Case* yang terkait.

Severity Rubric (P0-P3):

Penting untuk memahami dan menerapkan *severity rubric* ini secara konsisten:

Level	Definisi	Contoh
P0	Produksi <i>down</i> , kehilangan data, pelanggaran keamanan, memblokir semua pengguna.	<i>Login</i> rusak, pembayaran gagal, aplikasi tidak bisa dibuka.
P1	Fitur inti rusak untuk banyak pengguna, tanpa <i>workaround</i> .	Pencarian mengembalikan error 500, proses pendaftaran sering gagal.
P2	Fitur terganggu tetapi ada <i>workaround</i> .	Urutan <i>sort</i> salah, <i>typo</i> di UI, notifikasi tidak muncul tepat waktu.

P3	Kosmetik, ketidaknyamanan minor.	Piksel tidak sejajar, warna salah, masalah <i>timing</i> yang sangat jarang.
----	-------------------------------------	--

Contoh Entri Bug di Notion:

Properti	Nilai	Keterangan
Title	[Halaman Pembayaran] Pembayaran gagal saat menggunakan metode E-Wallet tertentu.	Judul yang jelas dan spesifik.
Steps to Reproduce	1. Buka halaman produk dan tambahkan item ke keranjang. 2. Lanjutkan ke halaman pembayaran. 3. Pilih metode pembayaran "E-Wallet X". 4. Masukkan nomor telepon yang terdaftar pada E-Wallet X. 5. Klik "Bayar Sekarang". 6. Tunggu hingga proses pembayaran selesai.	Langkah-langkah yang terperinci dan dapat direplikasi oleh siapa saja.
Expected Result	Transaksi pembayaran berhasil dan pengguna diarahkan ke halaman konfirmasi pembayaran. Saldo E-Wallet X berkurang.	

2. Slack: Jembatan Komunikasi Real-time

Slack adalah *tool* komunikasi utama bagi QA Thomas, memungkinkan interaksi yang cepat dan efisien dengan tim lain. Ini bukan hanya untuk *chat* santai, tetapi juga untuk eskalasi *bug* kritis, berbagi informasi penting, dan koordinasi harian.

Channel Mapping: Saluran yang Tepat untuk Pesan yang Tepat

Setiap *channel* di Slack memiliki tujuan spesifik untuk memastikan informasi sampai ke audiens yang relevan:

- **#engineering:** Ini adalah *channel* umum untuk tim *engineering* secara keseluruhan. Anda mungkin menggunakannya untuk pengumuman proyek besar, diskusi arsitektur, atau pertanyaan umum yang melibatkan banyak *developer*. QA Thomas bisa membagikan *update* progres *testing* yang berdampak luas di sini.

#qa: *Channel* khusus untuk tim QA. Ini adalah tempat Anda berdiskusi secara internal dengan sesama QA Engineer, berbagi tips, koordinasi *testing*, dan juga mengumumkan *bug* dengan *severity* P2.

- **Contoh Penggunaan:** "Tim, saya menemukan P2: [UI/UX] Warna tombol 'Submit' tidak sesuai spesifikasi di halaman profil. Detail sudah saya *file* di Notion."

#incidents: *Channel* paling penting untuk eskalasi *bug* dengan *severity* P0 (Produksi *down*) dan P1 (Fitur inti rusak). Setiap postingan di sini harus segera ditindaklanjuti.

- **Contoh Penggunaan:** "■ P0 INCIDENT ALERT ■ [Login] Pengguna tidak bisa login sama sekali di staging. Memblokir semua aktivitas. Detail Notion: [Link Bug Notion]. @TechLeadXYZ mohon segera dicek."

#uat-<feature-name>: *Channel* sementara yang dibuat khusus untuk setiap siklus *User Acceptance Testing* (UAT) sebuah fitur baru. Ini memudahkan komunikasi antara QA, PM, dan *stakeholder* yang berpartisipasi dalam UAT.

- **Contoh Penggunaan:** (di #uat-new-onboarding) "Halo tim UAT, *build* terbaru untuk *onboarding* sudah *deploy* di *staging*. Mohon bantuannya untuk melakukan *testing* sesuai skenario di Notion [Link Test Plan UAT]."

Alur Eskalasi Bug (Aturan 3: Alur Eskalasi Bug)

Salah satu fungsi krusial Slack adalah sebagai bagian dari alur eskalasi *bug*. Kecepatan dan visibilitas adalah kunci, terutama untuk *bug* yang kritis.

P0 / P1:

- **Tindakan:** Segera *file bug* di Notion.

- **Notifikasi Slack:** Posting ke *channel* `#incidents`.

- **Tambahan:** Kirim *Direct Message* (DM) ke Tech Lead atau *developer* yang bertanggung jawab secara langsung.

- **Tujuan:** Memastikan *bug* segera diketahui dan ditindaklanjuti karena dampaknya yang besar.

P2:

- **Tindakan:** *File bug* di Notion.

- **Notifikasi Slack:** Posting ke *channel* `#qa`.

- **Tujuan:** Memberikan visibilitas kepada tim QA dan *developer* terkait bahwa ada *bug* yang perlu diperbaiki, namun tidak se-kritis P0/P1.

P3:

- **Tindakan:** *File bug* di Notion saja.

- **Tujuan:** *Bug* ini akan muncul dalam *backlog review* berikutnya dan akan diprioritaskan sesuai keputusan tim produk, tanpa perlu notifikasi *real-time* di Slack.

Contoh Dialog Eskalasi P0 di Slack:

[Anda] di `#incidents`:

■ P0 INCIDENT ALERT ■

[Login] Pengguna tidak bisa login sama sekali di staging. Memblokir semua aktivitas.

Severity: P0

Steps to Reproduce:

1. Buka halaman login
2. Masukkan kredensial valid
3. Klik "Login"

Expected: Berhasil masuk dashboard

Actual: Error 500 "Internal Server Error"

Environment: Staging, Chrome latest, Build #1234

Link Notion Bug: [Link ke Bug Notion Anda]

@TechLeadXYZ mohon segera dicek.

■

[Anda] via DM ke Tech Lead:

Halo [Nama Tech Lead], saya baru saja posting P0 di `#incidents` terkait masalah login di staging. Ini s

[Tech Lead] via DM ke Anda:

Baik, Thomas. Saya langsung cek. Terima kasih infonya.

Tips: QA Thomas sering menggunakan `slack_connector.SlackConnector` dan skrip `scripts/qa_report_bug.py` untuk mengotomatiskan proses pelaporan *bug* ke Slack setelah *bug* difile di Notion, memastikan tidak ada langkah yang terlewat.

3. Google Drive: Gudang Bukti Visual dan Laporan Resmi

Google Drive berfungsi sebagai repositori untuk semua aset non-struktural yang penting bagi QA Thomas. Ini adalah tempat Anda menyimpan bukti visual *bug* dan laporan resmi yang perlu dibagikan atau diarsipkan.

Penyimpanan Video dan Screenshot Evidence

Ketika Anda menemukan *bug*, deskripsi teks saja seringkali tidak cukup. Bukti visual, seperti *screenshot* atau rekaman video, dapat menyampaikan masalah dengan lebih jelas dan cepat kepada *developer*.

Mengapa Penting:

- **Kejelasan:** *Screenshot* atau video dapat menunjukkan dengan tepat apa yang salah di UI/UX atau alur kerja, mengurangi ambiguitas.
- **Reproduksi:** Video dapat membantu *developer* memahami langkah-langkah reproduksi yang kompleks atau *bug* yang *intermittent*.
- **Efisiensi:** Menghemat waktu *developer* dalam mencoba mereplikasi *bug* berdasarkan deskripsi teks semata.

Konvensi:

- Setiap *screenshot* atau *video evidence* harus diberi nama yang jelas (e.g., `bug-login-error-500-20231026.mp4`).

- Simpan dalam folder yang terorganisir, misalnya `Drive/ProjectX/QA-Evidence/Bug-ID-XXX/`.

- Sertakan *link* ke *file* ini dalam deskripsi *bug* di Notion. Ini adalah bagian dari **Aturan 2: Standar Pelaporan Bug** yang mengharuskan adanya *screenshots or video for visual bugs*.

Contoh Link dalam Deskripsi Bug Notion:

...

Expected: Pengguna berhasil login dan diarahkan ke halaman dashboard.

Actual: Setelah klik tombol "Login", muncul error 500 di browser.

Evidence:

- Screenshot Error: [Link Google Drive ke Screenshot]
- Video Reproduksi: [Link Google Drive ke Video]

...

Laporan UAT (PDF Report)

Setelah siklus *User Acceptance Testing* (UAT) selesai, QA Thomas bertanggung jawab untuk menyusun Laporan UAT. Laporan ini merupakan dokumen formal yang merangkum hasil UAT, termasuk *feedback* dari *stakeholder*, *bug* yang ditemukan, dan status keseluruhan kesiapan fitur untuk *release*.

Mengapa Penting:

- **Rekam Resmi:** Laporan UAT berfungsi sebagai catatan resmi bahwa fitur telah diuji dan disetujui (atau ditolak) oleh *stakeholder*.
- **Dasar Keputusan Rilis:** PM dan manajemen akan menggunakan laporan ini sebagai salah satu dasar untuk memutuskan apakah sebuah fitur siap untuk *go-live*.
- **Transparansi:** Memberikan gambaran yang jelas kepada semua pihak tentang status kualitas fitur.

Konvensi:

- Laporan UAT biasanya disimpan dalam format PDF untuk menjaga integritas dokumen dan memudahkan berbagi.
- Simpan dalam folder `Drive/ProjectX/QA-Reports/UAT/`.

- Gunakan *template* yang konsisten (seperti `references/uat-report-template.md` yang mungkin Anda miliki).

Tips: Gunakan `gdrive_connector.GDriveConnector` jika Anda ingin mengintegrasikan proses pembuatan atau pengunggahan laporan UAT secara otomatis ke Google Drive, misalnya setelah laporan selesai dibuat dari *template*.

4. API dan Database Connector: Mengintip Balik Layar

Sebagai QA Engineer yang komprehensif, Anda tidak bisa hanya mengandalkan antarmuka pengguna (UI) untuk melakukan *testing* dan verifikasi. Seringkali, Anda perlu "mengintip balik layar" untuk menyiapkan data *test*, memverifikasi kondisi *backend*, atau bahkan memicu alur kerja tertentu secara langsung. Di sinilah peran API dan Database *Connector* menjadi sangat vital.

Setup dan Teardown Data

Salah satu tantangan terbesar dalam *testing* adalah memastikan lingkungan *test* berada dalam kondisi yang tepat. Anda mungkin perlu membuat pengguna dengan peran tertentu, mengisi *database* dengan data spesifik, atau membersihkan data setelah *test* selesai. Melakukan ini secara manual melalui UI bisa memakan waktu dan rentan kesalahan.

API Connector (`api_connector.APIConnector`):

Tujuan: Mengirim permintaan HTTP langsung ke *backend API*. Ini sangat berguna untuk:

- **Membuat data *test*:** Misalnya, membuat akun pengguna baru, pesanan, atau item inventaris secara terprogram.
- **Memicu *event*:** Mengirim *event* yang mungkin sulit atau tidak mungkin dipicu melalui UI (e.g., notifikasi *push* internal, *webhook*).
- **Mengatur *state* pengguna:** Mengubah status akun pengguna (e.g., dari `pending` menjadi `active`).
- **Tools:** Anda bisa menggunakan *tool* seperti Postman, Insomnia, atau bahkan `cURL` di *command line* untuk melakukan ini secara manual. Untuk otomatisasi, Anda bisa menulis skrip

Python yang menggunakan `requests` *library* untuk berinteraksi dengan `api_connector.APIConnector`.

Contoh Penggunaan API Connector (via Postman/cURL):

Anda perlu menguji skenario di mana pengguna memiliki 5 item di keranjang belanja. Daripada mengklik 5 kali di UI, Anda bisa langsung menggunakan API:

```
# Contoh cURL untuk menambahkan item ke keranjang
curl -X POST \
  http://api.your-app.com/cart/add \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <user_token>' \
  -d '{
    "userId": "user123",
    "productId": "prodABC",
    "quantity": 5
  }'
```

Database Connector (`db_connector.DBConnector`):

Tujuan: Berinteraksi langsung dengan *database* aplikasi. Ini sangat berguna untuk:

- **Memasukkan data *test*:** Membuat data *test* yang kompleks atau dalam jumlah besar langsung ke *database*.
- **Mengubah *state* data:** Memodifikasi nilai-nilai di *database* untuk mencapai kondisi *test* tertentu (e.g., mengubah status pesanan, memperbarui saldo pengguna).
- **Membersihkan data (*teardown*):** Menghapus data *test* setelah *test* selesai untuk menjaga kebersihan lingkungan *staging*.
- **Tools:** Anda akan menggunakan *tool* klien *database* seperti DBeaver, DataGrip, MySQL Workbench, atau PgAdmin. Untuk otomatisasi, Anda bisa menulis skrip Python yang menggunakan *library* konektor *database* (e.g., `psycopg2` untuk PostgreSQL, `mysql-connector` untuk MySQL) untuk berinteraksi dengan `db_connector.DBConnector`.

Contoh Penggunaan Database Connector (via SQL Client):

Anda perlu membuat pengguna dengan peran admin untuk menguji fungsionalitas admin.


```
-- Memasukkan pengguna baru dengan peran admin
INSERT INTO users (id, name, email, password_hash, role, status)
VALUES ('user_admin_001', 'Admin Thomas', 'admin.thomas@example.com', 'hashed_password_here', 'admin',
-- Mengupdate saldo pengguna menjadi 0 untuk skenario test tertentu
UPDATE user_accounts
SET balance = 0
WHERE user_id = 'user_test_user_id';
```

Verifikasi State Beyond UI

Terkadang, *bug* tidak terlihat langsung di UI. Sebuah transaksi mungkin terlihat berhasil di UI, tetapi di *backend*, data mungkin tidak disimpan dengan benar atau proses *background* gagal.

Verifikasi dengan API Connector:

- Setelah melakukan tindakan di UI (e.g., melakukan pembelian), Anda bisa menggunakan *API connector* untuk memanggil API *backend* yang relevan (e.g., API untuk mendapatkan detail pesanan) untuk memverifikasi bahwa data telah diproses dan disimpan dengan benar di *backend*.
- Ini juga berguna untuk menguji API itu sendiri (*API testing*), memastikan *endpoint* mengembalikan respons yang benar dengan *payload* yang diharapkan.

Verifikasi dengan Database Connector:

- Ini adalah cara paling pasti untuk memeriksa integritas data. Setelah sebuah tindakan dilakukan, Anda bisa langsung mengkueri *database* untuk memastikan data telah di-*insert*, di-*update*, atau di-*delete* sesuai harapan.
- Sangat penting untuk menguji skenario di mana UI mungkin menampilkan hasil yang salah, tetapi data *backend* sebenarnya benar (atau sebaliknya).

Contoh Verifikasi Database:

Setelah pengguna mengubah profilnya di UI, Anda ingin memastikan nama dan email baru benar-benar tersimpan di *database*.

```
-- Memverifikasi data profil pengguna setelah update di UI
SELECT name, email, last_updated_at
```

```
FROM users
WHERE id = 'user_test_user_id';
```

Dengan menguasai penggunaan API dan Database *Connector*, Anda akan menjadi QA Engineer yang jauh lebih powerful, mampu melakukan *testing* yang lebih mendalam, menyiapkan lingkungan *test* dengan efisien, dan memverifikasi kualitas di setiap lapisan aplikasi. Ini adalah bagian penting dari Automation Exploratory yang disebutkan dalam *skill set* QA Thomas, di mana Anda menggunakan *tool* seperti Charles/Postman untuk eksplorasi dan verifikasi *backend*.

5. Handoff: Estafet Kualitas Antar Tim

QA Thomas tidak bekerja dalam isolasi. Kualitas adalah tanggung jawab bersama, dan *handoff* adalah mekanisme penting untuk memastikan informasi mengalir dengan lancar antar tim. Ini adalah momen di mana Anda menyerahkan "tongkat estafet" kualitas kepada *stakeholder* lain, memastikan mereka memiliki informasi yang dibutuhkan untuk mengambil keputusan atau melakukan tindakan selanjutnya.

Kapan Handoff Terjadi dan Kepada Siapa

Handoff ke PM (Product Manager):

Kapan:

- **Ringkasan Bug:** Setelah menemukan *bug*, terutama yang memiliki dampak pada fungsionalitas atau pengalaman pengguna, ringkasan *bug* perlu disampaikan ke PM untuk *triage* dan prioritas.
- **Hasil UAT:** Setelah siklus UAT selesai, PM perlu menerima Laporan UAT untuk memahami status kesiapan fitur dan memutuskan rilis.
- **Klarifikasi Fitur:** Jika ada ambiguitas dalam *User Story* atau PRD selama *Requirement Analysis*, *handoff* ke PM diperlukan untuk klarifikasi.
- **Bagaimana:** Melalui *meeting* (standup, *sprint review*), Slack, atau Notion (dengan *me-mention* PM di *bug* atau *test plan*).

- **Tujuan:** Memastikan PM memiliki gambaran kualitas produk dan dapat membuat keputusan produk yang tepat. Proses ini juga memanfaatkan `handoff.handoff_to_pm` yang ada dalam *skill set* QA Thomas.

Handoff ke Tech Lead/Developer:

Kapan:

- **Detail Bug:** Segera setelah *bug* dilaporkan, terutama P0/P1, detail lengkapnya perlu di-*handoff* ke Tech Lead atau *developer* yang relevan untuk perbaikan.
- **Klarifikasi Teknis:** Jika ada pertanyaan teknis terkait implementasi atau desain selama *Test Design* atau *Test Execution*, *handoff* ke *developer* diperlukan.

- **Bagaimana:** Melalui Slack (DM atau *channel* `#incidents/#qa`), Notion (dengan *me-assign bug* ke *developer*), atau *daily standup*.

- **Tujuan:** Memfasilitasi perbaikan *bug* yang cepat dan efisien, serta menjaga pemahaman teknis yang sinkron.

Handoff ke Marketing/Komunikasi:

Kapan:

- **Release Notes:** Sebelum *deployment* ke *production*, tim Marketing atau Komunikasi perlu menerima *Release Notes* dari QA Thomas untuk mempersiapkan pengumuman fitur baru atau perbaikan kepada pengguna.

- **Highlight Fitur:** Informasi tentang fitur baru yang sudah teruji dan siap rilis bisa di-*handoff* agar Marketing bisa mempersiapkan materi promosi.

- **Bagaimana:** Melalui dokumen bersama (Google Docs/Notion), atau *meeting* koordinasi rilis.

- **Tujuan:** Memastikan pesan yang disampaikan ke pengguna akurat dan sesuai dengan apa yang telah diuji dan dirilis.

Bagaimana Handoff Dilakukan: Praktik Terbaik

Meeting Updates:

- **Daily Standup:** Berikan *update* singkat tentang progres *testing*, *bug* kritis yang ditemukan, dan potensi *blocking issue*.

- **Sprint Review/Demo:** Hadirkan demo *fitur* yang sudah diuji, sampaikan hasil *testing*, dan tunjukkan *bug* yang masih terbuka.

Pesan Slack:

- Gunakan *channel* yang tepat (seperti yang dibahas di bagian Slack) untuk notifikasi cepat.

- Selalu sertakan *link* ke Notion (untuk *bug* atau *test plan*) agar *stakeholder* dapat menggali detail lebih lanjut.

Notion Updates:

- Pastikan *status bug* dan *Test Case* selalu *up-to-date*.

- Gunakan fitur *@mention* di Notion untuk memberitahu *stakeholder* tertentu tentang perubahan atau *update* yang relevan.

Laporan Formal:

- Laporan UAT (PDF di Google Drive) adalah contoh *handoff* formal yang penting untuk keputusan rilis.

- *Test Execution Report* yang merangkum semua *test* yang dijalankan dan hasilnya.

Contoh Dialog Handoff Hasil UAT ke PM:

Bayangkan Anda baru saja menyelesaikan UAT untuk fitur "Pendaftaran Pengguna Baru" dan siap melaporkan hasilnya kepada PM.

[Anda] di #product-team Slack:

Halo @PM_Thomas, saya sudah menyelesaikan UAT untuk fitur Pendaftaran Pengguna Baru.

Secara keseluruhan, hasilnya positif dan **acceptance criteria** terpenuhi.

Ada 2 **bug** minor (P3) yang ditemukan dan sudah saya **file** di Notion, tidak memblokir rilis.

Laporan UAT lengkapnya bisa diakses di Google Drive: [Link Google Drive ke Laporan UAT PDF]

Apakah kita bisa atur waktu sebentar untuk saya jelaskan detailnya?

[PM_Thomas] di #product-team Slack:

Hai QA Thomas, terima kasih banyak atas **update** cepatnya!

Sangat bagus mendengar hasilnya positif. Saya sudah lihat laporannya sekilas.

Bagaimana kalau kita bahas di **standup** besok pagi? Saya akan siapkan beberapa pertanyaan.

[Anda] di #product-team Slack:

Siap, PM Thomas! Sampai besok.

Handoff yang efektif adalah tanda dari tim QA yang matang. Ini menunjukkan bahwa Anda tidak hanya fokus pada tugas Anda sendiri, tetapi juga pada bagaimana pekerjaan Anda berkontribusi pada tujuan keseluruhan tim dan perusahaan. Dengan menguasai *handoff*, Anda akan memperkuat peran Anda sebagai penghubung kualitas yang tak tergantikan.

Rangkuman

Selamat! Anda telah menjelajahi ekosistem *tools* dan alur kerja yang menjadi tulang punggung operasi QA Thomas. Kita telah melihat bagaimana Notion menjadi pusat komando untuk dokumentasi *testing* dan pelaporan *bug* yang terstruktur, dengan penekanan pada keterkaitan *Test Case* ke *User Story* dan standar pelaporan *bug* yang ketat. Slack berperan sebagai jembatan komunikasi *real-time*, memfasilitasi eskalasi *bug* yang efisien melalui *channel* yang tepat dan alur eskalasi P0-P3 yang jelas. Google Drive menjadi gudang aman untuk bukti visual *bug* dan laporan resmi seperti Laporan UAT.

Lebih jauh lagi, Anda juga memahami pentingnya API dan Database *Connector* untuk melakukan *setup* dan *teardown* data *test* serta memverifikasi kondisi *backend*, memberikan kemampuan *testing* yang lebih mendalam di luar UI. Terakhir, kita membahas esensi *handoff*—bagaimana dan kapan Anda mengomunikasikan informasi kritis kepada PM, Tech Lead, dan tim Marketing, memastikan kolaborasi yang mulus dan keputusan yang terinformasi.

Menguasai *tools* ini bukan hanya tentang mengetahui fungsinya, tetapi bagaimana mengintegrasikannya ke dalam alur kerja Anda untuk menjadi QA Engineer yang lebih efisien, transparan, dan berdampak. Setiap *tool* adalah perpanjangan dari proses Anda, memungkinkan Anda untuk tidak hanya menemukan *bug*, tetapi juga menjadi penjaga kualitas yang proaktif dan komunikatif.

Checklist Actionable

Berikut adalah beberapa langkah yang bisa Anda terapkan segera untuk mengoptimalkan penggunaan *tools* dan *workflow* Anda:

Notion - Test Documentation:

- Pastikan setiap *Test Case* yang Anda buat di Notion memiliki properti *Linked Task* yang terisi dan terhubung ke *User Story* yang relevan.
- Tinjau kembali *Test Case* lama Anda di Notion; apakah properti *Status*, *Owner*, dan *Type* sudah terisi dengan konsisten?

Notion - Bug Reporting:

- Saat melaporkan *bug* berikutnya, pastikan Anda mengisi semua properti sesuai **Aturan 2: Standar Pelaporan Bug** (Judul, Langkah Reproduksi, Expected vs Actual, Environment, Severity, Bukti Visual, Reproducible).
- Hafalkan *Severity Rubric* (P0-P3) agar Anda dapat mengklasifikasikan *bug* dengan tepat.

Slack - Komunikasi & Eskalasi:

- Pahami dan terapkan **Aturan 3: Alur Eskalasi Bug** untuk P0, P1, P2, dan P3, termasuk *channel* Slack yang sesuai dan kapan harus DM Tech Lead.
- Identifikasi *channel* Slack yang relevan untuk proyek Anda (e.g., *#engineering*, *#qa*, *#incidents*, *#uat-<feature>*) dan biasakan posting di *channel* yang tepat.

Google Drive - Dokumentasi:

- Biasakan diri untuk selalu menyimpan *screenshot* atau rekaman video sebagai bukti visual di Google Drive untuk setiap *bug* yang Anda laporkan.
- Pastikan Anda tahu di mana menyimpan Laporan UAT final dalam format PDF di Google Drive agar mudah diakses oleh *stakeholder* lain.

API/Database Connector:

- Eksplorasi penggunaan *tool* seperti Postman atau *SQL client* (DBeaver, DataGrip) untuk melakukan *setup data test* atau verifikasi *state backend*.
- Identifikasi satu skenario *testing* di mana Anda bisa menggunakan API atau *database connector* untuk verifikasi yang lebih efisien dibandingkan hanya mengandalkan UI.

Handoff:

- Identifikasi *stakeholder* utama (PM, Tech Lead, Marketing) dalam tim Anda dan pahami informasi apa yang mereka butuhkan dari Anda dan kapan.
- Biasakan memberikan *update* singkat mengenai kualitas atau *bug* kritis dalam *daily standup* atau *meeting* tim lainnya.

Chapter 09: Penutup — Resources, Templates, dan Next Steps

Selamat! Anda telah menyelesaikan perjalanan yang luar biasa bersama saya, Thomas, dalam buku "Journey QA Thomas: Panduan Lengkap Menjadi Quality Assurance Engineer — dari Requirement Analysis hingga Automation". Ini bukan sekadar buku, melainkan sebuah peta jalan yang telah membawa Anda melintasi berbagai tahapan krusial dalam dunia Quality Assurance. Dari menganalisis kebutuhan hingga mengotomatisasi pengujian, Anda kini memiliki pemahaman yang komprehensif tentang peran dan tanggung jawab seorang QA Engineer sejati.

Mungkin di awal Anda berpikir bahwa QA hanyalah tentang "klik-klik tombol" atau "mencari bug". Namun, sepanjang chapter demi chapter, kita telah membongkar mitos tersebut. Anda telah melihat bagaimana QA adalah gerbang kualitas pertama dan terakhir, penjaga utama yang memastikan setiap fitur dan setiap aliran kerja sesuai dengan kriteria penerimaan pengguna. Anda adalah mata dan telinga bagi pengguna, memastikan bahwa produk yang kita bangun benar-benar memberikan nilai dan pengalaman terbaik.

Chapter penutup ini bukan akhir, melainkan awal yang baru. Kita akan merangkum kembali apa yang telah Anda pelajari, memberikan Anda alat tambahan yang siap pakai, dan memandu langkah Anda selanjutnya untuk menerapkan semua pengetahuan ini ke dalam proyek nyata. Mari kita pastikan bahwa Anda tidak hanya mengerti, tetapi juga siap untuk *bertindak* sebagai Quality Assurance Engineer yang kompeten dan berharga.

Rekap Perjalanan QA Thomas: 5 Sub-flow Kunci

Selama perjalanan ini, kita telah membahas lima sub-flow utama yang membentuk tulang punggung peran QA Thomas. Kelima sub-flow ini saling terkait dan dijalankan pada tahapan yang berbeda dalam siklus pengembangan produk, memastikan kualitas terjaga dari hulu ke hilir. Ingatlah selalu **Prime Directive** kita:

"Komunikasi untuk mengecek apakah flow sudah sesuai dengan criteria user story."

Apapun yang menyimpang dari kriteria penerimaan, meskipun kodenya berjalan tanpa *error*, tetaplah sebuah cacat (defect) dari perspektif QA. Berikut adalah rekap singkat dari kelima sub-flow tersebut:

1. Requirement Analysis (Analisis Kebutuhan)

- **Kapan Dijalankan:** Tahap paling awal, bahkan sebelum tim engineering mulai menulis kode. Idealnya, ini dilakukan saat Product Manager (PM) masih menyusun Product Requirement Document (PRD) atau User Story.
- **Tujuan:** Memastikan bahwa kebutuhan pengguna dipahami dengan jelas, tidak ambigu, dan dapat diuji. QA bertindak sebagai "quality gate" pertama, menangkap potensi kesalahpahaman atau celah dalam spesifikasi sebelum menjadi masalah yang lebih besar di kemudian hari.
- **Peran Anda:** Membaca PRD secara kritis, mengajukan pertanyaan klarifikasi, memecah user story menjadi skenario pengujian, dan mengidentifikasi *acceptance criteria* yang jelas.

2. Test Design (Desain Pengujian)

- **Kapan Dijalankan:** Setelah Requirement Analysis selesai dan kebutuhan sudah jelas, namun masih sebelum atau di awal tahap pengembangan.
- **Tujuan:** Membuat strategi pengujian yang komprehensif, merencanakan estimasi waktu yang realistis, dan menulis *test case* yang mencakup berbagai skenario (positif, negatif, *edge case*, persiapan data).
- **Peran Anda:** Merancang *test strategy* yang sesuai dengan risiko dan prioritas, membuat estimasi waktu yang akurat untuk aktivitas QA, dan menulis *test case* yang detail dan dapat direproduksi.

3. Test Execution (Eksekusi Pengujian)

- **Kapan Dijalankan:** Sepanjang siklus pengembangan, mulai dari lingkungan pengembangan (DEV), *staging*, hingga pra-produksi. Dilakukan secara berulang untuk berbagai jenis pengujian.
- **Tujuan:** Menjalankan *test case* yang telah dirancang, mengidentifikasi *bug*, dan memverifikasi fungsionalitas produk. Fokus pada urutan eksekusi yang *fail-fast* untuk menemukan masalah sesegera mungkin.
- **Peran Anda:** Melakukan *smoke testing* (otomatis/manual), *functional testing per user story*, *exploratory testing* untuk menemukan kasus tak terduga, dan *regression testing* sebelum setiap rilis.

4. Reporting & Documentation (Pelaporan & Dokumentasi)

- **Kapan Dijalankan:** Berlangsung secara paralel dengan Test Execution dan pada setiap *milestone* penting (misalnya, setelah UAT, sebelum rilis).
- **Tujuan:** Mengkomunikasikan status kualitas produk kepada *stakeholder*, melacak *bug*, dan menyimpan catatan historis tentang aktivitas pengujian.
- **Peran Anda:** Membuat laporan *User Acceptance Testing* (UAT), menyusun *Release Notes*, mendokumentasikan *test case* dan *test strategy*, serta membuat *Test Execution Report*.

5. Stakeholder Engagement (Keterlibatan Pemangku Kepentingan)

- **Kapan Dijalankan:** Sepanjang siklus pengembangan. QA bukanlah peran yang bekerja sendiri; interaksi dengan tim lain sangatlah penting.
- **Tujuan:** Membangun hubungan yang kuat dengan *stakeholder* (PM, Developer, UI/UX, Business), mengkomunikasikan informasi kualitas secara efektif, dan mengkoordinasikan aktivitas seperti UAT.
- **Peran Anda:** Berpartisipasi aktif dalam rapat, memberikan *demo* yang jelas dan terstruktur, serta mengkoordinasikan proses UAT dengan pengguna akhir.

Berikut adalah ringkasan dalam bentuk tabel:

Sub-flow	Kapan Dijalankan	Tujuan Utama	Contoh Aktivitas
Requirement Analysis	Tahap awal (sebelum coding)	Memastikan kebutuhan jelas & dapat diuji	Membaca PRD, memecah User Story, mendefinisikan Acceptance Criteria
Test Design	Setelah Requirement Analysis, sebelum/awal coding	Membuat strategi pengujian & test case komprehensif	Menyusun Test Strategy, Estimasi Waktu QA, Menulis Test Case (positif, negatif, edge)
Test Execution	Sepanjang siklus (DEV, Staging, Prod)	Menjalankan pengujian, menemukan bug, memverifikasi fungsionalitas	Smoke Test, Functional Test, Exploratory Test, Regression Test
Reporting & Documentation	Paralel dengan Test Execution & Milestone penting	Mengkomunikasikan status kualitas & menyimpan catatan historis	UAT Report, Release Notes, Test Documentation, Test Execution Report
Stakeholder Engagement	Sepanjang siklus pengembangan	Membangun hubungan, komunikasi efektif, koordinasi UAT	Demo produk, Partisipasi rapat, Koordinasi UAT dengan user/PM

Kumpulan Template Siap Pakai

Salah satu kunci efisiensi dan konsistensi dalam kerja QA adalah penggunaan template standar. Kita telah membahas beberapa template penting yang akan sangat membantu Anda dalam menjalankan tugas sehari-hari. Template-template ini dirancang untuk memastikan semua informasi yang relevan tercatat dengan baik dan dikomunikasikan secara efektif.

Anda dapat menganggap template ini sebagai "bekal" yang telah disiapkan oleh QA Thomas untuk Anda. Mari kita lihat kembali template-template tersebut:

1. Template Test Case

- **Tujuan:** Mendokumentasikan langkah-langkah pengujian, hasil yang diharapkan, dan kriteria keberhasilan untuk setiap skenario.

- **Kapan Digunakan:** Selama fase Test Design, sebelum eksekusi pengujian.

Poin Kunci:

- **ID Test Case:** Unik untuk identifikasi.

- **Nama Test Case:** Deskripsi singkat tujuan pengujian.

- **Linked Task/User Story:** Mengaitkan dengan kebutuhan asli.

- **Pre-conditions:** Persyaratan sebelum test case dapat dijalankan.

- **Steps to Reproduce:** Langkah-langkah detail yang dapat direproduksi.

- **Expected Result:** Hasil yang diharapkan jika fungsionalitas bekerja dengan benar.

- **Actual Result:** Hasil yang sebenarnya setelah eksekusi.

- **Status:** Pass/Fail/Blocked/Skipped.

- **Severity:** Dampak jika gagal.

- **Environment:** Lingkungan pengujian (Dev/Staging/Prod).

- **Notes:** Catatan tambahan atau observasi.

Contoh Singkat: `**ID Test Case:** TC-LOGIN-001 **Nama Test Case:** Login dengan kredensial valid **Linked Task:** US-001: Sebagai pengguna, saya dapat login ke sistem. **Pre-conditions:** Pengguna terdaftar dengan`

`email``user@example.com`dan `password``password123``. **Steps to Reproduce:**

1. Buka halaman login.
2. Masukkan email `user@example.com` pada field 'Email'.
3. Masukkan password `password123` pada field 'Password'.
4. Klik tombol 'Login'. **Expected Result:** Pengguna berhasil login dan diarahkan ke halaman Dashboard. ``

- **Ditemukan di Chapter:** [Chapter 02: Test Design — Strategi, Estimasi, dan Test Case]

2. Template Bug Report

- **Tujuan:** Melaporkan *bug* atau *defect* secara jelas, ringkas, dan dapat ditindaklanjuti oleh tim pengembangan.

- **Kapan Digunakan:** Selama fase Test Execution, ketika *bug* ditemukan.

Poin Kunci:

- **Judul Bug:** [Area] Deskripsi singkat masalah.
- **Langkah-langkah Reproduksi:** Detail langkah-langkah untuk mengulang *bug*.
- **Hasil yang Diharapkan:** Apa yang seharusnya terjadi.
- **Hasil Aktual:** Apa yang sebenarnya terjadi.
- **Lingkungan:** Browser, OS, perangkat, URL, versi *build*.
- **Severity (P0-P3):** Tingkat dampak *bug*.
- **Reproducible:** Always / Intermittent / Once / Cannot reproduce.
- **Lampiran:** Screenshot atau video.

Contoh Singkat: ``markdown **Judul Bug:** [Login Page] Tombol "Lupa Password" tidak berfungsi **Langkah-langkah Reproduksi:**

1. Buka halaman login (URL: `staging.aplikasi.com/login`).

2. Klik tautan "Lupa Password". **Hasil yang Diharapkan:** Pengguna diarahkan ke halaman "Reset Password". **Hasil Aktual:** Halaman error 404 ditampilkan. **Lingkungan:** Chrome 120, Windows 11, Staging, Build #1.2.3 **Severity:** P1 (Core feature broken, no workaround)
Reproducible: Always ```

- **Ditemukan di Chapter:** [Chapter 04: Bug Reporting — Severity, Template, dan Escalation]

3. Template UAT Report (Laporan Pengujian Penerimaan Pengguna)

- **Tujuan:** Merangkum hasil pengujian yang dilakukan oleh pengguna akhir (UAT), termasuk *feedback*, *bug* yang ditemukan, dan status kesiapan rilis.

- **Kapan Digunakan:** Setelah fase UAT selesai.

Poin Kunci:

- **Proyek/Fitur:** Nama proyek atau fitur yang diuji.
- **Periode UAT:** Tanggal mulai dan berakhir.
- **Peserta UAT:** Daftar pengguna yang terlibat.
- **Skenario UAT Utama:** Fungsi utama yang diuji.
- **Ringkasan Hasil:** Status keseluruhan (Pass/Fail/Pending).
- **Bug Ditemukan:** Daftar bug kritis yang ditemukan selama UAT.
- **Feedback/Saran:** Masukan dari pengguna.
- **Rekomendasi QA:** Rekomendasi untuk rilis atau perbaikan lebih lanjut.

- **Ditemukan di Chapter:** [Chapter 05: Reporting & Documentation — UAT, Release Notes, Knowledge Base]

4. Template Release Notes

- **Tujuan:** Memberikan informasi kepada pengguna atau *stakeholder* tentang fitur baru, perbaikan *bug*, dan perubahan lain dalam rilis produk.
- **Kapan Digunakan:** Sebelum setiap rilis produk ke lingkungan produksi.

Poin Kunci:

- **Versi Rilis:** Nomor versi.
- **Tanggal Rilis:** Kapan rilis akan dilakukan.
- **Fitur Baru:** Daftar fitur baru dengan deskripsi singkat.
- **Perbaikan Bug:** Daftar *bug* penting yang telah diperbaiki.
- **Perubahan Lain:** Peningkatan kinerja, perubahan UI kecil, dll.
- **Known Issues:** Masalah yang diketahui tetapi belum diperbaiki.
- **Cara Update:** Instruksi jika diperlukan.
- **Ditemukan di Chapter:** [Chapter 05: Reporting & Documentation — UAT, Release Notes, Knowledge Base]

5. Struktur Test Strategy (dari Test Design)

- **Tujuan:** Mendefinisikan pendekatan pengujian secara keseluruhan untuk sebuah proyek atau fitur, termasuk ruang lingkup, jenis pengujian, dan sumber daya.
- **Kapan Digunakan:** Awal proyek atau saat merencanakan pengujian untuk fitur besar.

Poin Kunci:

- **Pendahuluan:** Tujuan pengujian.
- **Ruang Lingkup:** Apa yang akan dan tidak akan diuji.
- **Jenis Pengujian:** Functional, non-functional, automation, manual, dll.
- **Lingkungan Pengujian:** Kebutuhan lingkungan.
- **Data Pengujian:** Strategi persiapan data.
- **Metrik & Pelaporan:** Cara mengukur dan melaporkan kemajuan.
- **Peran & Tanggung Jawab:** Siapa melakukan apa.
- **Estimasi Waktu QA:** Penjelasan detail estimasi waktu (seperti yang dibahas di Chapter 2).

Contoh Estimasi Waktu (bagian dari Test Strategy): ```markdown **Estimasi Waktu QA untuk Fitur X:**

- Test case authoring: 8 jam
- Test data prep: 4 jam
- Execution (first pass): 12 jam
- Regression: 4 jam
- Buffer for bug verification: 6 jam **TOTAL: 34 jam / 4.25 hari kerja** ```

- **Ditemukan di Chapter:** [Chapter 02: Test Design — Strategi, Estimasi, dan Test Case]

Mengingat pentingnya template ini, saya sangat menyarankan Anda untuk mengunduh dan mengadaptasinya sesuai dengan kebutuhan tim atau proyek Anda. Ketersediaan template ini di awal perjalanan Anda akan mempercepat proses adopsi dan membantu Anda menjaga standar kualitas yang tinggi.

Self-Assessment: 10 Pertanyaan untuk Cek Kesiapan Menjalankan QA Cycle

Sekarang, mari kita evaluasi sendiri seberapa jauh pemahaman dan kesiapan Anda untuk terjun langsung ke dunia QA sebagai Quality Assurance Engineer. Jawablah pertanyaan-pertanyaan ini dengan jujur. Tidak ada jawaban benar atau salah mutlak, tujuannya adalah untuk mengidentifikasi area di mana Anda mungkin perlu mengulas kembali atau mendapatkan lebih banyak pengalaman.

Ambil pena dan kertas (atau aplikasi catatan favorit Anda) dan berikan skor pada diri sendiri atau tuliskan refleksi singkat.

Requirement Analysis: Bisakah Anda mengidentifikasi *missing requirements* atau ambiguitas dalam sebuah *user story* atau PRD bahkan sebelum tim mulai *coding*? Jelaskan bagaimana Anda akan melakukannya.

- *Refleksi:* Apakah saya akan bertanya "bagaimana jika" atau "skenario negatifnya seperti apa" kepada PM?

Test Design: Jika Anda diminta untuk menguji fitur login, bisakah Anda membuat setidaknya 4 kategori *test case* (positif, negatif, *edge case*, persiapan data) yang komprehensif? Sebutkan satu contoh untuk masing-masing kategori.

- *Refleksi:* Apakah saya memahami perbedaan dan pentingnya setiap kategori test case?

Test Execution Strategy: Bagaimana Anda akan menentukan jenis pengujian apa yang perlu dijalankan (Smoke, Functional, Exploratory, Regression) untuk sebuah rilis minor yang hanya mencakup perbaikan *bug* kecil?

- *Refleksi:* Apakah saya tahu kapan harus melakukan *full regression* versus *targeted regression*?

Bug Reporting: Anda menemukan *bug* di mana sebuah halaman penting di produksi tidak bisa diakses (error 500). Dengan *severity rubric* P0-P3, *severity* apa yang akan Anda berikan dan langkah eskalasi apa yang akan Anda ambil?

- *Refleksi:* Apakah saya memahami dampak *severity* dan pentingnya eskalasi yang tepat?

Documentation: Setelah UAT selesai, dokumen apa saja yang perlu Anda siapkan dan kepada siapa Anda akan mendistribusikannya?

- *Refleksi:* Apakah saya tahu tujuan dari setiap dokumen QA dan audiensnya?

Stakeholder Engagement: Anda diminta untuk memberikan *demo* fitur baru kepada tim Product dan Business. Bagaimana Anda mempersiapkan *demo* tersebut agar berjalan lancar dan tidak "meledak" di tengah jalan?

- *Refleksi:* Apakah saya siap menghadapi skenario terburuk dan tahu cara menjaga profesionalisme?

Automation Mindset: Meskipun belum menguasai *coding*, bisakah Anda mengidentifikasi fitur atau skenario mana yang paling cocok untuk diotomatisasi dalam sebuah aplikasi web, dan mengapa?

- *Refleksi:* Apakah saya memahami nilai dan batasan *automation testing*?

Estimasi Waktu: Jika Anda diminta untuk menguji sebuah fitur baru yang kompleks, bagaimana Anda akan menyusun estimasi waktu QA yang realistis dan detail untuk PM?

- *Refleksi:* Apakah saya bisa memecah tugas QA menjadi bagian-bagian yang lebih kecil untuk estimasi?

Prime Directive: Seorang developer menunjukkan kepada Anda bahwa *code* yang dia tulis sudah berjalan dengan baik dan tidak ada *error*. Namun, Anda menyadari bahwa *flow* tersebut tidak sesuai dengan *acceptance criteria* yang disepakati. Bagaimana Anda akan menangani situasi ini?

- *Refleksi:* Apakah saya berani menyuarakan perbedaan antara "berfungsi" dan "sesuai kebutuhan"?

Problem Solving: Anda menemukan *bug* yang *intermittent* (kadang muncul, kadang tidak). Langkah-langkah apa yang akan Anda lakukan untuk mencoba mereproduksinya dan mendapatkan informasi yang cukup untuk *bug report*?

- *Refleksi:* Apakah saya memiliki pendekatan sistematis untuk memecahkan masalah yang sulit?

Jika Anda merasa kesulitan menjawab beberapa pertanyaan, itu adalah hal yang wajar! Ini adalah kesempatan untuk kembali ke chapter-chapter terkait dan memperdalam pemahaman Anda. Jika Anda bisa menjawab sebagian besar dengan percaya diri, selamat! Anda sudah memiliki fondasi yang kuat untuk menjadi QA Engineer yang efektif.

Resources Tambahan: Belajar Tiada Henti

Perjalanan menjadi seorang Quality Assurance Engineer adalah proses belajar yang berkelanjutan. Teknologi terus berkembang, begitu juga dengan metodologi dan *best practice* dalam pengujian. Berikut adalah beberapa sumber daya tambahan yang dapat Anda jelajahi untuk terus mengasah kemampuan Anda:

1. Buku-buku Rekomendasi

Lessons Learned in Software Testing: A Context-Driven Approach oleh Cem Kaner, James Bach, Bret Pettichord

- Buku klasik yang memberikan wawasan mendalam tentang pengujian perangkat lunak dari perspektif praktis dan berbasis konteks. Sangat cocok untuk mengembangkan pola pikir pengujian Anda.

Agile Testing: A Practical Guide for Testers and Agile Teams oleh Lisa Crispin dan Janet Gregory

- Jika Anda bekerja di lingkungan Agile, buku ini adalah panduan wajib. Ini membahas bagaimana QA berintegrasi dengan tim Agile dan bagaimana pengujian dilakukan dalam iterasi pendek.

Clean Code: A Handbook of Agile Software Craftsmanship oleh Robert C. Martin

- Meskipun bukan buku QA secara langsung, memahami prinsip *clean code* akan membantu Anda berkomunikasi lebih baik dengan developer dan mengidentifikasi potensi masalah kualitas dari sudut pandang kode.

Software Testing: A Craftsman's Approach oleh Paul C. Jorgensen

- Buku teks yang lebih teknis, ideal jika Anda ingin mendalami teori di balik berbagai teknik pengujian.

2. Blog dan Sumber Daya Online

- **Ministry of Testing (ministryoftesting.com):** Salah satu komunitas pengujian terbesar di dunia. Menawarkan artikel, webinar, kursus, dan forum diskusi.
- **StickyMinds (stickyminds.com):** Sumber daya yang kaya dengan artikel, laporan, dan *webinar* tentang berbagai topik pengujian perangkat lunak.
- **The Test Automation University (testautomationu.com):** Platform gratis dari Applitools yang menawarkan kursus-kursus berkualitas tinggi tentang *test automation* dengan berbagai *tool* dan bahasa pemrograman.
- **Medium.com & Dev.to:** Cari penulis atau publikasi yang berfokus pada QA atau *software testing*. Banyak praktisi berbagi pengalaman dan tips mereka di sana. Contoh: Cari tag

#softwaretesting, #qa, #playwright.

3. Komunitas QA Indonesia

Bergabung dengan komunitas adalah cara terbaik untuk belajar dari pengalaman orang lain, bertanya, dan membangun jaringan. * **LinkedIn Groups**: Cari grup seperti "Software Quality Assurance Indonesia" atau "QA Engineers Indonesia". Ini adalah tempat yang bagus untuk berbagi lowongan, diskusi, dan tips. * **Telegram/WhatsApp Groups**: Seringkali ada grup-grup lokal atau regional yang dibentuk oleh para praktisi QA. Tanyakan kepada rekan-rekan atau cari di forum online. * **Meetup.com**: Cari *meetup* atau *event* terkait *software testing* atau teknologi di kota Anda. Ini adalah kesempatan bagus untuk bertemu langsung dengan para profesional dan belajar dari presentasi mereka. * **Konferensi dan Workshop**: Ikuti konferensi QA lokal atau internasional (misalnya, ISTQB, Agile Indonesia). Banyak yang menawarkan *workshop* praktis yang sangat bermanfaat.

Jangan ragu untuk aktif bertanya, berbagi, dan memberikan kontribusi dalam komunitas. Ingat, *learning is a two-way street!*

Next Step: Terapkan ke Project Nyata dalam 7 Hari

Anda telah menyerap banyak teori dan *best practice*. Sekarang, saatnya untuk mengaplikasikannya! Pengetahuan tanpa praktik hanyalah informasi. Tantang diri Anda untuk menerapkan setidaknya satu atau dua konsep yang telah Anda pelajari ke dalam proyek nyata dalam 7 hari ke depan.

Berikut adalah beberapa ide langkah konkret yang bisa Anda ambil:

Pilih Proyek Mini atau Aplikasi Sehari-hari:

- Pilih sebuah situs web yang sering Anda kunjungi (misalnya, situs e-commerce, portal berita, atau aplikasi *to-do list* pribadi Anda).
- Atau, jika Anda memiliki akses ke proyek *open-source* atau proyek pribadi (bahkan yang sangat sederhana), itu akan lebih baik.

Lakukan Requirement Analysis (Improvised):

- Pilih satu fitur kecil dari aplikasi/situs yang Anda pilih (misalnya, fitur pencarian, proses checkout, atau *submit* formulir kontak).

- Tuliskan *user story* dan *acceptance criteria* (AC) yang Anda bayangkan untuk fitur tersebut. Pikirkan: "Sebagai [tipe pengguna], saya ingin [melakukan sesuatu], agar [mendapatkan manfaat]." Dan "Diterima jika: [kondisi 1], [kondisi 2], dst."

- Identifikasi potensi ambiguitas atau *edge case* yang mungkin terlewat.

Desain Test Case Pertama Anda:

- Berdasarkan *user story* dan AC yang Anda buat, tuliskan minimal 5 *test case* menggunakan template yang telah kita bahas. Pastikan mencakup skenario positif, negatif, dan *edge case*.
- Contoh: Untuk fitur pencarian, *test case* bisa berupa: mencari dengan kata kunci valid, mencari dengan kata kunci tidak ada, mencari dengan karakter khusus, mencari dengan *input* kosong.

Eksekusi dan Laporkan Bug (Jika Ada):

- Jalankan *test case* yang Anda buat secara manual pada aplikasi/situs yang dipilih.
- Jika Anda menemukan *bug* (sesuatu yang tidak sesuai dengan *expected result* Anda atau *acceptance criteria*), tuliskan *bug report* menggunakan template yang telah dibahas. Berikan *severity* dan bayangkan bagaimana Anda akan mengeskalasikannya.
- Jangan khawatir jika Anda tidak menemukan *bug* — itu juga merupakan hasil!

Dokumentasikan Pengujian Anda:

- Setelah selesai, buat ringkasan singkat dari aktivitas pengujian Anda. Apa yang Anda uji? Berapa banyak *test case* yang *pass/fail*? Apa kesimpulan Anda tentang kualitas fitur tersebut?

Pelajari Otomatisasi (Opsional, tapi Sangat Direkomendasikan):

- Jika Anda merasa nyaman, coba instal Playwright seperti yang diajarkan di [Chapter 07: Automation Testing dengan Playwright].
- Coba tulis *test script* otomatis yang sangat sederhana, misalnya untuk membuka halaman login dan memverifikasi judul halaman. Ini adalah langkah kecil yang besar!

Ingat, setiap langkah kecil adalah kemajuan. Jangan takut membuat kesalahan; itulah cara terbaik untuk belajar. Konsistenlah dalam praktik, dan Anda akan melihat peningkatan signifikan dalam kemampuan QA Anda.

Rangkuman

Kita telah menempuh perjalanan yang panjang, mulai dari memahami peran fundamental QA hingga menyelami detail teknis *test automation*. Anda telah belajar bahwa Quality Assurance Engineer adalah jantung dari kualitas produk, bukan sekadar pelengkap. Anda adalah orang yang memastikan bahwa setiap baris kode, setiap fitur, dan setiap pengalaman pengguna memenuhi standar tertinggi dan selaras dengan kebutuhan bisnis serta ekspektasi pengguna.

Anda kini memahami: * Pentingnya **Requirement Analysis** sebagai gerbang kualitas pertama. * Seni **Test Design** untuk membangun strategi pengujian yang efektif. * Berbagai metode **Test Execution** untuk menemukan *bug* secepat mungkin. * Kekuatan **Bug Reporting** dan **Reporting & Documentation** untuk komunikasi yang jelas. * Nilai **Stakeholder Engagement** dalam membangun produk yang sukses. * Potensi **Automation Testing** untuk efisiensi dan cakupan yang lebih luas.

Buku ini adalah fondasi. Di atas fondasi ini, Anda memiliki kebebasan untuk terus membangun, belajar, dan berinovasi. Dunia QA adalah dinamis dan selalu menawarkan tantangan baru. Dengan pola pikir yang tepat, dedikasi untuk kualitas, dan semangat belajar yang tak pernah padam, Anda siap untuk menjadi Quality Assurance Engineer yang luar biasa.

Selamat berpetualang dalam karir QA Anda!

Checklist Actionable

Berikut adalah daftar tindakan yang bisa Anda ambil segera setelah menyelesaikan buku ini untuk mengkonsolidasikan pembelajaran Anda:

- [] **Unduh dan Adaptasi Template:** Dapatkan template Test Case, Bug Report, UAT Report, Release Notes, dan struktur Test Strategy. Sesuaikan dengan gaya dan kebutuhan Anda atau tim Anda.

- [] **Pilih Proyek Latihan:** Identifikasi sebuah aplikasi web atau fitur kecil yang akan Anda gunakan sebagai "laboratorium" praktik Anda dalam 7 hari ke depan.
- [] **Lakukan Requirement Analysis Mini:** Untuk fitur yang Anda pilih, coba tuliskan *user story* dan *acceptance criteria* Anda sendiri. Identifikasi potensi *edge case*.
- [] **Tulis Test Case Pertama Anda:** Buat setidaknya 5 *test case* (positif, negatif, *edge*, data) untuk fitur tersebut menggunakan template.
- [] **Eksekusi dan Laporkan:** Jalankan *test case* Anda secara manual. Jika menemukan *bug*, tuliskan *bug report* yang detail.
- [] **Coba Instal Playwright:** Jika Anda belum melakukannya, ikuti langkah-langkah di Chapter 07 untuk menginstal Playwright dan buat *test script* otomatis yang sangat sederhana (misalnya, membuka halaman dan memverifikasi elemen).
- [] **Bergabung dengan Komunitas QA:** Cari dan bergabunglah dengan setidaknya satu grup LinkedIn atau komunitas QA Indonesia lainnya. Perkenalan diri dan ikuti diskusi.
- [] **Jadwalkan Waktu Belajar Rutin:** Alokasikan 30-60 menit setiap hari (atau beberapa kali seminggu) untuk membaca artikel, menonton tutorial, atau mempraktikkan keterampilan QA.
- [] **Refleksikan Prime Directive:** Setiap kali Anda menguji sesuatu, tanyakan pada diri sendiri: "Apakah *flow* ini sudah sesuai dengan *criteria user story*?"

Lakukan tindakan-tindakan ini, dan Anda akan melihat bagaimana pengetahuan yang Anda dapatkan dari buku ini akan berubah menjadi keterampilan yang berharga. Sukses selalu untuk Anda, Quality Assurance Engineer!
